Verifiable Outsourced BTE with Silent Setup: Achieving Constant-Rate Batched Delegation

Kwangsu Lee*

Abstract

Batched threshold encryption (BTE) is a novel public-key paradigm in which, once a batch of B ciphertexts is designated, a decrypter evaluates them using decryption key shares generated by decryption committee nodes via threshold reconstruction. To mitigate Miner Extractable Value (MEV) attacks in fully decentralized environments like blockchains, it is essential to guarantee mempool privacy while supporting a silent setup that allows decentralized key generation among decryption nodes. Furthermore, to efficiently process large batches of ciphertexts, an outsourcing mechanism that delegates heavy decryption computations to a cloud server is indispensable. In this paper, we observe that naively adding decryption outsourcing to a threshold batched identity-based encryption with silent setup (TBIBE-SS) scheme of Gong et al. (EUROCRYPT 2026) causes the outsourcing communication overhead to scale undesirably with the number of decryption nodes. To overcome this limitation, we introduce a partial re-randomization technique–replacing conventional full re-randomization–and incorporate a handle-based oracle query mechanism into the security model. Based on these techniques, we propose a communication-efficient outsourced TBIBE-SS (O-TBIBE-SS) scheme and formally prove its security. We then combine our O-TBIBE-SS scheme with additional cryptographic primitives to construct a verifiable outsourced BTE-SS (VOBTE-SS) scheme, which supports both delegated decryption and verifiability of outsourced computation, proving its security under an augmented threat model. Our VOBTE-SS scheme represents the first construction to achieve efficient batched decryption outsourcing in terms of both communication and computation overhead within decentralized environments.

Keywords: Batched encryption, Batched Threshold encryption, Verifiable outsourcing, Silent setup.

*Sejong University, Seoul, Republic of Korea. Email: kwangsu@sejong.ac.kr.

1 Introduction

A Miner Extractable Value (MEV) attack is a prominent threat in blockchain networks where a greedy miner inspects unconfirmed transactions in the public mempool and manipulates their ordering within a block to extract financial gain prior to their execution at the application layer [9]. To mitigate MEV attacks in blockchain environments, Choudhuri et al. [7] introduced the concept of batched threshold encryption (BTE) and presented the first efficient BTE construction leveraging KZG polynomial commitments [20], where the size of the decryption key remains independent of the batch size. A BTE scheme is a public-key primitive that enables a decrypter to efficiently evaluate a designated set of B ciphertexts selected from a public transaction pool, utilizing decryption key shares generated by threshold committee nodes through a reconstruction process. By encrypting user transactions prior to their inclusion in a block and deferring their decryption until after block confirmation, BTE effectively prevents MEV attacks by concealing transaction content during ordering. A vital security property required for BTE is mempool privacy, which guarantees that encrypted transactions excluded from a confirmed block remain un-decryptable indefinitely.

Various approaches have been proposed to construct BTE schemes. Naive BTE constructions utilizing traditional threshold public-key encryption (TPKE) or threshold identity-based encryption (TIBE) suffer from notable drawbacks, such as excessive communication overhead between key generation servers and the decrypter, or an inability to guarantee mempool privacy [2, 11, 23]. To achieve communication efficiency while preserving mempool privacy, several BTE schemes leveraging cryptographic primitives such as KZG polynomial commitments and key-homomorphic pseudo-random functions (khPRF) have been proposed [1, 5, 7, 8]. However, these constructions exhibit inherent computational inefficiency, as evaluating a batch of B ciphertexts escalates the decryption overhead to approximately $O(B \log B)$. To resolve this limitation, Lee [22] introduced the concept of verifiable outsourced batched threshold encryption (VOBTE), delegating the heavy decryption operations of BTE to an untrusted cloud server while enabling verification of the outsourced results, and presented a secure and efficient VOBTE scheme. The core design of the VOBTE framework adapts the threshold batched identity-based encryption (TBIBE) scheme of Gong et al. [19] that provides mempool privacy into an outsourced variant (O-TBIBE), synthesizing it with additional cryptographic primitives to achieve CCA security, decryption outsourcing, and verifiability of outsourced computation.

For fully decentralized blockchain environments, supporting a silent setup–where N decryption nodes independently generate their own keys without relying on a trusted central authority–is essential. Gong et al. [19] proposed a TBIBE-SS scheme supporting silent setup and proved its security under a q-type assumption. To simultaneously accommodate silent setup and decryption outsourcing, one could naively transform the TBIBE-SS scheme into a simple O-TBIBE-SS scheme by applying Lee's re-randomization technique. However, for the batch size B, this simple O-TBIBE-SS variant causes the size of the transformed tasks generated by the cloud server to expand to $O(BN)$. This introduces a severe inefficiency, as the outsourcing communication overhead scales additionally with the number of decryption nodes N. Consequently, designing a BTE scheme that supports silent setup while achieving optimal outsourcing communication efficiency remains a critical, unresolved challenge.

1.1 Our Contribution

In this paper, we provide an affirmative answer to the unresolved challenge of supporting communication-efficient decryption outsourcing in BTE schemes with a silent setup. Our main contributions are summarized as follows:

First, we extend the conventional security framework of TBIBE-SS to formalize a realistic adversarial

model for O-TBIBE-SS that effectively captures fully malicious behavior. In the standard semi-malicious TBIBE-SS model, the adversary is restricted to submitting the key-generation randomness, which the challenger subsequently utilizes to construct the public keys and aggregation hints. To elevate this semi-malicious model to a fully malicious model, a conventional approach incorporates proofs of knowledge within the public keys and introduces a VerifyPubKey algorithm to filter out malformed keys and hints. Our O-TBIBE-SS model adapts this full-malicious model and integrates an augmented outsourcing oracle that accommodates handle-based oracle queries. This handle-based query mechanism plays a central role in enabling the security reduction and formal proof of our newly proposed outsourcing framework. Furthermore, we define a comprehensive security model against fully malicious adversaries by incorporating a robustness property, ensuring that the adversary cannot disrupt the correctness of decryption even when registering maliciously generated public keys and aggregation hints.

Second, we propose an efficient O-TBIBE-SS scheme that supports the delegation of decryption operations based on the baseline TBIBE-SS scheme. As previously noted, a naive adaptation of Lee's re-randomization technique for generating transformation keys causes the size of the transformed tasks produced by the cloud server to expand to $O(BN)$. To circumvent this structural bottleneck, we introduce a partial re-randomization method that refrains from full re-randomization; instead, it exclusively re-randomizes the group elements while isolating the exponents embedded within the decryption key shares generated by individual nodes. This approach restricts the size of the transformed tasks strictly to $O(B)$, thereby achieving an optimal constant-ratio outsourcing efficiency. However, this partial re-randomization creates a non-trivial algebraic correlation between the exponents of the decryption key shares and the delegated transformation keys, which introduces significant challenges in the security reduction. We overcome this hurdle by leveraging the handle-based query mechanism of our security model, which ensures that the intrinsic correlation between the actual protocol components is seamlessly preserved within the simulation. Finally, we formally prove that our proposed O-TBIBE-SS scheme is secure under the q-type hardness assumption and the soundness of the proofs of knowledge within the newly strengthened security framework.

Third, we define the conceptual framework of VOBTE-SS, which integrates silent setup, threshold decryption, and decryption outsourcing into batched encryption, and construct a secure and efficient VOBTE-SS scheme by combining our O-TBIBE-SS scheme with additional cryptographic components. The proposed VOBTE-SS framework simultaneously achieves silent setup, threshold decryption, chosen-ciphertext (CCA) security, batched decryption, verifiable outsourcing of decryption operations, and mempool privacy in blockchain environments. Inheriting the architectural advantages of the underlying O-TBIBE-SS scheme, our VOBTE-SS implementation preserves the optimal constant-ratio outsourcing efficiency. Furthermore, we formally demonstrate that the proposed VOBTE-SS scheme guarantees message hiding under static corruption and public key registration environments, verifiability of outsourced decryption operations, and robust execution even in the presence of corruption queries and malicious public key registrations.

1.2 Related Work

Driven by the growing interest in blockchain technology [6, 24], research on threshold cryptography–including threshold encryption and threshold signatures–has been actively pursued. The traditional paradigm for converting standard public-key encryption and signature schemes into threshold variants employs distributed key generation (DKG) protocols to decentralize the key generation process [18], followed by non-interactive reconstruction of key shares or signature shares via threshold secret sharing. However, this approach exhibits the drawback that the communication overhead of DKG protocols scales with the number of decryption or signing nodes N. Recently, cryptographic protocols supporting a silent setup have been actively investigated, enabling the construction of threshold encryption and signature schemes without relying

on DKG protocols [13–16, 26].

The initial application of threshold cryptography in blockchain environments focused on utilizing threshold signatures to enhance the security of individual users' signing keys [17,21]. As modern blockchains have evolved beyond simple transaction processing to execute automated computations via smart contracts [6], various unprecedented issues have emerged. A representative example is the MEV attack [9], where a greedy miner analyzes transaction details in the public mempool and deliberately manipulates the transaction order or block inclusion timing to extract financial gains from the transaction information. To mitigate MEV attacks, various approaches employing threshold encryption have been proposed to encrypt transactions beforehand and subsequently decrypt them after block commitment [2, 10, 11, 23].

An initial approach straightforwardly employs TPKE schemes [2, 25]. However, this method exhibits a drawback where the communication overhead with decryption nodes scales with the batch size B, demanding $O(BN)$ communication. An alternative approach utilizes TIBE schemes to generate decryption key shares for a specific block identity and combines them to perform batch decryption [4, 11, 23]. While this method achieves an efficient communication overhead of $O(N)$, it suffers from a critical security flaw: once the decryption key for a given block is exposed, all encrypted transactions associated with that block are decrypted. Consequently, ciphertexts that were not selected for inclusion in the block are also decrypted, failing to guarantee mempool privacy. To resolve the mempool privacy issue, Choudhuri et al. [7] introduced the concept of BTE and proposed a KZG commitment based BTE scheme, which additionally requires multi-party computation (MPC) protocols, proving its security in the Generic Group Model (GGM). Bormet et al. [5] proposed an epoch-free BTE scheme by leveraging a time-lock puzzle (TLP) mechanism based on khPRFs, which additionally requires coordination among ciphertext indices. To simultaneously provide communication efficiency and mempool privacy, batched IBE (BIBE) schemes have been proposed [1, 8]. These BIBE schemes utilize KZG commitments to fix the selected subset of ciphertexts included in a block, and eliminate the need for complex MPC protocols by issuing decryption keys combined with BLS threshold signatures in each epoch.

While existing BTE schemes are efficient, their security has been analyzed primarily in the GGM. To establish security in the standard model rather than the GGM, Gong et al. [19] proposed a BIBE scheme, a TBIBE scheme supporting threshold decryption, and a TBIBE-SS scheme supporting silent threshold setup, proving their security under q-type assumptions. Subsequently, to reduce the decryption computational overhead in BTE schemes, Lee [22] proposed O-BIBE and O-TBIBE schemes that delegate decryption operations to an untrusted cloud server and presented a VOBE scheme and a threshold-supported VOBTE scheme that simultaneously achieve CCA security for ciphertexts and verifiability of the outsourced results, and formally proved their security.

2 Preliminaries

In this section, we begin by defining bilinear groups and the associated complexity assumptions. Following the algebraic foundations, we formally introduce the underlying cryptographic primitives: proofs of knowledge, cryptographic hash functions, pseudo-random functions, one-time signatures, and symmetric key encryption.

2.1 Notations

Let $\lambda \in \mathbb{N}$ denote the security parameter. A function $f(\lambda)$ is said to be negligible, denoted as $f(\lambda) = \mathsf{negl}(\lambda)$, if for every positive polynomial $p(\lambda)$ there exists an integer λ_0 such that $|f(\lambda)| < 1/p(\lambda)$ for all

$\lambda > \lambda_0$. We use the abbreviation PPT to denote probabilistic polynomial-time.

For a finite set S, $s \leftarrow S$ represents the operation of sampling an element s uniformly at random from S, and $|S|$ denotes its cardinality. For a positive integer B, we define $[B] := \{1, 2, \ldots, B\}$ and $[0, B] := \{0, 1, \ldots, B\}$. The string $\{0,1\}^\lambda$ represents the set of all binary strings of length λ. An algorithmic assignment $y \leftarrow \mathsf{Alg}(x)$ outputs y by running a deterministic or probabilistic algorithm Alg on input x. If Alg is probabilistic, the output is a random variable determined by the internal random coins of Alg. We use $\perp$ to denote a special failure or error symbol returned by an algorithm.

2.2 Bilinear Groups

Definition 2.1 (Prime-Order Asymmetric Bilinear Group)**.** An asymmetric bilinear group generator $\mathcal{G}$ takes as input the security parameter λ and outputs the description of a bilinear group $\mathsf{GD} = (p, \mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T, e, g, \hat{g})$ where p is a random prime, $\mathbb{G}, \hat{\mathbb{G}}$, and $\mathbb{G}_T$ are three cyclic groups of prime order p, and g and $\hat{g}$ are generators of $\mathbb{G}$ and $\hat{\mathbb{G}}$, respectively. The bilinear map $e : \mathbb{G} \times \hat{\mathbb{G}} \rightarrow \mathbb{G}_T$ has the following properties:

1. Bilinearity: $\forall u \in \mathbb{G}, \forall \hat{v} \in \hat{\mathbb{G}}$ and $\forall a, b \in \mathbb{Z}_p$, $e(u^a, \hat{v}^b) = e(u, \hat{v})^{ab}$.
2. Non-degeneracy: $\exists g \in \mathbb{G}, \hat{g} \in \hat{\mathbb{G}}$ such that $e(g, \hat{g})$ has order p in $\mathbb{G}_T$.

We say that $\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T$ are asymmetric bilinear groups with no efficiently computable isomorphisms if the group operations in $\mathbb{G}, \hat{\mathbb{G}}$, and $\mathbb{G}_T$ as well as the bilinear map e are all efficiently computable, but there are no efficiently computable isomorphisms between $\mathbb{G}$ and $\hat{\mathbb{G}}$.

Assumption 1 ((B,N)-Bilinear Diffie-Hellman Exponent Variant [19])**.** Let $\mathsf{GD} = (p, \mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T, e, g, \hat{g})$ be an asymmetric bilinear group generated by $\mathcal{G}(1^\lambda)$. The (B,N)-bilinear Diffie-Hellman exponent variant ((B,N)-BDHEV) assumption is that if the challenge tuple

$$D = \Big(\mathsf{GD},\ \hat{g}^a,\ \hat{g}^b,\ g^{ab},\ \hat{g}^{ab},\ g^{bc^{N+1}},\ \hat{g}^{bc^{N+1}},\ g^s,\ g^\tau,\ \left\{\hat{g}^{\tau^i}\right\}_{i\in[0,B]},$$
$$\left\{g^{\tau^i c^j},\ \hat{g}^{\tau^i c^j}\right\}_{i\in[B], j\in[2N]},\ \left\{\hat{g}^{\tau^i abc^j}\right\}_{i\in[0,B], j\in[2N]\setminus\{N+1\}},\ \left\{\hat{g}^{\tau^i abc^{N+1}}\right\}_{i\in[B]},$$
$$\left\{\hat{g}^{c^j s},\ \hat{g}^{\tau c^j s},\ \hat{g}^{\tau abc^j s}\right\}_{j\in[N]}\Big) \text{ and } Z$$

are given, no PPT algorithm $\mathcal{A}$ can distinguish $Z = Z_0 = e(g,\hat{g})^{abc^{N+1}s}$ from $Z = Z_1 = e(g,\hat{g})^d$ with more than a negligible advantage. The advantage of $\mathcal{A}$ is defined as $\mathsf{Adv}_{\mathcal{A}}^{(B,N)\text{-BDHEV}}(\lambda) = |\Pr[\mathcal{A}(D, Z_0) = 0] - \Pr[\mathcal{A}(D, Z_1) = 0]|$ where the probability is taken over random choices of $\tau, a, b, c, s, d \in \mathbb{Z}_p$.

2.3 Non-Interactive Proofs of Knowledge with Straight-Line Extraction

Definition 2.2 (Non-Interactive Proofs of Knowledge)**.** Let $R \subseteq \mathcal{X} \times \mathcal{W}$ be a relation where $\mathcal{X}$ is the statement space and $\mathcal{W}$ is the witness space. A non-interactive proof of knowledge (PoK) for the relation R consists of three algorithms:

$\mathsf{Setup}(1^\lambda, \mathsf{GD}, G_1, G_2) \rightarrow \mathsf{CRS}$: The setup algorithm takes the security parameter λ, a group description GD, and two base generators G_1, G_2 as inputs and outputs a common reference string CRS.

$\mathsf{Prove}(\mathsf{CRS}, X, W) \rightarrow \psi$: The proving algorithm takes the string CRS, a statement $X \in \mathcal{X}$, and a witness $W \in \mathcal{W}$ such that $(X, W) \in R$ as inputs, and outputs a proof ψ.

$\mathsf{Verify}(\mathsf{CRS}, X, \psi) \to \{0,1\}$: The verification algorithm takes the string CRS, a statement X, and a proof ψ as inputs, and outputs 1 (accept) or 0 (reject).

We require PoK to satisfy the standard properties of perfect completeness and straight-line (online) extractability defined as follows:

- Perfect Completeness: For every $\lambda \in \mathbb{N}$ and any $(X,W) \in R$, it holds that

$$\Pr\left[\mathsf{Verify}(\mathsf{CRS}, X, \psi) = 1 \;\middle|\; \mathsf{CRS} \leftarrow \mathsf{Setup}(1^\lambda, \mathsf{GD}, G_1, G_2);\ \psi \leftarrow \mathsf{Prove}(\mathsf{CRS}, X, W) \right] = 1.$$

- Straight-Line Extractability: There exists an efficient simulator $\mathsf{Extract}$ that runs in a single execution trace (without rewinding). For any adversary $\mathcal{A}$ that outputs a valid statement-proof pair (X, ψ) such that $\mathsf{Verify}(\mathsf{CRS}, X, \psi) = 1$, the extraction algorithm outputs a witness W' using a cryptographic trapdoor ξ embedded in $\mathsf{CRS}_{\mathsf{ext}} \leftarrow \mathsf{Setup}_{\mathsf{ext}}(1^\lambda, \mathsf{GD}, G_1, G_2)$ such that:

$$\Pr\left[(X,W') \in R \;\middle|\; (X,\psi) \leftarrow \mathcal{A}(\mathsf{CRS}_{\mathsf{ext}});\ W' \leftarrow \mathsf{Extract}(\mathsf{CRS}_{\mathsf{ext}}, \xi, X, \psi) \right] \geq 1 - \mathsf{negl}(\lambda),$$

  where $\mathsf{CRS}_{\mathsf{ext}}$ is computationally indistinguishable from a standard CRS.

Concrete Relation for Key Registration. We specifically instantiate PoK over a bilinear group setting $\mathsf{GD} = (p, \mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T, e, g, \hat{g})$ by using the PoK scheme of Fischlin [12]. Let the public key elements be the statement $X = (X_1, X_2) \in \mathbb{G}_T \times \mathbb{G}$, and the secret exponents be the witness $W = (\alpha, w) \in \mathbb{Z}_p \times \mathbb{Z}_p$. The underlying relation R evaluated by the sub-protocol is explicitly defined as

$$R = \left\{ \big((X_1, X_2), (\alpha, w)\big) \;\middle|\; X_1 = G_1^\alpha \ \wedge\ X_2 = G_2^w \right\}.$$

2.4 Collision-Resistant Hash Function

Definition 2.3 (Collision-Resistant Hash Function)**.** A family of hash functions (HF) is a collection of efficiently computable functions $\mathcal{H} = \{\mathsf{H}_s : \mathcal{X} \to \mathcal{Y}\}_{s \in \mathcal{K}}$ where $\mathcal{K}$ is the key space, $\mathcal{X}$ is the domain, and $\mathcal{Y}$ is the range. The collision-resistance of an HF is defined as the following experiment $\mathsf{Exp}^{\mathsf{CR}}_{HF,\mathcal{A}}(\lambda)$:

1. A hash function H_s is sampled from $\mathcal{H}$ uniformly at random.
2. The adversary $\mathcal{A}$ is given the hash function H_s and outputs a pair of inputs $(x, x') \in \mathcal{X}^2$.
3. The experiment returns 1 if $x \neq x'$ and $\mathsf{H}_s(x) = \mathsf{H}_s(x')$. Otherwise, it returns 0.

The advantage of $\mathcal{A}$ in the collision resistance experiment is defined as $\mathsf{Adv}^{\mathsf{CR}}_{HF,\mathcal{A}}(\lambda) = \Pr[\mathsf{Exp}^{\mathsf{CR}}_{HF,\mathcal{A}}(\lambda) = 1]$. We say that an HF is collision-resistant if this advantage is negligible for all PPT adversaries.

2.5 Pseudo-Random Function

Definition 2.4 (Pseudo-Random Function)**.** A pseudo-random function (PRF) is an efficiently computable function $\mathsf{F} : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$, where $\mathcal{K}$ is the key space, $\mathcal{X}$ is the domain, and $\mathcal{Y}$ is the range. Let $\mathcal{F}_{all} = \{f : \mathcal{X} \to \mathcal{Y}\}$ be the set of all functions mapping $\mathcal{X}$ to $\mathcal{Y}$. We say that F satisfies pseudo-randomness if for all PPT adversaries $\mathcal{A}$, the advantage

$$\mathsf{Adv}^{\mathsf{PR}}_{PRF,\mathcal{A}}(\lambda) = \left| \Pr[k \xleftarrow{\$} \mathcal{K} : \mathcal{A}^{\mathsf{F}(k,\cdot)}(1^\lambda) = 1] - \Pr[f \xleftarrow{\$} \mathcal{F}_{all} : \mathcal{A}^{f(\cdot)}(1^\lambda) = 1] \right|$$

is negligible, where the first probability is over the uniform choice of $k \in \mathcal{K}$ and the second is over the uniform choice of $f \in \mathcal{F}_{all}$, and both include the internal coins of $\mathcal{A}$.

2.6 One-Time Signature

Definition 2.5 (One-Time Signature)**.** A one-time signature (OTS) scheme with message space of $\mathcal{M}$ consists of the following three algorithms:

$\mathsf{KeyGen}(1^\lambda) \to (\mathsf{SK}, \mathsf{VK})$: The key generation algorithm takes as input a security parameter λ. It outputs a signing key SK and a verification key VK.

$\mathsf{Sign}(\mathsf{SK}, M) \to \sigma$: The signing algorithm takes as input the signing key SK and a message $M \in \mathcal{M}$. It outputs a signature σ.

$\mathsf{Verify}(\mathsf{VK}, M, \sigma) \to \{0,1\}$: The (deterministic) verification algorithm takes as input the verification key VK, a message $M \in \mathcal{M}$, and a signature σ. It outputs 1 if the signature is valid and 0 otherwise.

The correctness of an OTS scheme is defined as follows: For all $(\mathsf{SK}, \mathsf{VK}) \leftarrow \mathsf{KeyGen}(1^\lambda)$ and any message $M \in \mathcal{M}$, it is required that $\mathsf{Verify}(\mathsf{VK}, M, \mathsf{Sign}(\mathsf{SK}, M)) = 1$.

Definition 2.6 (Strong Unforgeability)**.** The strong unforgeability (SUF) of an OTS scheme is defined in via the following experiment $\mathsf{Exp}^{\mathsf{SUF}}_{OTS,\mathcal{A}}(\lambda)$ between a challenger $\mathcal{C}$ and a PPT adversary $\mathcal{A}$:

1. $\mathcal{C}$ first generates a key pair $(\mathsf{SK}, \mathsf{VK}) \leftarrow \mathsf{KeyGen}(1^\lambda)$ and gives VK to $\mathcal{A}$.
2. $\mathcal{A}$ requests at most one signature query on a message M. $\mathcal{C}$ generates a signature $\sigma \leftarrow \mathsf{Sign}(\mathsf{SK}, M)$ and gives σ to $\mathcal{A}$.
3. Finally, $\mathcal{A}$ outputs a forged pair (M^*, σ^*). The experiment returns 1 if the forged pair satisfies the following conditions, and returns 0 otherwise: 1) $\mathsf{Verify}(\mathsf{VK}, M^*, \sigma^*) = 1$, 2) $(M^*, \sigma^*) \neq (M, \sigma)$ where (M, σ) is the pair of the signature query.

The advantage of $\mathcal{A}$ is defined as $\mathsf{Adv}^{\mathsf{SUF}}_{OTS,\mathcal{A}}(\lambda) = \Pr[\mathsf{Exp}^{\mathsf{SUF}}_{OTS,\mathcal{A}}(\lambda) = 1]$ where the probability is taken over all internal randomness of the experiment. An OTS scheme is said to be SUF secure if the advantage is negligible for all PPT adversaries.

2.7 Symmetric Key Encryption

Definition 2.7 (Symmetric Key Encryption)**.** A symmetric key encryption (SKE) scheme consists of the following three algorithms:

$\mathsf{KeyGen}(1^\lambda) \to K$: The key generation algorithm takes as input a security parameter λ. It outputs a symmetric key $K \in \{0,1\}^\lambda$.

$\mathsf{Encrypt}(K, M) \to \mathsf{CT}$: The encryption algorithm takes as input a symmetric key K and a message M. It outputs a ciphertext CT.

$\mathsf{Decrypt}(K, \mathsf{CT}) \to M$: The decryption algorithm takes as input a symmetric key K and a ciphertext CT. It outputs a message M.

The correctness of an SKE scheme is defined as follows: For all $K \leftarrow \mathsf{KeyGen}(1^\lambda)$ and any message M, it is required that $\mathsf{Decrypt}(K, \mathsf{Encrypt}(K, M)) = M$.

Definition 2.8 (Indistinguishability under One-Time Security)**.** The indistinguishability under one-time security (IND-OT) of a SKE scheme is defined via the following experiment $\mathsf{Exp}^{\mathsf{IND\text{-}OT}}_{SKE,\mathcal{A}}(\lambda)$ between a challenger $\mathcal{C}$ and a PPT adversary $\mathcal{A}$:

1. $\mathcal{C}$ generates a symmetric key $K \leftarrow \mathsf{KeyGen}(1^\lambda)$.
2. $\mathcal{A}$ submits two challenge messages M_0^*, M_1^* of equal length ($|M_0^*| = |M_1^*|$). $\mathcal{C}$ flips a random coin $\mu \in \{0,1\}$, computes the challenge ciphertext $\mathsf{CT}^* \leftarrow \mathsf{Encrypt}(K, M_\mu^*)$, and gives CT^* to $\mathcal{A}$.
3. Finally, $\mathcal{A}$ outputs a guess $\mu' \in \{0,1\}$. The experiment returns 1 if $\mu = \mu'$ and 0 otherwise.

The advantage of $\mathcal{A}$ in the IND-OT experiment is defined as $\mathsf{Adv}^{\mathsf{IND\text{-}OT}}_{SKE,\mathcal{A}}(\lambda) = \left|\Pr[\mathsf{Exp}^{\mathsf{IND\text{-}OT}}_{SKE,\mathcal{A}}(\lambda) = 1] - \frac{1}{2}\right|$ where the probability is taken over all internal randomness of the experiment. An SKE scheme is said to be IND-OT secure if the advantage is negligible for all PPT adversaries.

3 Outsourced TBIBE with Silent Setup

In this section, we formalize the syntax and the security models for the O-TBIBE-SS scheme. We then present an efficient O-TBIBE-SS construction designed to eliminate the performance bottleneck and provide security proof of the proposed scheme under cryptographic hardness assumptions.

3.1 Definition

The O-TBIBE-SS scheme extends the foundational TBIBE-SS framework of Gong et al. [19], which enables the batch decryption of multiple ciphertexts via threshold reconstruction from decryption key shares provided by distributed decryption nodes, by allowing a client to outsource a significant portion of the decryption workload to a cloud server.

In the O-TBIBE-SS scheme, each decryption node independently generates its own secret key, public key, and hint utilizing trusted public parameters. An aggregator then collects these public keys and hints to transparently generate and publish an encryption key and an aggregated public key. Individual users utilize the encryption key to generate ciphertexts associated with their identity ID and a batch label BL, subsequently uploading them to a ciphertext pool. The decryptor determines a set of B ciphertexts to be decrypted, computes a digest DIG for the identity set S comprising the B target ciphertexts, and each decryption node generates a decryption key share DK_j. Thereafter, using the shares DK_j provided by the nodes, the decryptor can sequentially decrypt the ciphertexts if $\mathsf{ID} \in S$ to derive the respective capsule keys. If a client entity executing the decryption intends to delegate the batch decryption of B ciphertexts to a cloud server, it executes the delegation algorithm to generate a recovery key, a transformation key, and work tasks, and then transmits the transformation key and work tasks to the cloud. Utilizing the transformation key, the cloud server processes the received B work tasks to generate transformed tasks, which is then returned to the decryptor client. Finally, the decryption client uses the recovery key to seamlessly derive the B capsule keys from the transformed tasks. We formalize the syntax and interfaces of these outsourcing-related algorithms by strictly adhering to the syntax established by Lee [22] in the definition of O-BIBE schemes.

Guided by this high-level operational flow, we formally define the syntax of the O-TBIBE-SS scheme as follows:

Definition 3.1 (O-TBIBE-SS)**.** An outsourced TBIBE with silent setup (O-TBIBE-SS) scheme for a maximum batch size $B \in \mathbb{N}$, identity space of $\mathcal{I}$, batch label space of $\mathcal{L}$, and capsule key space of $\mathcal{K}$ consists of the following eight algorithms:

$\mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T) \to \mathsf{PP}$: The setup algorithm takes as input the security parameter λ, the maximum batch size B, the size of the decryption committee N, and the threshold T such that $T \leq N$. It outputs the public parameters PP.

$\mathsf{KeyGen}(\mathsf{PP}, j) \to (\mathsf{SK}_j, \mathsf{PK}_j, \mathsf{HT}_j)$: The key generation algorithm takes as input the public parameters PP and an index $j \in [N]$. It outputs a secret key SK_j, a public key PK_j, and an aggregation hint HT_j.

$\mathsf{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j, \mathsf{HT}_j) \to \{0,1\}$: The public key verification algorithm takes as input the public parameters PP, a public key PK_j, and an aggregation hint HT_j. It outputs a bit $b \in \{0,1\}$, where $b = 1$ indicates that the public key and the aggregation hint are validly structured, and $b = 0$ otherwise.

$\mathsf{Preprocess}(\mathsf{PP}, (\mathsf{HT}_j)_{j\in[N]}) \to (\mathsf{EK}, \mathsf{PK})$: The preprocessing algorithm takes as input the public parameters PP, aggregation hints $(\mathsf{HT}_j)_{j\in[N]}$. It outputs a public encryption key EK and an aggregated public key PK.

$\mathsf{Encaps}(\mathsf{EK}, \mathsf{ID}, \mathsf{BL}) \to (\mathsf{CH}, \mathsf{CK})$: The encapsulation algorithm takes as input the public encryption key EK, an identity $\mathsf{ID} \in \mathcal{I}$, and a batch label $\mathsf{BL} \in \mathcal{L}$. It outputs a ciphertext header CH and a capsule key $\mathsf{CK} \in \mathcal{K}$.

$\mathsf{Digest}(\mathsf{PP}, S) \to \mathsf{DIG}$: The digest algorithm takes as input the public parameters PP and a set of identities $S \subseteq \mathcal{I}$. It outputs a digest DIG associated with S.

$\mathsf{ComputeKeyShare}(\mathsf{SK}_j, \mathsf{DIG}, \mathsf{BL}) \to \mathsf{DK}_j$: The key share computation algorithm takes as input the secret key SK_j, a digest DIG, and a batch label BL. It outputs a decryption key share DK_j.

$\mathsf{VerifyKeyShare}(\mathsf{PK}_j, \mathsf{DK}_j, \mathsf{DIG}, \mathsf{BL}) \to \{0,1\}$: The key share verification algorithm takes as input the public key PK_j, the decryption key share DK_j, a digest DIG, and a batch label BL. It outputs $b \in \{0,1\}$, where $b = 1$ indicates that the decryption key share is validly structured, and $b = 0$ otherwise.

$\mathsf{Decaps}(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, S, \mathsf{CH}, \mathsf{ID}, \mathsf{BL}) \to \mathsf{CK}$: The decapsulation algorithm takes as input the public key PK, decryption key shares $(\mathsf{DK}_j)_{j\in A}$ where $A \subseteq [N]$ with $|A| = T$, a set of identities S, a ciphertext header CH, an identity ID, and a batch label BL. It outputs the corresponding capsule key $\mathsf{CK} \in \mathcal{K}$.

$\mathsf{Delegate}(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, (\mathsf{CH}_i)_{i\in U}, \mathsf{BL}) \to (\mathsf{RK}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in U})$: The delegation algorithm takes as input the public key PK, decryption key shares $(\mathsf{DK}_j)_{j\in A}$ where $A \subseteq [N]$ with $|A| = T$, ciphertext headers $(\mathsf{CH}_i)_{i\in U}$ where $U \subseteq [B]$, and a batch label BL. It outputs a recovery key RK, a transformation key TK, and work tasks $(\mathsf{WT}_i)_{i\in U}$.

$\mathsf{Transform}(\mathsf{PK}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in U}, (\mathsf{ID}_i)_{i\in U}, \mathsf{BL}) \to (\mathsf{TT}_i)_{i\in U}$: The transformation algorithm takes as input the public key PK, a transformation key TK, work tasks $(\mathsf{WT}_i)_{i\in U}$ where $U \subseteq [B]$, identities $(\mathsf{ID}_i)_{i\in U}$, and a batch label BL. It outputs transformed tasks $(\mathsf{TT}_i)_{i\in U}$.

$\mathsf{Recover}(\mathsf{RK}, (\mathsf{TT}_i)_{i\in U}) \to (\mathsf{CK}_i)_{i\in U}$: The recovery algorithm takes as input a recovery key RK and transformed tasks $(\mathsf{TT}_i)_{i\in U}$ where $U \subseteq [B]$. It outputs a sequence of capsule keys $(\mathsf{CK}_i)_{i\in U} \in \mathcal{K}^{|U|}$.

Definition 3.2 (Correctness of O-TBIBE-SS)**.** An O-TBIBE-SS scheme is correct if for any security parameter λ and any batch size $B \leq \mathrm{poly}(\lambda)$, the following four conditions hold:

- Correctness of Key Share: For any size N, any threshold $T \leq N$, any set $S \subseteq \mathcal{I}$, and any batch label $\mathsf{BL} \in \mathcal{L}$,

$$\Pr\left[\begin{array}{l|l} \mathsf{VerifyKeyShare}(\mathsf{PK}_j, \mathsf{DK}_j, \mathsf{DIG}, \mathsf{BL}) & \mathsf{PP} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T) \\ = 1 & \forall j \in [N] : (\mathsf{SK}_j, \mathsf{PK}_j, \mathsf{HT}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j) \\ & (\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}(\mathsf{PP}, (\mathsf{HT}_j)_{j\in[N]}) \\ & \mathsf{DIG} \leftarrow \mathsf{Digest}(\mathsf{PP}, S) \\ & \mathsf{DK}_j \leftarrow \mathsf{ComputeKeyShare}(\mathsf{SK}_j, \mathsf{DIG}, \mathsf{BL}) \end{array}\right] = 1.$$

- Correctness of Decryption: For any size N, any threshold $T \leq N$, any identity $\mathsf{ID} \in \mathcal{I}$, any batch label $\mathsf{BL} \in \mathcal{L}$, any set $S \subseteq \mathcal{I}$ such that $\mathsf{ID} \in S$, and any subset $A \subseteq [N]$ such that $|A| = T$,

$$\Pr\left[\begin{array}{l|l} \mathsf{Decaps}(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, S, \mathsf{CH}, \mathsf{ID}, \mathsf{BL}) & \mathsf{PP} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T) \\ = \mathsf{CK} & \forall j \in [N] : (\mathsf{SK}_j, \mathsf{PK}_j, \mathsf{HT}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j) \\ & (\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}(\mathsf{PP}, (\mathsf{HT}_j)_{j\in[N]}) \\ & (\mathsf{CH}, \mathsf{CK}) \leftarrow \mathsf{Encaps}(\mathsf{EK}, \mathsf{ID}, \mathsf{BL}) \\ & \mathsf{DIG} \leftarrow \mathsf{Digest}(\mathsf{PP}, S) \\ & \forall j \in A : \mathsf{DK}_j \leftarrow \mathsf{ComputeKeyShare}(\mathsf{SK}_j, \mathsf{DIG}, \mathsf{BL}) \end{array}\right] = 1.$$

- Correctness of Public Key: For any size N, any threshold $T \leq N$, and any $j \in [N]$,

$$\Pr\left[\mathsf{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j, \mathsf{HT}_j) = 1 \;\middle|\; \begin{array}{l} \mathsf{PP} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T) \\ (\mathsf{SK}_j, \mathsf{PK}_j, \mathsf{HT}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j) \end{array}\right] = 1.$$

- Correctness of Outsourced Batch Decryption: For any size N, any threshold $T \leq N$, any $S = \{\mathsf{ID}_i\}_{i\in[B]} \subseteq \mathcal{I}^B$, any subset $A \subseteq [N]$ such that $|A| = T$, and $\mathsf{BL} \in \mathcal{L}$,

$$\Pr\left[\begin{array}{l|l} \mathsf{Recover}(\mathsf{RK}, (\mathsf{TT}_i)_{i\in[B]}) & \mathsf{PP} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T) \\ = (\mathsf{CK}_i)_{i\in[B]} & \forall j \in [N] : (\mathsf{SK}_j, \mathsf{PK}_j, \mathsf{HT}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j) \\ & (\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}(\mathsf{PP}, (\mathsf{HT}_j)_{j\in[N]}) \\ & \forall i \in [B] : (\mathsf{CH}_i, \mathsf{CK}_i) \leftarrow \mathsf{Encaps}(\mathsf{EK}, \mathsf{ID}_i, \mathsf{BL}) \\ & \mathsf{DIG} \leftarrow \mathsf{Digest}(\mathsf{PP}, S) \\ & \forall j \in A : \mathsf{DK}_j \leftarrow \mathsf{ComputeKeyShare}(\mathsf{SK}_j, \mathsf{DIG}, \mathsf{BL}) \\ & (\mathsf{RK}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in[B]}) \leftarrow \mathsf{Delegate}(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, (\mathsf{CH}_i)_{i\in[B]}, \mathsf{BL}) \\ & (\mathsf{TT}_i)_{i\in[B]} \leftarrow \mathsf{Transform}(\mathsf{PK}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in[B]}, (\mathsf{ID}_i)_{i\in[B]}, \mathsf{BL}) \end{array}\right] = 1.$$

Definition 3.3 (Outsourcing Rate)**.** Let $\mathsf{Size}_{\mathrm{in}}$ denote the total size of the input batch ciphertexts designated for outsourcing, and let $\mathsf{Size}_{\mathrm{out}}$ denote the total size of the data transmitted between the client and the cloud to execute the outsourcing protocol. The outsourcing rate $\mathcal{R}_{\mathrm{out}}(B)$ of an outsourcing scheme is defined as $\mathcal{R}_{\mathrm{out}}(B) = \mathsf{Size}_{\mathrm{out}}/\mathsf{Size}_{\mathrm{in}}$. We say that the outsourcing scheme achieves *constant-rate* outsourcing efficiency if $\mathcal{R}_{\mathrm{out}}(B)$ is asymptotically bounded by a constant c as the batch size B approaches infinity.

In the O-TBIBE-SS scheme, the input and output sizes, $\mathsf{Size}_{\mathrm{in}}$ and $\mathsf{Size}_{\mathrm{out}}$, are concretely evaluated based on the specific objects as follows:

$$\mathsf{Size}_{\mathrm{in}} = \sum_{i=1}^{B} \mathsf{Size}(\mathsf{CH}_i),\ \mathsf{Size}_{\mathrm{out}} = \mathsf{Size}(\mathsf{TK}) + \sum_{i=1}^{B} \mathsf{Size}(\mathsf{WT}_i) + \sum_{i=1}^{B} \mathsf{Size}(\mathsf{TT}_i)$$

where CH_i is the i-th ciphertext header in the batch, WT_i is the corresponding work task, TK is the transformation key, and TT_i is the transformed task.

We formalize the security model of the O-TBIBE-SS scheme by extending the foundational TBIBE-SS security model defined by Gong et al. [19] to additionally accommodate malicious nodes and outsourcing oracles. In the selective CPA security with static corrupted-and-malicious nodes (SE-SCM-CPA) model, an adversary first commits to a target identity-label pair $(\mathsf{ID}^*, \mathsf{BL}^*)$, the number of decryption nodes N with the threshold T, and the subsets of corrupted and malicious nodes. Following this commitment, the adversary is provided with the corrupted secret keys, and the non-malicious public keys and hints. In the pre-challenge queries phase, the adversary is permitted to adaptively query decryption key shares via the $\mathsf{OCreateKeyShare}$ and $\mathsf{ORevealKeyShare}$ oracles. In the challenge phase, the adversary submits the public keys and hints of the malicious nodes, and in return receives a target challenge ciphertext header CH^* along with a challenge capsule key CK^*_μ, which is distributed either as the valid capsule key or as a uniformly sampled random string. During the post-challenge queries phase, the adversary can continue to invoke the prior oracles while additionally executing outsourcing queries on arbitrary ciphertexts via the $\mathsf{ODelegate}$ oracle. Finally, the adversary outputs a guess bit μ' for the challenge random bit μ, winning the game if the guess is correct. To preclude trivial winning strategies, explicit restrictions are imposed on the adversary's oracle queries.

A more rigorous definition of the SE-SCM-CPA security model for the O-TBIBE-SS scheme is detailed as follows:

Definition 3.4 (Selective CPA Security with Static Corrupted-and-Malicious Nodes)**.** The selective CPA security with static corrupted-and-malicious nodes (SE-SCM-CPA) of O-TBIBE-SS is defined via the following experiment $\mathsf{Exp}^{\mathsf{SE\text{-}SCM\text{-}CPA}}_{\mathit{O\text{-}TBIBE\text{-}SS},\mathcal{A}}(\lambda, B)$ between a challenger $\mathcal{C}$ and a PPT adversary $\mathcal{A}$ where λ and B are given as input:

1. **Init**: $\mathcal{A}$ commits to a challenge identity-label pair $(\mathsf{ID}^*, \mathsf{BL}^*)$, the decryption committee size N, the threshold $T \leq N$, and the disjoint subsets of corrupted and malicious committee nodes $C_{\text{cor}}, C_{\text{mal}} \subseteq [N]$ such that $|C_{\text{cor}} \cup C_{\text{mal}}| < T$. For notational simplicity, we define $C_{\text{all}} = C_{\text{cor}} \cup C_{\text{mal}}$.

2. **Setup**: $\mathcal{C}$ runs $\mathsf{PP} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T)$ and invokes the following initialization oracle:

   - $\mathsf{OInit}()$: It initializes $L_{\text{sk}} \leftarrow \emptyset$, $L_{\text{dks}} \leftarrow \emptyset$, and $L_{\text{del}} \leftarrow \emptyset$, which are used to track committee keys, decryption key shares, and delegation queries, respectively.

   Next, for all nodes $j \in [N] \setminus C_{\text{mal}}$, it generates $(\mathsf{SK}_j, \mathsf{PK}_j, \mathsf{HT}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j)$ and updates the key list $L_{\text{sk}} \leftarrow L_{\text{sk}} \cup \{(j, \mathsf{SK}_j, \mathsf{PK}_j, \mathsf{HT}_j)\}$. It gives $\big(\mathsf{PP}, (\mathsf{SK}_j)_{j \in C_{\text{cor}}}, (\mathsf{PK}_j)_{j \in [N] \setminus C_{\text{mal}}}, (\mathsf{HT}_j)_{j \in [N] \setminus C_{\text{mal}}}\big)$ to $\mathcal{A}$.

3. **Pre-challenge Queries**: $\mathcal{A}$ adaptively requests $\mathsf{OCreateKeyShare}$ and $\mathsf{ORevealKeyShare}$ queries. $\mathcal{C}$ handles these queries as follows:

   - $\mathsf{OCreateKeyShare}(j, S, \mathsf{BL})$: If $j \notin [N]$ or $j \in C_{\text{all}}$, it responds with $\perp$. Otherwise, if there exists $(j, S, \mathsf{BL}, \dots) \in L_{\text{dks}}$, it retrieves $(j, \cdot, \cdot, \mathsf{hk}, \cdot) \leftarrow L_{\text{dks}}$ associated with (j, S, BL) and responds with hk. Otherwise:
     (a) It computes $\mathsf{DIG} \leftarrow \mathsf{Digest}(\mathsf{PP}, S)$ and an honest key share $\mathsf{DK}_j \leftarrow \mathsf{ComputeKeyShare}(\mathsf{SK}_j, \mathsf{DIG}, \mathsf{BL})$ by retrieving $\mathsf{SK}_j \leftarrow L_{\text{sk}}$.
     (b) It picks a unique handle $\mathsf{hk} \in \{0,1\}^\lambda$, updates $L_{\text{dks}} \leftarrow L_{\text{dks}} \cup \{(j, S, \mathsf{BL}, \mathsf{hk}, \mathsf{DK}_j)\}$, and responds with hk.
   - $\mathsf{ORevealKeyShare}(\mathsf{hk})$: If there is no entry $(\cdot, \cdot, \cdot, \mathsf{hk}, \dots) \in L_{\text{dks}}$, it responds with $\perp$. Otherwise, it retrieves $(j, S, \mathsf{BL}, \mathsf{hk}, \mathsf{DK}_j) \leftarrow L_{\text{dks}}$ associated with hk, and responds with DK_j.

4. **Challenge**: $\mathcal{A}$ submits the public keys and hints $(\mathsf{PK}_j, \mathsf{HT}_j)_{j \in C_{\mathrm{mal}}}$ of the malicious nodes. $\mathcal{C}$ proceeds the following steps:

   (a) It checks that $1 = \mathsf{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j, \mathsf{HT}_j)$ for each $j \in C_{\mathrm{mal}}$. If any check fails, it outputs 0 and aborts the experiment. Otherwise, it updates $L_{\mathrm{sk}} \leftarrow L_{\mathrm{sk}} \cup \{(j, \perp, \mathsf{PK}_j, \mathsf{HT}_j)\}_{j \in C_{\mathrm{mal}}}$ and proceeds.

   (b) It collects all hints from L_{sk} and runs $(\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}\big(\mathsf{PP}, (\mathsf{HT}_j)_{j \in [N]}\big)$.

   (c) It computes a challenge ciphertext pair $(\mathsf{CH}^*, \mathsf{CK}^*) \leftarrow \mathsf{Encaps}(\mathsf{EK}, \mathsf{ID}^*, \mathsf{BL}^*)$. It sets $\mathsf{CK}^*_0 = \mathsf{CK}^*$ and $\mathsf{CK}^*_1 \xleftarrow{\$} \mathcal{K}$. It flips a random coin $\mu \in \{0,1\}$ and gives $(\mathsf{CH}^*, \mathsf{CK}^*_\mu)$ to $\mathcal{A}$.

5. **Post-challenge Queries**: $\mathcal{A}$ continues to make the oracle queries permitted in the pre-challenge queries, and additionally requests $\mathsf{ODelegate}$ and $\mathsf{ORevealRK}$ queries. $\mathcal{C}$ handles these queries as follows:

   - $\mathsf{OCreateKeyShare}(j, S, \mathsf{BL})$: This oracle operates identically to the pre-challenge queries.
   - $\mathsf{ORevealKeyShare}(\mathsf{hk})$: This oracle operates identically to the pre-challenge queries.
   - $\mathsf{ODelegate}((\mathsf{hk}_j)_{j \in A \setminus C_{\mathrm{all}}}, (\mathsf{DK}_j)_{j \in A \cap C_{\mathrm{all}}}, (\mathsf{CH}_i)_{i \in U})$: Let $(\mathsf{hk}_j)_{j \in A \setminus C_{\mathrm{all}}}$ be handles of honest key shares and $(\mathsf{DK}_j)_{j \in A \cap C_{\mathrm{all}}}$ be dishonest key shares where $A = (A \setminus C_{\mathrm{all}}) \cup (A \cap C_{\mathrm{all}}) \subseteq [N]$ and $|A| = T$. If there is no entry $(j, \cdot, \cdot, \mathsf{hk}_j, \cdot) \in L_{\mathrm{dks}}$ for each $j \in A \setminus C_{\mathrm{all}}$, it responds with $\perp$. Otherwise:
     (a) For each honest node $j \in A \setminus C_{\mathrm{all}}$, it retrieves $(j, S_j, \mathsf{BL}_j, \mathsf{hk}_j, \mathsf{DK}_j) \leftarrow L_{\mathrm{dks}}$. Next, it checks that $(\mathsf{DK}_j)_{j \in A \setminus C_{\mathrm{all}}}$ are associated with the same (S, BL). If any check fails, it responds with $\perp$.
     (b) For each dishonest node $j \in A \cap C_{\mathrm{all}}$, it checks that $1 = \mathsf{VerifyKeyShare}(\mathsf{PK}_j, \mathsf{DK}_j, \mathsf{DIG}, \mathsf{BL})$ by retrieving $\mathsf{PK}_j \leftarrow L_{\mathrm{sk}}$ where $\mathsf{DIG} \leftarrow \mathsf{Digest}(\mathsf{PP}, S)$. If any check fails, it responds with $\perp$.
     (c) It computes $(\mathsf{RK}, \mathsf{TK}, (\mathsf{WT}_i)_{i \in U}) \leftarrow \mathsf{Delegate}\big(\mathsf{PK}, (\mathsf{DK}_j)_{j \in A}, (\mathsf{CH}_i)_{i \in U}, \mathsf{BL}\big)$.
     (d) It picks a unique handle $\mathsf{hd} \in \{0,1\}^\lambda$, updates $L_{\mathrm{del}} \leftarrow L_{\mathrm{del}} \cup \{(\mathsf{hd}, \mathsf{RK})\}$, and responds with $(\mathsf{hd}, \mathsf{TK}, (\mathsf{WT}_i)_{i \in U})$.
   - $\mathsf{ORevealRK}(\mathsf{hd})$: If there is no entry $(\mathsf{hd}, \cdot) \in L_{\mathrm{del}}$, it responds with $\perp$. Otherwise, it retrieves $(\mathsf{hd}, \mathsf{RK}) \leftarrow L_{\mathrm{del}}$ and responds with RK.

6. **Output**: Finally, $\mathcal{A}$ outputs a guess $\mu' \in \{0,1\}$. The experiment returns 1 if $\mu' = \mu$ and the following oracle constraints are satisfied:

   (a) **(Challenge Privacy)**: For the challenge label BL^* and any set S such that $\mathsf{ID}^* \in S$, the adversary $\mathcal{A}$ is restricted as follows:
   - $\mathcal{A}$ never queries $\mathsf{ORevealKeyShare}(\mathsf{hk})$ for the key handle hk associated with (j, S, BL^*).
   - $\mathcal{A}$ never queries $\mathsf{ORevealRK}(\mathsf{hd})$ for any delegation handle hd associated with (S, BL^*).

   (b) **(Label Coherence)**: For the challenge label BL^*, the adversary $\mathcal{A}$ is restricted as follows:
   - $\mathcal{A}$ makes at most one $\mathsf{OCreateKeyShare}(j, S_j, \mathsf{BL}^*)$ query on a set S_j for each honest node $j \in [N] \setminus C_{\mathrm{all}}$.
   - $\mathcal{A}$ makes at most one $\mathsf{ODelegate}((\mathsf{hk}_j)_{j \in A \setminus C_{\mathrm{all}}}, \cdot, \cdot)$ query where all honest key-share handles $(\mathsf{hk}_j)_{j \in A \setminus C_{\mathrm{all}}}$ are originating from the identical (S, BL^*) on a set S. Additionally, if $\mathsf{ID}^* \notin S$, $\mathcal{A}$ is allowed to query $\mathsf{ORevealRK}(\mathsf{hd})$ to retrieve RK associated with (S, BL^*).

Otherwise, it returns 0.

The advantage of $\mathcal{A}$ is defined as $\mathsf{Adv}^{\mathsf{SE\text{-}SCM\text{-}CPA}}_{\textit{O-TBIBE-SS},\mathcal{A}}(\lambda) = \left|\Pr\left[\mathsf{Exp}^{\mathsf{SE\text{-}SCM\text{-}CPA}}_{\textit{O-TBIBE-SS},\mathcal{A}}(\lambda, B) = 1\right] - \frac{1}{2}\right|$ where the probability is taken over all internal randomness of the experiment. An O-TBIBE-SS scheme is said to be SE-SCM-CPA secure if the advantage is negligible for all PPT adversaries.

Remark 1 (On the Strict Oracle Restriction). The strict restriction on querying $\mathsf{ORevealKeyShare}$ for the challenge session (S, BL^*) such that $\mathsf{ID}^* \in S$ does not realistically diminish the capabilities of an adversary under the static corruption model, as noted in [19]. If the adversary wishes to obtain a specific decryption key share DK_j for an honest node $j \in [N] \setminus C_{\text{all}}$, it can strategically include the target node j within the static corrupted nodes C_{cor} at the beginning of the experiment. By doing so, the adversary acquires the secret key SK_j and can compute the decryption key share DK_j on its own.

Remark 2 (On the Rationality of Label Coherence). We clarify the rationale behind the extended Label Coherence constraints regarding the $\mathsf{ODelegate}$ oracle under the challenge label BL^*, demonstrating that our security model serves as a conservative extension of the baseline TBIBE-SS model in [19]. If the adversary $\mathcal{A}$ does not invoke the $\mathsf{ODelegate}$ oracle for BL^*, $\mathcal{A}$ maintains the complete freedom to query $\mathsf{OCreateKeyShare}(j, S_j, \mathsf{BL}^*)$ with mutually different sets S_j for each honest node $j \in [N] \setminus C_{\text{all}}$. This execution path is strictly equivalent to the TBIBE-SS security model, introducing no artificial restrictions on the underlying KEM layer. Conversely, if $\mathcal{A}$ chooses to utilize the outsourcing by querying $\mathsf{ODelegate}$ under BL^*, a structural alignment becomes necessary. To successfully execute the outsourcing, the input honest key-share handles (hk_j) must be bound to a single target set S. Consequently, the preceding $\mathsf{OCreateKeyShare}$ queries for all participating honest nodes must be restricted to at most a single identical pair of (S, BL^*).

Remark 3 (Relation to the Baseline Security Model). The proposed SE-SCM-CPA security model for O-TBIBE-SS is a natural and strict extension of the baseline static security model defined for the TBIBE-SS scheme in [19]. The primary distinction lies in strengthening the security guarantee from a semi-malicious setup to a fully malicious setup: instead of requiring the adversary $\mathcal{A}$ to submit the key generation randomness for malicious nodes, our model allows $\mathcal{A}$ to submit only the public keys and associated hints (or proofs of knowledge) of malicious nodes. In addition, our model introduces the outsourcing oracles, namely $\mathsf{ODelegate}$ and $\mathsf{ORevealRK}$. Furthermore, while the baseline model utilizes a single combined oracle to compute and reveal a key share, our model refines this process into a two-stage mechanism via $\mathsf{OCreateKeyShare}$ and $\mathsf{ORevealKeyShare}$ using handles.

We formalize the robustness model of the O-TBIBE-SS scheme to guarantee that an adversary cannot disrupt the correctness of batch decryption, even when registering malicious public keys and hints for malicious nodes or submitting forged decryption key shares of dishonest nodes. In the robustness model against malicious key attacks, the adversary first commits to the total number of decryption nodes N and the threshold size T, and subsequently receives the public parameters. During the pre-challenge queries phase, the adversary is permitted to invoke node key generation, node key corruption, and node key registration queries via the $\mathsf{OCreateKey}$, $\mathsf{OCorruptKey}$, and $\mathsf{ORegisterKey}$ oracles, respectively. In the challenge phase, the adversary submits a target identity ID^* and a target batch label BL^*, and in return receives a target challenge ciphertext header CH^* along with a target capsule key CK^*. In the post-challenge queries phase, the adversary can continue to query decryption key shares for honest nodes via the $\mathsf{OComputeKeyShare}$ oracle. Finally, in the output phase, the adversary submits a target identity set S^*, a target authorized node subset A^*, and dishonest key shares $\{\mathsf{DK}_j\}$. The adversary wins the robustness game if the challenge ciphertext decryption, executed by combining the honest key shares with the submitted dishonest key shares, yields a decrypted valuation different from the challenge capsule key CK^*.

A more rigorous formulation of the robustness model against malicious keys for the O-TBIBE-SS scheme is detailed as follows:

Definition 3.5 (Robustness against Malicious Key Attacks)**.** The robustness against malicious key attacks (ROB-MKA) of O-TBIBE-SS is defined via the following experiment $\mathsf{Exp}^{\mathsf{ROB\text{-}MKA}}_{\mathit{O\text{-}TBIBE\text{-}SS},\mathcal{A}}(\lambda, B)$ between a challenger $\mathcal{C}$ and a PPT adversary $\mathcal{A}$, given inputs λ and B:

1. **Init**: $\mathcal{A}$ commits to the decryption committee size N and the threshold $T \leq N$.
2. **Setup**: $\mathcal{C}$ runs $\mathsf{PP} \leftarrow \mathsf{Setup}(1^{\lambda}, 1^{B}, 1^{N}, 1^{T})$ and invokes the following initialization oracle:
   - $\mathsf{OInit}()$: It initializes $L_{\mathrm{sk}} \leftarrow \emptyset$, $Q_{\mathrm{hon}} \leftarrow \emptyset$, $Q_{\mathrm{cor}} \leftarrow \emptyset$, and $Q_{\mathrm{mal}} \leftarrow \emptyset$, which are used to track secret keys, honest nodes, corrupted nodes, and malicious nodes, respectively. For notational simplicity, we define $Q_{\mathrm{all}} = Q_{\mathrm{hon}} \cup Q_{\mathrm{cor}} \cup Q_{\mathrm{mal}}$.

   Next, it provides PP to $\mathcal{A}$.
3. **Pre-challenge Queries**: $\mathcal{A}$ adaptively requests $\mathsf{OCreateKey}$, $\mathsf{OCorruptKey}$, and $\mathsf{ORegisterKey}$ queries. $\mathcal{C}$ handles these queries as follows:
   - $\mathsf{OCreateKey}(j)$: If $j \notin [N]$ or $j \in Q_{\mathrm{all}}$, it responds with $\perp$. Otherwise, it generates $(\mathsf{SK}_j, \mathsf{PK}_j, \mathsf{HT}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j)$, updates $L_{\mathrm{sk}} \leftarrow L_{\mathrm{sk}} \cup \{(j, \mathsf{SK}_j, \mathsf{PK}_j, \mathsf{HT}_j)\}$, adds j to Q_{hon}, and responds with $(\mathsf{PK}_j, \mathsf{HT}_j)$.
   - $\mathsf{OCorruptKey}(j)$: If $j \notin [N]$, $j \notin Q_{\mathrm{hon}}$, or $|Q_{\mathrm{cor}} \cup Q_{\mathrm{mal}}| \geq T - 1$, it responds with $\perp$. Otherwise, it retrieves $(j, \mathsf{SK}_j, \cdot, \cdot) \leftarrow L_{\mathrm{sk}}$, moves j from Q_{hon} to Q_{cor}, and responds with SK_j.
   - $\mathsf{ORegisterKey}(j, \mathsf{PK}_j, \mathsf{HT}_j)$: If $j \notin [N]$, $j \in Q_{\mathrm{all}}$, or $|Q_{\mathrm{cor}} \cup Q_{\mathrm{mal}}| \geq T - 1$, it responds with $\perp$. Otherwise, it verifies that $1 = \mathsf{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j, \mathsf{HT}_j)$. If the check passes, it updates $L_{\mathrm{sk}} \leftarrow L_{\mathrm{sk}} \cup \{(j, \perp, \mathsf{PK}_j, \mathsf{HT}_j)\}$, adds j to Q_{mal}, and responds with 1. Otherwise, it responds with 0.

   It is required that $\mathcal{A}$ must saturate the committee nodes by ensuring $Q_{\mathrm{all}} = [N]$ before moving to the next phase. If this condition is not met, the experiment returns 0 and aborts.
4. **Challenge**: $\mathcal{A}$ submits a challenge identity-label pair $(\mathsf{ID}^*, \mathsf{BL}^*)$. $\mathcal{C}$ proceeds as follows:
   (a) It collects all hints from L_{sk} and runs $(\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}\big(\mathsf{PP}, (\mathsf{HT}_j)_{j \in [N]}\big)$.
   (b) It computes a challenge ciphertext pair $(\mathsf{CH}^*, \mathsf{CK}^*) \leftarrow \mathsf{Encaps}(\mathsf{EK}, \mathsf{ID}^*, \mathsf{BL}^*)$, and gives $(\mathsf{CH}^*, \mathsf{CK}^*)$ to $\mathcal{A}$.
5. **Post-challenge Queries**: $\mathcal{A}$ adaptively requests $\mathsf{OComputeKeyShare}$ queries. $\mathcal{C}$ handles the queries as follows:
   - $\mathsf{OComputeKeyShare}(j, S, \mathsf{BL})$: If $j \notin Q_{\mathrm{hon}}$, it responds with $\perp$. It derives $\mathsf{DIG} \leftarrow \mathsf{Digest}(\mathsf{PP}, S)$. It retrieves $(j, \mathsf{SK}_j, \cdot, \cdot) \leftarrow L_{\mathrm{sk}}$, computes $\mathsf{DK}_j \leftarrow \mathsf{ComputeKeyShare}(\mathsf{SK}_j, \mathsf{DIG}, \mathsf{BL})$, and responds with DK_j.
6. **Output**: Finally, $\mathcal{A}$ outputs
$$\big(A^*,\ (\mathsf{DK}^*_j)_{j \in A^*_{\mathrm{dis}}},\ S^*\big)$$
where $A^* \subseteq [N]$ is an authorized subset such that $|A^*| = T$ where $A^*_{\mathrm{hon}} = A^* \cap Q_{\mathrm{hon}}$ and $A^*_{\mathrm{dis}} = A^* \cap (Q_{\mathrm{cor}} \cup Q_{\mathrm{mal}})$, $(\mathsf{DK}^*_j)_{j \in A^*_{\mathrm{dis}}}$ is a sequence of dishonest key shares, and S^* is an identity set such that $\mathsf{ID}^* \in S^*$. Next, $\mathcal{C}$ proceeds with the following steps:

(a) It derives $\mathsf{DIG}^* \leftarrow \mathsf{Digest}(\mathsf{PP}, S^*)$. It checks that $1 = \mathsf{VerifyKeyShare}(\mathsf{PK}_j, \mathsf{DK}_j^*, \mathsf{DIG}^*, \mathsf{BL}^*)$ for each $j \in A_{\text{dis}}^*$. If any check fails, it returns 0.

(b) It retrieves $(j, \mathsf{SK}_j, \cdot, \cdot) \leftarrow L_{\text{sk}}$ and computes the key share $\mathsf{DK}_j^* \leftarrow \mathsf{ComputeKeyShare}(\mathsf{SK}_j, \mathsf{DIG}^*, \mathsf{BL}^*)$ for each honest node $j \in A_{\text{hon}}^*$.

(c) It recovers the session key $\mathsf{CK}' \leftarrow \mathsf{Decaps}(\mathsf{PK}, (\mathsf{DK}_j^*)_{j \in A^*}, \mathsf{CH}^*, \mathsf{ID}^*, \mathsf{BL}^*)$.

The experiment returns 1 if $A_{\text{dis}}^* \neq \emptyset$ and $\mathsf{CK}' \neq \mathsf{CK}^*$. Otherwise, it returns 0.

The advantage of $\mathcal{A}$ is defined as $\mathsf{Adv}^{\text{ROB-MKA}}_{\textit{O-TBIBE-SS},\mathcal{A}}(\lambda) = \Pr\left[\mathsf{Exp}^{\text{ROB-MKA}}_{\textit{O-TBIBE-SS},\mathcal{A}}(\lambda, B) = 1\right]$ where the probability is taken over all internal randomness of the experiment. An O-TBIBE-SS scheme is said to be ROB-MKA secure if this advantage is negligible for all PPT adversaries.

Remark 4 (On the Minimality of Oracles for Robustness). We explicitly note that the robustness experiment does not provide the ODelegate and OReveaIRK oracles to the adversary. This omission is justified because the adversary can already access the full, unblinded honest decryption key shares $(\mathsf{DK}_j)_{j \in Q_{\text{hon}}}$ via the OComputeKeyShare oracle. Since the re-randomized transformation key (TK) and the recovery key (RK) are bounded by the entropy of decryption key shares, providing these outsourced derivatives yields no additional advantage to the adversary. Consequently, focusing solely on OComputeKeyShare establishes a strictly stronger robustness guarantee.

3.2 O-TBIBE-SS Construction

Our proposed O-TBIBE-SS scheme is based upon the TBIBE-SS scheme of Gong et al. [19] that supports the silent setup. To achieve our target architecture, we introduce several pivotal modifications to the baseline framework. First, we incorporate a VerifyPubKey algorithm to enable the explicit verification of the public keys and hints independently generated by each decryption node. Second, to facilitate the seamless modular integration of our O-TBIBE-SS scheme into higher-level cryptographic schemes, we transition the underlying architecture from a standard public-key encryption structure to a key encapsulation mechanism (KEM) framework that outputs a capsule key. Finally, to accommodate outsourced decryption operations, we expand the original TBIBE-SS scheme by introducing three outsourcing-dedicated algorithms: Delegate, Transform, and Recover.

The starting idea for the outsourced decryption originates from the complete re-randomization methodology utilized by Lee [22] in the context of the O-BIBE scheme, which generates the transformation key (TK) by entirely re-randomizing the decryption key DK. However, this complete re-randomization approach suffers from a structural inefficiency: the cloud server executing the outsourced decryption cannot eliminate the random exponents utilized in the re-randomization process, inherently forcing the output of the Transform procedure to scale up to $O(NB)$. To resolve this performance bottleneck, we propose a partial re-randomization mechanism. Specifically, given decryption key shares $\{\mathsf{DK}_j = (V_{j,1}, V_{j,2}, y_j)\}$, we restrict the re-randomization strictly to the two group elements $V_{j,1}$ and $V_{j,2}$, while leaving the exponent y_j unaltered. This partial re-randomization successfully compresses the transformed task size to $O(B)$, rendering it completely independent of the number of decryption nodes N. However, because the resulting TK retains an algebraic dependency on the underlying decryption key shares, establishing the security reduction becomes non-trivial, requiring a highly sophisticated simulation strategy. During the recover phase, the outsourcing client can seamlessly derive the correct capsule key by decrypting the transformed task via its local recovery key. Crucially, due to the presence of the randomness embedded within the TK, it is computationally infeasible for the malicious cloud server to extract any leakage regarding the underlying capsule key from the transformed tasks.

Following the structural overview, we now present the rigorous algorithmic specifications of the proposed O-TBIBE-SS scheme as follows:

O-TBIBE-SS.Setup$(1^\lambda, 1^B, 1^N, 1^T)$**:** It first samples an asymmetric bilinear group $\mathsf{GD} = (p, \mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T, e, g, \hat{g}) \leftarrow \mathcal{G}(1^\lambda)$. It selects random exponents $\tau, c, \beta, u', h' \in \mathbb{Z}_p$ and sets $u = g^{u'}, h = g^{h'}, \hat{u} = \hat{g}^{u'}, \hat{h} = \hat{g}^{h'}$. Let $\mathbf{M} \in \mathbb{Z}_p^{N \times T}$ be the share-generation matrix for a T-out-of-N threshold policy over $\mathbb{Z}_p$. It samples a random vector $\mathbf{b} \in \mathbb{Z}_p^N$ where $b_1 = \beta$. For each $j \in [N]$, let $\mathbf{m}_j^\top$ be the jth row of $\mathbf{M}$. Then, it computes

$$\hat{g}^{\gamma_0} = \prod_{j \in [N]} \hat{g}^{c^j \mathbf{m}_j^\top \mathbf{b}}, \quad \forall j \in [N] : \hat{g}^{v_{j,0}} = \prod_{k \in [N], k \neq j} \hat{g}^{c^{N+1-j+k} \mathbf{m}_j^\top \mathbf{b}}.$$

It generates $\mathsf{CRS} \leftarrow \mathsf{PoK.Setup}(1^\lambda, \mathsf{GD}, e(g,\hat{g})^{c^{N+1}}, g)$ and outputs the public parameters

$$\mathsf{PP} = \Big(\mathsf{GD},\ g^\tau,\ \{\hat{g}^{\tau^i}\}_{i \in [B]},\ \{g^{\tau^i c^j},\ \hat{g}^{\tau^i c^j}\}_{i \in [0,B], j \in [2N]},\ \hat{g}^{\gamma_0},\ \{\hat{g}^{v_{j,0}}\}_{j \in [N]},$$
$$u,\ h,\ \hat{u},\ \hat{h},\ \Omega = e(g,\hat{g})^{c^{N+1}\beta},\ \mathsf{CRS}\Big).$$

O-TBIBE-SS.KeyGen(PP, j)**:** It samples random exponents $\alpha_j, w_j \in \mathbb{Z}_p$. It creates an NIZK proof $\psi_j \leftarrow \mathsf{PoK.Prove}(\mathsf{CRS}, (e(g,\hat{g})^{c^{N+1}\alpha_j}, g^{w_j}), (\alpha_j, w_j))$. It outputs the secret key, the public key, and the aggregation hint as

$$\mathsf{SK}_j = \Big(\mathsf{GD},\ \alpha_j,\ w_j,\ \hat{u},\ \hat{h},\ \hat{g}^{c^{N+1}\alpha_j}\Big),\ \mathsf{PK}_j = \Big(\mathsf{GD},\ g^{w_j},\ u,\ h,\ e(g,\hat{g})^{c^{N+1}\alpha_j},\ \psi_j\Big),$$
$$\mathsf{HT}_j = \Big(\Big\{A_{j,k} = \big(\hat{g}^{c^k}\big)^{\alpha_j},\ W_{j,i,k} = \big(\hat{g}^{\tau^i c^k}\big)^{w_j}\Big\}_{i \in [0,B], k \in [2N] \setminus \{N+1\}}\Big).$$

O-TBIBE-SS.VerifyPubKey$(\mathsf{PP}, \mathsf{PK}_j, \mathsf{HT}_j)$**:** Let $\mathsf{PK}_j = (\mathsf{GD}, g^{w_j}, u, h, e(g,\hat{g})^{c^{N+1}\alpha_j}, \psi_j)$ and $\mathsf{HT}_j = (\{A_{j,k}, W_{j,i,k}\}_{i \in [0,B], k \in [2N] \setminus \{N+1\}})$. It checks that $1 \stackrel{?}{=} \mathsf{PoK.Verify}(\mathsf{CRS}, (e(g,\hat{g})^{c^{N+1}\alpha_j}, g^{w_j}), \psi_j)$. It checks that $u, h \in \mathsf{PK}_j$ are equal to the elements in PP. It checks the following equations

$$e(g,\hat{g})^{c^{N+1}\alpha_j} \stackrel{?}{=} e\big(g^{c^{N+1-k}}, A_{j,k}\big) \quad \forall k \in [2N] \setminus \{N+1\} \quad \text{and}$$
$$e\big(g, W_{j,i,k}\big) \stackrel{?}{=} e\big(g^{w_j}, \hat{g}^{\tau^i c^k}\big) \quad \forall i \in [0,B],\ k \in [2N] \setminus \{N+1\}.$$

It outputs 1 if all checks pass and 0 otherwise.

O-TBIBE-SS.Preprocess$(\mathsf{PP}, (\mathsf{HT}_j)_{j \in [N]})$**:** Let $\mathsf{HT}_j = (\{A_{j,k}, W_{j,i,k}\}_{i \in [0,B], k \in [2N] \setminus \{N+1\}})$. It computes the aggregated components

$$\hat{g}^\gamma = \hat{g}^{\gamma_0} \prod_{j \in [N]} A_{j,j} = \hat{g}^{\gamma_0} \prod_{j \in [N]} \hat{g}^{c^j \alpha_j},\ \hat{g}^w = \prod_{j \in [N]} W_{j,0,j} = \prod_{j \in [N]} \hat{g}^{c^j w_j}, \quad \text{and}$$
$$\hat{g}^{\tau w} = \prod_{j \in [N]} W_{j,1,j} = \prod_{j \in [N]} \hat{g}^{\tau c^j w_j}.$$

For each $i \in [0,B]$ and $j \in [N]$, it computes the aggregated cross terms

$$A_j = \hat{g}^{v_j} = \hat{g}^{v_{j,0}} \prod_{k \in [N], k \neq j} A_{k,N+1-j+k} = \hat{g}^{v_{j,0}} \prod_{k \in [N], k \neq j} \hat{g}^{c^{N+1-j+k}\alpha_k} \quad \text{and}$$
$$W_{i,j} = \hat{g}^{\tau^i d_j} = \prod_{k \in [N], k \neq j} W_{i,k,N+1-j+k} = \prod_{k \in [N], k \neq j} \hat{g}^{\tau^i c^{N+1-j+k} w_k}.$$

It outputs the public encryption key and the aggregated public key as

$$\mathsf{EK} = \Big(\mathsf{GD},\ \hat{g}^{w},\ \hat{g}^{\tau w},\ \hat{g}^{\gamma},\ u,\ h,\ \Omega\Big),$$
$$\mathsf{PK} = \Big(\mathsf{GD}, \big\{g^{c^j},\ g^{\tau^i c^j},\ A_j = \hat{g}^{v_j},\ W_{i,j} = \hat{g}^{\tau^i d_j}\big\}_{i\in[0,B],j\in[N]},\ \hat{u},\ \hat{h}\Big).$$

O-TBIBE-SS.Encaps$(\mathsf{EK},\mathsf{ID},\mathsf{BL})$**:** Let $\mathsf{EK} = (\mathsf{GD},\hat{g}^{w},\hat{g}^{\tau w},\hat{g}^{\gamma},u,h,\Omega)$. It samples a random exponent $t \in \mathbb{Z}_p$ and outputs a ciphertext header

$$\mathsf{CH} = \Big(C_1 = g^t,\ C_2 = \big(\hat{g}^{\tau w}\big)^t\big(\hat{g}^{w}\big)^{-\mathsf{ID}t},\ C_3 = \big(u^{\mathsf{BL}}h\big)^t,\ C_4 = \big(\hat{g}^{\gamma}\big)^t\Big)$$

and a capsule key $\mathsf{CK} = \Omega^t$.

O-TBIBE-SS.Digest(PP,S)**:** Let $S = \{\mathsf{ID}_i\}_{i\in U}$ where $U \subseteq [B]$. It defines the polynomial $F_S(x) = \prod_{\mathsf{ID}_i\in S}(x-\mathsf{ID}_i) = \sum_{k\in[0,|S|]} f_k x^k$ and outputs

$$\mathsf{DIG} = \prod_{k\in[0,|S|]}\big(\hat{g}^{\tau^k c^{N+1}}\big)^{f_k} = \hat{g}^{F_S(\tau)\cdot c^{N+1}}.$$

O-TBIBE-SS.ComputeKeyShare$(\mathsf{SK}_j,\mathsf{DIG},\mathsf{BL})$**:** Let $\mathsf{SK}_j = \big(\mathsf{GD},\alpha_j,w_j,\hat{u},\hat{h},\hat{g}^{c^{N+1}\alpha_j}\big)$. It samples $y_j \in \mathbb{Z}_p^*$ and $r_j \in \mathbb{Z}_p$ and outputs a decryption key share

$$\mathsf{DK}_j = \Big(K_{j,1} = \hat{g}^{c^{N+1}\alpha_j}\big(\mathsf{DIG}\big)^{w_j y_j}\big(\hat{u}^{\mathsf{BL}}\hat{h}\big)^{r_j},\ K_{j,2} = \hat{g}^{r_j},\ y_j\Big).$$

O-TBIBE-SS.VerifyKeyShare$(\mathsf{PK}_j,\mathsf{DK}_j,\mathsf{DIG},\mathsf{BL})$**:** Let $\mathsf{PK}_j = (\mathsf{GD},g^{w_j},u,h,e(g,\hat{g})^{c^{N+1}\alpha_j})$ and $\mathsf{DK}_j = (K_{j,1},K_{j,2},y_j)$. It outputs 1 if the equation holds

$$e(g,\hat{g})^{c^{N+1}\alpha_j} \stackrel{?}{=} e(g,K_{j,1})\cdot e(g^{w_j},\mathsf{DIG})^{-y_j}\cdot e((u^{\mathsf{BL}}h),K_{j,2})^{-1}$$

and 0 otherwise.

O-TBIBE-SS.Decaps$(\mathsf{PK},(\mathsf{DK}_j)_{j\in A},S,\mathsf{CH},\mathsf{ID},\mathsf{BL})$**:** Let $\mathsf{DK}_j = (K_{j,1},K_{j,2},y_j)$ and $\mathsf{CH} = (C_1,C_2,C_3,C_4)$ where DK_j is associated with (S,BL), CH is associated with $(\mathsf{ID},\mathsf{BL})$, and $\mathsf{ID} \in S$.

1. It computes the interpolation vector $\boldsymbol{\lambda}_A = (\lambda_{j,A})_{j\in[N]} \in \mathbb{Z}_p^N$ such that $\boldsymbol{\lambda}_A^\top \mathbf{M} = \mathbf{e}_1^\top$ and $\lambda_{j,A} = 0$ for all $j \notin A$ where $\mathbf{M}$ is the share-generation matrix for the T-out-of-N threshold policy.
2. It defines polynomials $F_S(x) = \prod_{\mathsf{ID}_k\in S}(x-\mathsf{ID}_k) = \sum_{k\in[0,|S|]} f_k x^k$ and $F_{S\setminus\{\mathsf{ID}\}}(x) = \prod_{\mathsf{ID}_k\in S\setminus\{\mathsf{ID}\}}(x-\mathsf{ID}_k) = \sum_{k\in[0,|S|-1]} \tilde{f}_k x^k$. Then, it computes the witnesses π_j and ϕ_j as

$$\pi_j = \prod_{k\in[0,|S|-1]}\big(g^{\tau^k c^{N+1-j}}\big)^{\tilde{f}_k} = g^{F_{S\setminus\{\mathsf{ID}\}}(\tau)\cdot c^{N+1-j}} \quad \forall j\in A \quad \text{and}$$
$$\phi_j = \prod_{k\in[0,|S|]}\big(W_{k,j}\big)^{f_k} = \prod_{k\in[0,|S|]}\big(\hat{g}^{\tau^k d_j}\big)^{f_k} = \hat{g}^{F_S(\tau)\cdot d_j} \quad \forall j\in A.$$

3. For each $j \in A$, it computes a node-dependent element

$$E_j = e(C_1,K_{j,1})\cdot\big(e(\pi_j,C_2)\cdot e(C_1,\phi_j)^{-1}\big)^{-y_j}\cdot e(C_3,K_{j,2})^{-1}.$$

4. It outputs the capsule key $\mathsf{CK} = E$ by computing

$$E = e\Big(\prod_{j\in A}\big(g^{c^{N+1-j}}\big)^{\lambda_{j,A}}, C_4\Big)\cdot\Big(\prod_{j\in A}E_j^{\lambda_{j,A}}\Big)^{-1}\cdot e\Big(C_1, \prod_{j\in A}A_j^{\lambda_{j,A}}\Big)^{-1}.$$

O-TBIBE-SS.Delegate$(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, (\mathsf{CH}_i)_{i\in U}, \mathsf{BL})$**:** Let $\mathsf{DK}_j = (K_{j,1}, K_{j,2}, y_j)$ for $j \in A$ and $\mathsf{CH}_i = (C_{i,1}, C_{i,2}, C_{i,3}, C_{i,4})$ for $i \in U$ where $|A| = T$ and $U \subseteq [B]$.

1. It computes the interpolation vector $\boldsymbol{\lambda}_A = (\lambda_{j,A})_{j\in[N]} \in \mathbb{Z}_p^N$ such that $\boldsymbol{\lambda}_A^\top \mathbf{M} = \mathbf{e}_1^\top$ and $\lambda_{j,A} = 0$ for all $j \notin A$ where $\mathbf{M}$ is the share-generation matrix for the T-out-of-N threshold policy.
2. It samples $z_{j,1}, z_{j,2} \in \mathbb{Z}_p$ uniformly at random for each $j \in A$ and sets a recover key $\mathsf{RK} = (z_1 = \sum_{j\in A} z_{j,1}\cdot\lambda_{j,A} \mod p)$. It then constructs a transformation key $\mathsf{TK} = (\{V_{j,1}, V_{j,2}, y_j\}_{j\in A})$ by computing, for all $j \in A$:

$$V_{j,1} = K_{j,1}\cdot\hat{g}^{z_{j,1}}\cdot\big(\hat{u}^{\mathsf{BL}}\hat{h}\big)^{z_{j,2}} \quad \text{and} \quad V_{j,2} = K_{j,2}\cdot\hat{g}^{z_{j,2}}.$$

3. It sets a work task $\mathsf{WT}_i = (D_{i,1} = C_{i,1}, D_{i,2} = C_{i,2}, D_{i,3} = C_{i,3}, D_{i,4} = C_{i,4})$ for each $i \in U$.
4. It outputs $(\mathsf{RK},\ \mathsf{TK},\ (\mathsf{WT}_i)_{i\in U})$.

O-TBIBE-SS.Transform$(\mathsf{PK}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in U}, (\mathsf{ID}_i)_{i\in U}, \mathsf{BL})$**:** Let $\mathsf{TK} = (\{V_{j,1}, V_{j,2}, y_j\}_{j\in A})$ for $j \in A$ and $\mathsf{WT}_i = (D_{i,1}, D_{i,2}, D_{i,3}, D_{i,4})$ for $i \in U$. It sets $S = \{\mathsf{ID}_i\}_{i\in U}$.

1. It computes the interpolation vector $\boldsymbol{\lambda}_A = (\lambda_{j,A})_{j\in[N]} \in \mathbb{Z}_p^N$ such that $\boldsymbol{\lambda}_A^\top \mathbf{M} = \mathbf{e}_1^\top$ and $\lambda_{j,A} = 0$ for all $j \notin A$ where $\mathbf{M}$ is the share-generation matrix for the T-out-of-N threshold policy.
2. It defines polynomials $F_S(x) = \prod_{\mathsf{ID}_k\in S}(x - \mathsf{ID}_k) = \sum_{k\in[0,|S|]} f_k x^k$ and $F_{S\setminus\{\mathsf{ID}_i\}}(x) = \prod_{\mathsf{ID}_k\in S\setminus\{\mathsf{ID}_i\}}(x - \mathsf{ID}_k) = \sum_{k\in[0,|S|-1]}\tilde{f}_{i,k}x^k$ for $i \in U$. Then, it computes the witnesses $\pi_{i,j}$ and ϕ_j that satisfy

$$\pi_{i,j} = \prod_{k\in[0,|S|-1]}\big(g^{\tau^k c^{N+1-j}}\big)^{\tilde{f}_{i,k}} = g^{F_{S\setminus\{\mathsf{ID}_i\}}(\tau)\cdot c^{N+1-j}} \quad \forall i \in U, j \in A \quad \text{and}$$

$$\phi_j = \prod_{k\in[0,|S|]}\big(W_{k,j}\big)^{f_k} = \prod_{k\in[0,|S|]}\big(\hat{g}^{\tau^k d_j}\big)^{f_k} = \hat{g}^{F_S(\tau)\cdot d_j} \quad \forall j \in A.$$

3. It pre-computes the following fixed components that depend only on the set A as

$$T_{A,1} = \prod_{j\in A}\big(g^{c^{N+1-j}}\big)^{\lambda_{j,A}} \quad \text{and} \quad T_{A,2} = \prod_{j\in A}A_j^{\lambda_{j,A}}.$$

4. For each $i \in U$, it proceeds as follows:
   (a) For each $j \in A$, it computes a node-dependent element

$$E_{i,j,1} = e(D_{i,1}, V_{j,1})\cdot\big(e(\pi_{i,j}, D_{i,2})\cdot e(D_{i,1}, \phi_j)^{-1}\big)^{-y_j}\cdot e(D_{i,3}, V_{j,2})^{-1}.$$

   (b) It constructs the transformed task $\mathsf{TT}_i = (E_{i,1}, E_{i,2})$, where the task elements $E_{i,1}$ and $E_{i,2}$ are computed as

$$E_{i,1} = e\big(T_{A,1}, D_{i,4}\big)\cdot\Big(\prod_{j\in A}E_{i,j,1}^{\lambda_{j,A}}\Big)^{-1}\cdot e\big(D_{i,1}, T_{A,2}\big)^{-1} \quad \text{and} \quad E_{i,2} = e(D_{i,1}, \hat{g}).$$

5. It outputs transformed tasks $(\mathsf{TT}_i)_{i\in U}$.

O-TBIBE-SS.Recover$(\mathsf{RK}, (\mathsf{TT}_i)_{i\in U})$**:** Let $\mathsf{RK} = (z_1)$ and $\mathsf{TT}_i = (E_{i,1}, E_{i,2})$ for $i \in U$.

1. For each $i \in U$, it recovers the capsule key $\mathsf{CK}_i = E_i$ by computing $E_i = E_{i,1}\cdot E_{i,2}^{z_1}$.
2. It outputs the capsule keys $(\mathsf{CK}_i)_{i\in U}$.

3.3 Correctness

Theorem 3.1 ([19]). *Let TBIBE-SS-KEM be the core scheme obtained by excluding the outsourcing algorithms from our proposed O-TBIBE-SS scheme. The underlying TBIBE-SS-KEM scheme is correct.*

The complete proof of correctness for this theorem is provided in Appendix A.

Theorem 3.2. *The proposed O-TBIBE-SS scheme is correct.*

Proof. The correctness of individual decryption key shares and the correctness of the threshold decryption are directly derived from the underlying TBIBE-SS-KEM scheme, as firmly established in Theorem 3.1. The correctness of the public keys and the aggregation hints is strictly guaranteed by the public key verification algorithm, as explicitly proven in Lemma 3.3. Finally, the correctness of the outsourced batch decryption $(\mathsf{Delegate}, \mathsf{Transform}, \mathsf{Recover})$ is satisfied by the structural analysis in Lemma 3.4. □

Lemma 3.3. *The proposed O-TBIBE-SS scheme satisfies the correctness of public keys.*

Proof. During the setup and public key generation phases, a benign committee node $j \in A$ publishes its matrix parameters and dynamic identity witnesses within HT_j, where the off-diagonal components are defined as $A_{j,k} = \hat{g}^{c^k \alpha_j}$ and the structural polynomial hints are generated as $W_{j,i,k} = \hat{g}^{\tau^i c^k w_j}$ for all $i \in [0, B]$ and $k \in [2N] \setminus \{N+1\}$. When the verifier evaluates the verification equations, the algebraic consistency is established sequentially via the bilinearity of the pairing map:

1. Verification of Off-Diagonal Public Components: For each index $k \in [2N] \setminus \{N+1\}$, the right-hand side of the first target equation expands by substituting the explicit matrix entry $A_{j,k}$:

$$e\big(g^{c^{N+1-k}}, A_{j,k}\big) = e\big(g^{c^{N+1-k}}, \hat{g}^{c^k \alpha_j}\big) = e(g, \hat{g})^{c^{N+1-k} \cdot c^k \alpha_j} = e(g, \hat{g})^{c^{N+1} \alpha_j}.$$

   As the index parameter k cancels out dynamically inside the exponent, the expression reduces identically to the public key component $e(g, \hat{g})^{c^{N+1} \alpha_j}$ embedded in PK_j.

2. Verification of Structured Polynomial Hints: For each $i \in [0, B]$ and $k \in [2N] \setminus \{N+1\}$, the left-hand side of the second pairing equation maps directly to the target system parameters by substituting $W_{j,i,k}$:

$$e\big(g, W_{j,i,k}\big) = e\big(g, \hat{g}^{\tau^i c^k w_j}\big) = e(g, \hat{g})^{\tau^i c^k w_j} = e\big(g^{w_j}, \hat{g}^{\tau^i c^k}\big).$$

   By shifted scalar property, the exponents slide symmetrically to the left parameter, matching the right-hand evaluation exactly.

Since both relational criteria hold under uniform balance, the algorithm successfully outputs 1, completing the proof. □

Lemma 3.4. *The proposed O-TBIBE-SS scheme satisfies the correctness of outsourced batch decryption.*

Proof. We verify the correctness of the outsourced batch decryption mechanism $(\mathsf{Delegate}, \mathsf{Transform}, \mathsf{Recover})$ by showing a structural parallel reduction, isolating only the dynamic blinding factors introduced during delegation.

Observe that the $\mathsf{Transform}$ algorithm is identical to the basic Decaps algorithm, with the sole algebraic divergence being the randomization of the decryption key shares. Specifically, during $\mathsf{Delegate}$, each decryption key share $K_{j,1}$ for $j \in A$ is blinded uniformly at random via

$$V_{j,1} = K_{j,1} \cdot \hat{g}^{z_{j,1}} \cdot \big(\hat{u}^{\mathsf{BL}} \hat{h}\big)^{z_{j,2}}.$$

The secondary random coin $z_{j,2}$ interacts with the ciphertext elements in an identical manner to the standard encryption coin r_j, and is completely absorbed and canceled out during the pairing evaluation of $E_{i,j,1}$ in Step 3-(a). Consequently, the only remaining algebraic variation embedded within the transformation component $E_{i,j,1}$ compared to the basic decryption element E_j from Theorem 3.1 is the explicit multi-linear blinding factor $\hat{g}^{z_{j,1}}$. By mapping this linear insertion directly into the base relation, each intermediate element reduces to

$$E_{i,j,1} = E_j \cdot e(D_{i,1}, \hat{g})^{z_{j,1}} = e(g^t, \hat{g}^{c^{N+1}\alpha_j}) \cdot e(D_{i,1}, \hat{g})^{z_{j,1}}.$$

Next, the cloud server aggregates these components in Step 3-(b) to construct the final task element $E_{i,1}$ using the threshold interpolation vector $\boldsymbol{\lambda}_A$ as

$$\begin{aligned} E_{i,1} &= e\Big(\prod_{j\in A} \big(g^{c^{N+1-j}}\big)^{\lambda_{j,A}}, D_{i,4}\Big) \cdot \Big(\prod_{j\in A} E_{i,j,1}^{\lambda_{j,A}}\Big)^{-1} \cdot e\Big(D_{i,1}, \prod_{j\in A} A_j^{\lambda_{j,A}}\Big)^{-1} \\ &= \left[e\Big(\prod_{j\in A} \big(g^{c^{N+1-j}}\big)^{\lambda_{j,A}}, D_{i,4}\Big) \cdot \Big(\prod_{j\in A} E_j^{\lambda_{j,A}}\Big)^{-1} \cdot e\Big(D_{i,1}, \prod_{j\in A} A_j^{\lambda_{j,A}}\Big)^{-1} \right] \cdot \\ &\quad \Big(\prod_{j\in A} e(D_{i,1}, \hat{g})^{z_{j,1}\cdot\lambda_{j,A}}\Big)^{-1}. \end{aligned}$$

Notice that the matrix-elimination kernel inside the brackets is structurally identical to the core assembly of the basic Decaps algorithm. By directly invoking the algebraic reduction matrix properties proven in Theorem 3.1, the bracketed expression collapses cleanly into the session capsule key Ω^{t_i}.

By factoring out the remaining product of the dynamic blinding factors, the element $E_{i,1}$ simplifies precisely to

$$E_{i,1} = \Omega^{t_i} \cdot e(D_{i,1}, \hat{g})^{-\sum_{j\in A} z_{j,1}\cdot\lambda_{j,A}} = \Omega^{t_i} \cdot e(D_{i,1}, \hat{g})^{-z_1}$$

where z_1 is the compressed scalar recovery key precomputed by the user during Delegate. Finally, the user runs the Recover algorithm on the received task $\mathsf{TT}_i = (E_{i,1}, E_{i,2})$. Since $E_{i,2} = e(D_{i,1}, \hat{g})$, a single scalar multi-exponentiation in $\mathbb{G}_T$ yields

$$\mathsf{CK}_i = E_{i,1} \cdot E_{i,2}^{z_1} = \big(\Omega^{t_i} \cdot e(D_{i,1}, \hat{g})^{-z_1}\big) \cdot e(D_{i,1}, \hat{g})^{z_1} = \Omega^{t_i}.$$

The threshold blinding factor $e(D_{i,1}, \hat{g})^{-z_1}$ is completely eliminated, and the algorithm outputs the exact sequence of capsule keys $(\mathsf{CK}_i)_{i\in U}$. This completes the proof. □

3.4 Security Analysis

In this section, we present rigorous proofs to establish the security of the proposed O-TBIBE-SS scheme. Specifically, we formally demonstrate its CPA security (SE-SCM-CPA) and its robustness (ROB-MKA).

3.4.1 CPA Security

Theorem 3.5. *Let TBIBE-SS-KEM be the core scheme obtained by excluding the outsourcing algorithms (Delegate, Transform, Recover) from our proposed O-TBIBE-SS scheme. The TBIBE-SS-KEM scheme is SE-SCM-CPA secure for any polynomial batch size B and any polynomial size of decryption committee N if the PoK satisfies straight-line extractability and the (B,N)-BDHEV assumption holds.*

Proof Sketch. The proof flows by reduction to the hardness of the (B,N)-BDHEV problem, fundamentally building upon the simulation framework of the original GWWW-TBIBE-SS scheme [19]. We justify the security of our modified construction by analyzing the three structural divergence points from the original construction.

In our TBIBE-SS-KEM scheme, the public key PK explicitly encapsulates the auxiliary elements $\hat{u} = \hat{g}^{u'}$ and $\hat{h} = \hat{g}^{h'}$. Crucially, these elements are already predefined as part of the public parameters PP, which the simulator $\mathcal{B}$ constructs at the beginning of the game under the standard (B,N)-BDHEV setting without knowing the secret exponents $u', h' \in \mathbb{Z}_p$. Since the simulator can effortlessly copy these pre-existing elements from PP directly into the simulated public key PK, this explicit exposure introduces no algebraic difficulties and causes no threat to the computational indistinguishability of the simulation.

The shift from direct PKE to a KEM framework alters only the operational interface, leaving the underlying algebraic security semantics intact. The capsule key generation and encapsulation processes in TBIBE-SS-KEM adhere strictly to the identical bilinear map track used in the GWWW-TBIBE-SS scheme. Consequently, the semantic security of the targeted challenge capsule key CK^* directly reduces to the hardness of the (B,N)-BDHEV instance.

The main departure is that the original TBIBE-SS security model assumes semi-malicious adversaries, whereas our framework guarantees security against fully malicious adversaries. This gap is closed by the integration of the PoK sub-protocol within the public key validation guard as pointed in [26]. When an adversary $\mathcal{A}$ registers a public key for a malicious node $j \in C_{\text{mal}}$ along with a valid proof ψ_j, the simulator $\mathcal{B}$ invokes the straight-line extractor: $(\alpha_j, w_j) \leftarrow \mathsf{PoK.Extract}(\mathsf{CRS}_{\mathsf{ext}}, \xi, X_j, \psi_j)$. By leveraging the embedded cryptographic trapdoor ξ inside the simulation string $\mathsf{CRS}_{\mathsf{ext}}$, $\mathcal{C}$ recovers the underlying secret exponents (α_j, w_j) without rewinding. $\mathcal{B}$ then injects these extracted secret scalars into the exact slot reserved for the semi-malicious randomness in the original reduction. This straight-line knowledge extraction completely neutralizes the malicious behavior of the malicious nodes, ensuring a sound reduction to the underlying mathematical hard problem.

Hence, any PPT adversary achieving a non-negligible advantage in the SE-SCM-CPA game can be converted into a solver for the (B,N)-BDHEV problem with an equivalent tight bound. □

Theorem 3.6. *The proposed O-TBIBE-SS scheme satisfies SE-SCM-CPA security for any polynomial batch size B if the underlying TBIBE-SS-KEM scheme is SE-SCM-CPA secure.*

Proof. To prove the SE-SCM-CPA security of the O-TBIBE-SS scheme, we employ the hybrid game technique. We define a sequence of hybrid games $\mathbf{G}_0$, $\mathbf{G}_1$, $\mathbf{G}_2$, and $\mathbf{G}_3$ where the first game will be the original SE-SCM-CPA security game and the last one will be a game in which an adversary has no advantage. Let $\mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_j}$ be the advantage of $\mathcal{A}$ in the game $\mathbf{G}_j$. The hybrid games are defined as follows:

Game $\mathbf{G}_0$. This game corresponds to the SE-SCM-CPA security experiment as formalized in Definition 3.4. In this game, the simulator $\mathcal{B}$ follows the specification of the proposed algorithms to generate the challenge ciphertext and evaluate all incoming oracle queries. Consequently, the advantage of the adversary $\mathcal{A}$ in this game is identical to its advantage in the SE-SCM-CPA security game as

$$\mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_0} = \mathsf{Adv}_{\textit{O-TBIBE-SS},\mathcal{A}}^{\mathsf{SE\text{-}SCM\text{-}CPA}}(\lambda).$$

Game $\mathbf{G}_1$. This game modifies the oracle simulation of $\mathbf{G}_0$ by introducing a selective, indistinguishable simulation strategy. The simulator $\mathcal{B}$ manages the adversary's queries as follows:

- In $\mathsf{OCreateKeyShare}$, $\mathcal{B}$ faithfully generates an honest key share DK_j via SK_j if $\mathsf{BL} \neq \mathsf{BL}^*$ or $(\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \notin S)$. Otherwise (i.e., when $\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \in S$), $\mathcal{B}$ explicitly samples a random

secret exponent $y_j \in \mathbb{Z}_p^*$ but leaves the remaining key share elements as $\perp$ to accommodate lazy evaluation.

- In $\mathsf{ODelegate}$, $\mathcal{B}$ executes the standard delegation algorithm if $\mathsf{BL} \neq \mathsf{BL}^*$ or $(\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \notin S)$, as valid honest key shares can be retrieved from the tracking list. Otherwise, since legitimate honest key shares do not exist for the challenge statement, $\mathcal{B}$ seamlessly emulates the transformation key TK via uniform random sampling from the target group to maintain perfect functional and statistical consistency.
- In $\mathsf{ORevealKeyShare}$ and $\mathsf{ORevealRK}$, $\mathcal{B}$ responds with the requested key components if and only if the corresponding target entry exists in the respective tracking lists. Under the strict Challenge Privacy constraints of the security game, any invalid or forbidden reveal requests are structurally blocked by the oracle rules.

As formally established in Lemma 3.7, the view of the adversary $\mathcal{A}$ remains perfectly indistinguishable from that in $\mathbf{G}_0$. Thus, we have $\mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_1} = \mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_0}$.

Game $\mathbf{G}_2$. This game proceeds exactly as $\mathbf{G}_1$ except for the method by which the honest key shares are generated. The simulator $\mathcal{B}$ no longer utilizes the secret keys SK_j directly. Instead, $\mathcal{B}$ embeds the challenge instance and leverages the $\mathsf{OComputeKeyShare}(\cdot)$ oracle provided by the underlying TBIBE-SS-KEM framework. Specifically, when evaluating an $\mathsf{OCreateKeyShare}(j, S, \mathsf{BL})$ query, $\mathcal{B}$ obtains the valid honest key share DK_j by issuing a query to $\mathsf{OComputeKeyShare}(j, S, \mathsf{BL})$ for the following two disjoint cases: (i) $\mathsf{BL} \neq \mathsf{BL}^*$, and (ii) $\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \notin S$. Since the decryption key share retrieved via $\mathsf{OComputeKeyShare}$ is identically distributed to that generated directly using SK_j, the view of the adversary $\mathcal{A}$ remains perfectly invariant. Consequently, we have $\mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_2} = \mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_1}$.

Game $\mathbf{G}_3$. This game deviates from $\mathbf{G}_2$ solely in the generation of the challenge capsule key CK_{μ}^*. In the challenge phase, the simulator provides $\mathcal{A}$ with a challenge ciphertext where CK_{μ}^* is replaced by a value chosen uniformly at random from the key space, independent of the random bit $\mu \in \{0,1\}$. The indistinguishability between $\mathbf{G}_2$ and $\mathbf{G}_3$ is reduced to the SE-SCM-CPA security of the underlying TBIBE-SS-KEM scheme, as formally established in Lemma 3.9. Accordingly, we have

$$\left|\mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_2} - \mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_3}\right| \leq \mathsf{Adv}_{\mathit{TBIBE\text{-}SS\text{-}KEM},\mathcal{B}}^{\mathsf{SE\text{-}SCM\text{-}CPA}}(\lambda).$$

Since the challenge capsule key is now purely random and independent of μ, the adversary $\mathcal{A}$ obtains no information regarding the challenge bit. Consequently, the advantage of $\mathcal{A}$ in $\mathbf{G}_3$ is exactly zero: $\mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_3} = 0$.

By applying the advantage differences established in Lemmas 3.7 , 3.8, and 3.9, we obtain the following upper bound

$$\mathsf{Adv}_{\mathit{O\text{-}TBIBE\text{-}SS},\mathcal{A}}^{\mathsf{SE\text{-}SCM\text{-}CPA}}(\lambda) \leq \sum_{j=1}^{3} \left|\mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_{j-1}} - \mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_j}\right| + \mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_3} \leq \mathsf{Adv}_{\mathit{TBIBE\text{-}SS\text{-}KEM},\mathcal{B}}^{\mathsf{SE\text{-}SCM\text{-}CPA}}(\lambda).$$

This completes the proof. □

Lemma 3.7. *The games $\mathbf{G}_0$ and $\mathbf{G}_1$ are statistically indistinguishable.*

Proof. The proof establishes that the transition from $\mathbf{G}_0$ to $\mathbf{G}_1$ introduces no statistical divergence in the view of the adversary $\mathcal{A}$. The indistinguishability relies on a combination of algebraic consistency and

the architectural boundaries enforced by the security game constraints. For all oracle queries falls into non-challenge domains ($\mathsf{BL} \neq \mathsf{BL}^*$ or $(\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \notin S)$), the simulator $\mathcal{B}$ executes the exact honest specifications. The resulting honest key shares and transformation keys are generated using the secret keys and valid key shares, ensuring perfect identity to $\mathbf{G}_0$. For the oracle query falls into the challenge domain ($\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \in S$), $\mathcal{B}$ alters its behavior by employing a lazy evaluation strategy. In $\mathsf{OCreateKeyShare}$, only the secret exponent $y_j \in \mathbb{Z}_p^*$ is sampled while the remaining elements are left as $\perp$. In $\mathsf{ODelegate}$, the transformation key TK is emulated via uniform random sampling. Because the TK except the exponent y_j in the hybrid games are originally distributed uniformly at random, this random emulation remains statistically indistinguishable from the honest execution path.

The detailed procedures for each oracle in $\mathbf{G}_1$ are given as follows:

- $\mathsf{OCreateKeyShare}(j, S, \mathsf{BL})$: If $j \notin [N]$ or $j \in C_{\text{all}}$, it responds with $\perp$. Otherwise, if there exists $(j, S, \mathsf{BL}, \dots) \in L_{\text{dks}}$, it retrieves $(j, \cdot, \cdot, \mathsf{hk}, \cdot) \leftarrow L_{\text{dks}}$ associated with (j, S, BL) and responds with hk. Otherwise:
  1. If $(\mathsf{BL} \neq \mathsf{BL}^*)$ or $(\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \notin S)$: It computes $\mathsf{DK}_j \leftarrow \mathsf{ComputeKeyShare}(\mathsf{SK}_j, \mathsf{Digest}(\mathsf{PP}, S), \mathsf{BL})$ by retrieving $\mathsf{SK}_j \leftarrow L_{\text{sk}}$.
  2. Otherwise $(\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \in S)$: It samples $y_j \in \mathbb{Z}_p^*$ and sets $\mathsf{DK}_j = (\perp, \perp, y_j)$.
  3. It picks a unique handle $\mathsf{hk} \in \{0,1\}^\lambda$, updates $L_{\text{dks}} \leftarrow L_{\text{dks}} \cup \{(j, S, \mathsf{BL}, \mathsf{hk}, \mathsf{DK}_j)\}$, and responds with hk.
- $\mathsf{ORevealKeyShare}(\mathsf{hk})$: If there is no entry $(\cdot, \cdot, \cdot, \mathsf{hk}, \cdot) \in L_{\text{dks}}$, it responds with $\perp$. Otherwise:
  1. It retrieves $(j, S, \mathsf{BL}, \mathsf{hk}, \mathsf{DK}_j) \leftarrow L_{\text{dks}}$ associated with hk, and responds with DK_j.
- $\mathsf{ODelegate}((\mathsf{hk}_j)_{j \in A \setminus C_{\text{all}}}, (\mathsf{DK}_j)_{j \in A \cap C_{\text{all}}}, (\mathsf{CH}_i)_{i \in U})$: Let $(\mathsf{hk}_j)_{j \in A \setminus C_{\text{all}}}$ be handles of honest key shares and $(\mathsf{DK}_j)_{j \in A \cap C_{\text{all}}}$ be dishonest key shares where $A = (A \setminus C_{\text{all}}) \cup (A \cap C_{\text{all}}) \subseteq [N]$ and $|A| = T$. If there is no entry $(j, \cdot, \cdot, \mathsf{hk}_j, \cdot) \in L_{\text{dks}}$ for each $j \in A \setminus C_{\text{all}}$, it responds with $\perp$. Otherwise:
  1. For each honest node $j \in A \setminus C_{\text{all}}$, it retrieves $(j, S_j, \mathsf{BL}_j, \mathsf{hk}_j, \mathsf{DK}_j) \leftarrow L_{\text{dks}}$. Next, it checks that $(\mathsf{DK}_j)_{j \in A \setminus C_{\text{all}}}$ are associated with the same (S, BL). If any check fails, it responds with $\perp$.
  2. For each dishonest node $j \in A \cap C_{\text{all}}$, it checks that $1 = \mathsf{VerifyKeyShare}(\mathsf{PK}_j, \mathsf{DK}_j, \mathsf{Digest}(\mathsf{PP}, S), \mathsf{BL})$ by retrieving $\mathsf{PK}_j \leftarrow L_{\text{sk}}$. If any check fails, it responds with $\perp$.
  3. If $(\mathsf{BL} \neq \mathsf{BL}^*)$ or $(\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \notin S)$: It computes $(\mathsf{RK}, \mathsf{TK}, (\mathsf{WT}_i)_{i \in U}) \leftarrow \mathsf{Delegate}\big(\mathsf{PK}, (\mathsf{DK}_j)_{j \in A}, (\mathsf{CH}_i)_{i \in U}, \mathsf{BL}\big)$.
  4. Otherwise $(\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \in S)$:
     (a) It samples $\{V_{j,1}, V_{j,2}\}_{j \in A} \in \hat{\mathbb{G}}^{2|A|}$ uniformly at random.
     (b) It gathers $\{y_j\}_{j \in A \setminus C_{\text{all}}}$ from honest key shares $\{\mathsf{DK}_j\}_{j \in A \setminus C_{\text{all}}}$ and $\{y_j\}_{j \in A \cap C_{\text{all}}}$ from dishonest key shares $\{\mathsf{DK}_j\}_{j \in A \cap C_{\text{all}}}$.
     (c) It sets $\mathsf{RK} = \perp$ and $\mathsf{TK} = (\{V_{j,1}, V_{j,2}, y_j\}_{j \in A})$. It derives $(\mathsf{WT}_i)_{i \in U}$ from $(\mathsf{CH}_i)_{i \in U}$ accordingly.
  5. It picks a unique handle $\mathsf{hd} \in \{0,1\}^\lambda$, updates $L_{\text{del}} \leftarrow L_{\text{del}} \cup \{(\mathsf{hd}, \mathsf{RK})\}$, and responds with $(\mathsf{hd}, \mathsf{TK}, (\mathsf{WT}_i)_{i \in U})$.
- $\mathsf{ORevealRK}(\mathsf{hd})$: If there is no entry $(\mathsf{hd}, \cdot) \in L_{\text{del}}$, it responds with $\perp$. Otherwise:

1. It retrieves $(\mathsf{hd}, \mathsf{RK}) \leftarrow L_{\text{del}}$ and responds with RK.

To establish the perfect statistical indistinguishability between the honest execution and the random emulation in the challenge domain, we analyze the algebraic structure of the simulated components. For each node $j \in A$, the relationship between the emulated group elements $(V_{j,1}, V_{j,2}) \in \hat{\mathbb{G}}^2$ and the underlying randomized exponents $(z_{j,1}, z_{j,2}) \in \mathbb{Z}_p^2$ can be explicitly described in the discrete logarithm domain as follows:

$$\begin{pmatrix} \log_{\hat{g}}(V_{j,1}) \\ \log_{\hat{g}}(V_{j,2}) \end{pmatrix} = \begin{pmatrix} \log_{\hat{g}}(K_{j,1}) \\ \log_{\hat{g}}(K_{j,2}) \end{pmatrix} + \begin{pmatrix} 1 & \log_{\hat{g}}(\hat{u}^{\mathsf{BL}}\hat{h}) \\ 0 & 1 \end{pmatrix} \begin{pmatrix} z_{j,1} \\ z_{j,2} \end{pmatrix}.$$

Let $\mathbf{A} \in \mathbb{Z}_p^{2\times 2}$ denote the transformation matrix specified above. Crucially, $\mathbf{A}$ is structured as an upper triangular matrix whose diagonal entries are both 1. It immediately follows that the determinant of the matrix is $\det(\mathbf{A}) = 1 \neq 0 \pmod{p}$, implying that $\mathbf{A}$ is strictly invertible over $\mathbb{Z}_p$. Due to this invertibility, the linear mapping induced by $\mathbf{A}$ constitutes a perfect bijection between the exponent vector $(z_{j,1}, z_{j,2}) \in \mathbb{Z}_p^2$ and the discrete logarithm of the emulated vector $(V_{j,1}, V_{j,2}) \in \hat{\mathbb{G}}^2$. Consequently, when the exponents $z_{j,1}$ and $z_{j,2}$ are chosen uniformly at random from $\mathbb{Z}_p$, the resulting elements $V_{j,1}$ and $V_{j,2}$ are distributed uniformly at random, completely independent of the key share components $(K_{j,1}, K_{j,2})$. This bijection guarantees that the view of the adversary $\mathcal{A}$ in $\mathbf{G}_1$ introduces zero statistical divergence from $\mathbf{G}_0$. This completes the proof. □

Lemma 3.8. *The games $\boldsymbol{G}_1$ and $\boldsymbol{G}_2$ are identical.*

Proof. The divergence between $\mathbf{G}_1$ and $\mathbf{G}_2$ is strictly confined to the mechanism employed to generate the honest key shares (DK_j). In $\mathbf{G}_1$, the simulator $\mathcal{B}$ evaluates the target elements directly utilizing the secret keys SK_j, whereas in $\mathbf{G}_2$, it delegates this computation to the $\mathsf{OComputeKeyShare}$ oracle provided by the underlying TBIBE-SS-KEM framework. By the correctness of the TBIBE-SS-KEM key-share generation algorithm, the outputs generated by $\mathsf{OComputeKeyShare}$ for any valid configuration (j, S, BL) replicate the exact distribution of the outputs generated via the $\mathsf{ComputeKeyShare}$ algorithm using SK_j for all honest nodes $j \in [N] \setminus C_{\text{all}}$. Consequently, the joint distribution of all oracle responses remains unaltered between the two experiments. □

Lemma 3.9. *The games $\boldsymbol{G}_2$ and $\boldsymbol{G}_3$ are computationally indistinguishable, assuming that the underlying TBIBE-SS-KEM scheme is SE-SCM-CPA secure.*

Proof. Suppose there exists an adversary $\mathcal{A}$ that breaks the security of the SE-SCM-CPA game of O-TBIBE-SS with a non-negligible advantage. We construct a simulator $\mathcal{B}$ that breaks the underlying TBIBE-SS-KEM scheme by invoking $\mathcal{A}$ as follows:

Init: $\mathcal{A}$ commits to the challenge identity-label pair $(\mathsf{ID}^*, \mathsf{BL}^*)$, the decryption committee parameters (N, T), and the subsets of corrupted and malicious nodes $C_{\text{cor}}, C_{\text{mal}}$. $\mathcal{B}$ immediately forwards this adversarial profile $(\mathsf{ID}^*, \mathsf{BL}^*, N, T, C_{\text{cor}}, C_{\text{mal}})$ to the underlying TBIBE-SS-KEM challenger, and in return, receives the target system instances: $(\mathsf{PP}_{TBSS}, (\mathsf{SK}_{j,TBSS})_{j\in C_{\text{cor}}}, (\mathsf{PK}_{j,TBSS})_{j\in[N]\setminus C_{\text{mal}}}, (\mathsf{HT}_{j,TBSS})_{j\in[N]\setminus C_{\text{mal}}})$.

Setup: $\mathcal{B}$ initializes the environment by invoking the $\mathsf{OInit}(\cdot)$ oracle inherited from $\mathbf{G}_2$. $\mathcal{B}$ embeds the received TBIBE-SS-KEM instance directly into the O-TBIBE-SS public parameters by setting $\mathsf{PP} = \mathsf{PP}_{TBSS}$, $(\mathsf{SK}_j = \mathsf{SK}_{j,TBSS})_{j\in C_{\text{cor}}}$, $(\mathsf{PK}_j = \mathsf{PK}_{j,TBSS})_{j\in[N]\setminus C_{\text{mal}}}$, and $(\mathsf{HT}_j = \mathsf{HT}_{j,TBSS})_{j\in[N]\setminus C_{\text{mal}}}$. Finally, $\mathcal{B}$ provides $(\mathsf{PP}, (\mathsf{SK}_j)_{j\in C_{\text{cor}}}, (\mathsf{PK}_j)_{j\in[N]\setminus C_{\text{mal}}}, (\mathsf{HT}_j)_{j\in[N]\setminus C_{\text{mal}}})$ to $\mathcal{A}$.

Pre-challenge Queries: $\mathcal{B}$ handles all incoming adversarial requests by executing the procedures defined in $\mathbf{G}_2$. Specifically, for any $\mathsf{OCreateKeyShare}$ query requiring an honest key share DK_j, $\mathcal{B}$ bypasses the use of the secret key SK_j (which remains outside its possession). Instead, $\mathcal{B}$ extracts the honest key share by issuing a corresponding query to the $\mathsf{OComputeKeyShare}$ oracle provided by the TBIBE-SS-KEM challenger.

Since the distribution of these oracle responses is the same, the consistency of the simulation environment is preserved.

Challenge: $\mathcal{A}$ submits the public keys and hints $(\mathsf{PK}_j, \mathsf{HT}_j)_{j \in C_{\text{mal}}}$ of the malicious nodes. $\mathcal{B}$ relays them to the TBIBE-SS-KEM challenger. Upon receiving the challenge ciphertext-key pair $(\mathsf{CH}^*_{TBSS}, \mathsf{CK}^*_{TBSS})$, $\mathcal{B}$ sets the challenge as $\mathsf{CH}^* = \mathsf{CH}^*_{TBSS}$ and $\mathsf{CK}^* = \mathsf{CK}^*_{TBSS}$, and gives $(\mathsf{CH}^*, \mathsf{CK}^*)$ to $\mathcal{A}$.

Post-challenge Queries: $\mathcal{B}$ processes all post-challenge key share queries following the logic in the pre-challenge phase. All delegation queries are evaluated via the precise procedures in $\mathbf{G}_2$.

Output: $\mathcal{A}$ outputs its guess bit μ'. $\mathcal{B}$ terminates the reduction by outputting μ'.

The advantage of $\mathcal{B}$ in the TBIBE-SS-KEM security game is derived as follows: If the TBIBE-SS-KEM challenger's bit is $b = 0$, the view of $\mathcal{A}$ is identical to $\mathbf{G}_2$. If the TBIBE-SS-KEM challenger's bit is $b = 1$, the view of $\mathcal{A}$ is identical to $\mathbf{G}_3$. Therefore, $\mathcal{B}$'s advantage in winning the TBIBE-SS-KEM game is exactly equal to $\mathcal{A}$'s advantage in distinguishing $\mathbf{G}_2$ from $\mathbf{G}_3$. □

3.4.2 Robustness

Theorem 3.10. *The proposed O-TBIBE-SS scheme satisfies ROB-MKA security for any polynomial batch size B.*

Proof. The ROB-MKA security of the proposed O-TBIBE-SS scheme is established through an algebraic contradiction derived from the decryption correctness of the O-TBIBE-SS scheme, enabled by the validity guaranteed in Lemma 3.11 and Lemma 3.12.

First, we observe that the underlying secret sharing space is initialized during the honest Setup algorithm. This honest setup ensures that the secret sharing values $\mathbf{m}_j^\top \mathbf{b} \in \mathbb{Z}_p$ are generated in a strictly trusted manner, completely insulated from any malicious adversarial behavior.

Second, during the pre-challenge queries phase, the adversary $\mathcal{A}$ maliciously registers public keys and corresponding hints by interacting with the $\mathsf{ORegisterKey}$ oracle. For each dishonest node $j \in A^*_{\text{dis}}$, the $\mathsf{ORegisterKey}$ oracle strictly executes the verification algorithm $\mathsf{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j, \mathsf{HT}_j)$. Suppose $\mathcal{A}$ outputs a forged public key that successfully passes this verification. By Lemma 3.11, this successful verification under $\mathsf{ORegisterKey}$ guarantees that the malicious public key PK_j and hint HT_j are structurally bound to the legitimate range of the primitive. Specifically, this ensures that the adversary's choices are collapsed into the designated domain, possessing unique secret exponents $(\alpha^*_j, w^*_j) \in \mathbb{Z}_p^2$.

Third, during the final output phase, $\mathcal{A}$ outputs a sequence of dishonest key shares $(\mathsf{DK}^*_j)_{j \in A^*_{\text{dis}}}$ for the dishonest nodes. To achieve a successful forgery in the game, these dishonest key shares must satisfy the verification $1 = \mathsf{VerifyKeyShare}(\mathsf{PK}_j, \mathsf{DK}^*_j, \mathsf{DIG}^*, \mathsf{BL}^*)$ for all active dishonest nodes. By Lemma 3.12, because the underlying public key PK_j has already been bound to the unique exponents (α^*_j, w^*_j) via the $\mathsf{ORegisterKey}$ oracle, passing this second check strips the adversary of any remaining algebraic degrees of freedom. Specifically, this successful verification guarantees that each forged key share instance DK^*_j is structurally compatible with the legitimate range of the primitive.

Consequently, passing these progressive checks guarantees that the dishonest components introduced by $\mathcal{A}$ are entirely canceled out and eliminated within the pairing layer. Through this perfect algebraic cancellation, the adversary's degree of freedom for malicious deviation is completely stripped away, leaving only the uncorrupted public parameters, the legally generated shares $(\mathsf{DK}_j)_{j \in \mathsf{A}^*_{\text{hon}}}$ from honest nodes, and the trusted secret components from the initial setup phase. Because every remaining active element is valid and bound to the genuine domain, the execution of Decaps becomes perfectly equivalent to the correctness of decryption. By the deterministic property of the secret-sharing reconstruction under the threshold setting,

this evaluation must yield the unique capsule key, forcing $\mathsf{CK}' = \mathsf{CK}^*_{OSS}$ to hold with probability 1. Therefore, the wining event of $\mathcal{A}$, which strictly requires $\mathsf{CK}' \neq \mathsf{CK}^*_{OSS}$, contradicts the correctness of decryption. Hence, the advantage of any PPT adversary in the ROB-MKA game of O-TBIBE-SS is unconditionally zero. □

Lemma 3.11. *Let* PP *be the public parameters honestly generated in O-TBIBE-SS. For any maliciously registered public key* PK_j *and corresponding hint* HT_j *submitted by a PPT adversary* $\mathcal{A}$*, if* $1 = \mathit{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j, \mathsf{HT}_j)$ *holds, then the instance* $(\mathsf{PK}_j, \mathsf{HT}_j)$ *is algebraically valid and structurally compatible with the range of the honest key generation algorithm.*

Proof. We formally clarify that this lemma does not imply that the adversary's sampling randomness mirrors the uniform distribution of the honest key generation. Even if the adversary completely fixes its internal coins to constants passing the verification predicate strictly binds the resulting single instance into the deterministic range of the primitive.

Specifically, the verification equations successfully collapse the adversary's operational outputs into a rigid algebraic structure. By the extractability of $\mathsf{PoK.Verify}$, there exist implicit discrete logarithms $\alpha^*_j, w^*_j \in \mathbb{Z}_p$ mapped to the group elements. Furthermore, the bilinearity of the pairing operation uniquely restrict the adversarial hints to satisfy $A_{j,k} = \hat{g}^{c^k \alpha^*_j}$ and $W_{j,i,k} = \hat{g}^{w^*_j \tau^i c^k}$. Consequently, while the ensemble distribution of the adversary's generation process is highly distinguishable from the honest uniform sampling due to the lack of repeated randomness, the individual verified instance structurally coincides with a well-formed public key parameterized by the fixed exponents (α^*_j, w^*_j). Consequently, passing the check guarantees that the verified public key and hint instance is algebraically valid, completing the proof. □

Lemma 3.12. *Let* PK_j *be a valid public key registered in O-TBIBE-SS, and let* DIG^* *and* BL^* *be the digest and label, respectively. For any forged key share* DK^*_j *submitted by a PPT adversary* $\mathcal{A}$*, if* $1 = \mathit{VerifyKeyShare}(\mathsf{PK}_j, \mathsf{DK}^*_j, \mathsf{DIG}^*, \mathsf{BL}^*)$ *holds, then the instance* DK^*_j *is algebraically valid and structurally compatible with the range of the honest key share generation algorithm.*

Proof. We formally clarify that this lemma does not imply that the adversary's sampling randomness mirrors the uniform distribution of the honest key share generation. Even if the adversary completely fixes its internal coins to constants–resulting in a point distribution upon repeated experiments–passing the verification predicate strictly binds the resulting single instance into the deterministic range of the primitive.

Let $\mathsf{DK}^*_j = (K^*_{j,1}, K^*_{j,2}, y^*_j)$ be the forged key share submitted by the adversary $\mathcal{A}$ that successfully satisfies the verification equation. By the bilinearity and non-degeneracy of the pairing operation over the prime-order groups, the verification uniquely determines the leading component $K^*_{j,1}$ as a deterministic function of $K^*_{j,2}$ and y^*_j, such that $K^*_{j,1} = \hat{g}^{c^{N+1}\alpha_j} \cdot (\mathsf{DIG}^*)^{w_j y^*_j} \cdot (\hat{u}^{\mathsf{BL}^*}\hat{h})^{\log_{\hat{g}}(K^*_{j,2})}$. Because $K^*_{j,2}$ is a valid element of the cyclic group $\hat{\mathbb{G}}$, it possesses a unique, well-defined implicit discrete logarithm $r^*_j = \log_{\hat{g}}(K^*_{j,2}) \in \mathbb{Z}_p$. Since the verification leaves zero degrees of algebraic freedom outside this designated space, this algebraic validity ensures that the adversary's output instance is structurally bound to the legitimate range of the primitive. Although $\mathcal{A}$ may select $K^*_{j,2}$ and y^*_j maliciously, the successful verification collapses the choices into a well-formed key share uniquely parameterized by the fixed exponents. Consequently, passing the check guarantees that the verified key share instance DK^*_j is algebraically valid, completing the proof. □

3.5 Discussions

Efficiency Analysis of the Basic Algorithms. Excluding the outsourcing algorithms, the computational and storage efficiency of the basic algorithms in our proposed O-TBIBE-SS scheme is largely identical

to that of the baseline TBIBE-SS scheme of Gong et al. [19]. However, a central distinction arises from our security model: unlike the standard TBIBE-SS framework, our scheme explicitly incorporates the VerifyPubKey algorithm to counter fully malicious adversaries. Verifying the public key and the corresponding hints of a single decryption node individually requires $2N+4BN$ pairing operations. In the preprocessing algorithm, the encryption key consists of two elements in $\mathbb{G}$, three elements in $\hat{\mathbb{G}}$, and one element in $\mathbb{G}_T$. The public key comprises $2BN$ elements in $\mathbb{G}$ and $2BN+2$ elements in $\hat{\mathbb{G}}$. The encapsulation algorithm outputs a ciphertext header consisting of two elements in $\mathbb{G}$ and two elements in $\hat{\mathbb{G}}$, along with a capsule key as a single element in $\mathbb{G}_T$. The decryption key share generation algorithm requires four exponentiations and outputs a decryption key share consisting of two elements in $\hat{\mathbb{G}}$ and one element in $\mathbb{Z}_p$. The decryption key share verification algorithm requires one exponentiation and four pairing operations. Finally, the decapsulation algorithm executes $2BN+4N$ exponentiations and $4N+2$ pairing operations. Consequently, evaluating a single ciphertext header requires $O(BN)$ operations, whereas a naive sequential processing of B ciphertexts scales quadratically at $O(B^2N)$ (or $O(N \cdot B \log B)$ via DFT).

Efficiency Analysis of the Outsourcing Algorithms. For the outsourcing operation, the delegation algorithm requires $4N$ exponentiations and outputs a recovery key consisting of one element in $\mathbb{Z}_p$, a transformation key consisting of two elements in $\hat{\mathbb{G}}$, and work tasks comprising approximately B ciphertext headers. The transformation algorithm, which processes these B ciphertext headers, executes $B^2N+3BN+2N$ exponentiations and $4BN+3B$ pairing operations. The resulting transformed tasks consist of $2B$ elements in $\mathbb{G}_T$. Finally, the recovery algorithm performs B exponentiations and yields B group elements in $\mathbb{G}_T$. Based on this efficiency profile, a naive execution of the decapsulation algorithm to decrypt B ciphertexts demands a computational complexity of $O(B^2N)$ (or $O(N \cdot B \log B)$ via DFT). In sharp contrast, a decrypter leveraging our outsourcing algorithms only needs to perform $O(N+B)$ operations locally. Consequently, the local computational overhead at the user side is successfully reduced from $O(N \cdot B \log B)$ down to $O(N+B)$. To comprehensively evaluate the outsourcing efficiency, we analyze the outsourcing rate in Definition 3.3 by comparing the input size against the output size for transmitted data. The input allocated for outsourcing consists of $O(B)$ ciphertext headers, and the output for outsourcing amounts to $O(B)$ group elements. Therefore, the proposed outsourcing mechanism achieves an optimal outsourcing ratio of $\mathcal{R}_{\text{out}}(B) = O(B)/O(B) = O(1)$, demonstrating constant-rate efficiency.

Single-Node Batch Verification for Public Keys. We discuss a specialized performance optimization for the VerifyPubKey algorithm focusing on a single node j. According to the specification, validating hints elements $A_{j,k}$ and $W_{j,i,k}$ for all indices i and k requires $2N+2BN$ pairing operations, which scales with the product of the batch size B and node size N. To significantly accelerate this verification process for a single node, the verifier can apply a batch verification technique using small random exponents [3]. By choosing random weights $\delta_{i,k} \in \mathbb{Z}_p^*$, the equations can be compressed into a single aggregated pairing check: $e(g, \prod_{i,k} W_{j,i,k}^{\delta_{i,k}}) = e(g^{w_j}, \prod_{i,k} \hat{g}^{\tau^i c^k \cdot \delta_{i,k}})$. This optimization allows the verifier to combine the inner algebraic components via highly efficient multi-scalar exponentiations (MSE) before executing the heavy pairing operations. Although this batching approach introduces additional MSE overhead proportional to $O(BN)$ due to the random exponentiations, the total pairing complexity for a single node collapses from $O(BN)$ down to $O(1)$ (and $O(N)$ for the overall validation sequence). Since a pairing operation is significantly heavier than a single exponentiation, this trade-off yields a substantial performance gain in practical deployments.

4 Verifiable Outsourced BTE with Silent Setup

In this section, we define the syntax and the formal security models of our proposed VOBTE-SS scheme. Guided by these definitions, we construct a concrete VOBTE-SS scheme by leveraging the previously designed O-TBIBE-SS scheme. Finally, we establish the overall security and robustness of the VOBTE-SS scheme by reducing its security to the security properties of the underlying cryptographic components.

4.1 Definition

VOBE is a novel cryptographic primitive that enables the efficient batch decryption of multiple designated ciphertexts using a single decryption key, allows the outsourcing of the underlying decryption operations to an untrusted server, and provides mechanisms to rigorously verify the correctness of the outsourced execution [22]. In this work, we extend the foundational VOBE paradigm to the VOBTE-SS framework, which additionally supports distributed threshold decryption operations under a silent setting.

In a VOBTE-SS scheme, each individual decryption node belonging to a decryption committee independently executes the key generation algorithm to produce its own secret key SK_j and public key PK_j. Subsequently, an aggregator collects these public keys $\{\mathsf{PK}_j\}$ generated by the distributed nodes to derive and publish an encryption key EK and an aggregated public key PK. Individual users utilize this EK to generate multiple ciphertexts $\{\mathsf{CT}_i\}$ associated with a specific batch label BL, which are then uploaded to a shared ciphertext pool. To decrypt, the decryptor selects a batch of B ciphertexts sharing the identical batch label BL, requests the generation of decryption key shares $\{\mathsf{DK}_j\}$ from multiple decryption nodes, and sequentially executes decryption over the ciphertexts using these retrieved key shares. When the decryptor client intends to delegate the decryption computation to a cloud server, it executes the delegation algorithm to generate a recovery token RT, a transformation key TK, and work tasks $\{\mathsf{WT}_i\}$, and subsequently transmits the TK and the $\{\mathsf{WT}_i\}$ to the cloud server. By using the TK, the cloud server transforms the received work tasks to generate transformed tasks $\{\mathsf{TT}_i\}$, which are then returned to the decryptor. Finally, the decryptor utilizes the RT to restore the transformed tasks, verifies the correctness of the server's computation, and successfully recovers the original plaintext messages.

The rigorous syntax and formal definition of the VOBTE-SS scheme are detailed as follows:

Definition 4.1 (VOBTE with Silent Setup)**.** A VOBTE with silent setup (VOBTE-SS) scheme for a maximum batch size $B \in \mathbb{N}$, message space of $\mathcal{M}$, and batch label space of $\mathcal{L}$ consists of the following seven algorithms. Let $\mathcal{M}_{\perp\!\!\!\perp} = \mathcal{M} \cup \{\perp, \perp\!\!\!\perp\}$ denote the extended message space, which includes the verification failure symbol $\perp$ (resulting from invalid input or dummy padding) and the transformation failure symbol $\perp\!\!\!\perp$.

$\mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T) \to \mathsf{PP}$: The setup algorithm takes as input the security parameter λ, the maximum batch size B, the size of the decryption committee N, and the threshold $T \leq N$. It outputs the public parameters PP.

$\mathsf{KeyGen}(\mathsf{PP}, j) \to (\mathsf{SK}_j, \mathsf{PK}_j)$: The key generation algorithm takes as input the public parameters PP and an index $j \in [N]$. It outputs a secret key SK_j and a public key PK_j.

$\mathsf{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j) \to \{0,1\}$: The public key verification algorithm takes as input the public parameters PP, the public key PK_j. It outputs 1 or 0 depending on the validity of the public key.

$\mathsf{Preprocess}(\mathsf{PP}, (\mathsf{PK}_j)_{j\in[N]}) \to (\mathsf{EK}, \mathsf{PK})$: The (deterministic) preprocessing algorithm takes as input the public parameters PP and public keys $(\mathsf{PK}_j)_{j\in[N]}$. It outputs the public encryption key EK, and the aggregated public key PK.

$\mathsf{Encrypt}(\mathsf{EK}, M, \mathsf{BL}) \to \mathsf{CT}$: The encryption algorithm takes as input the public encryption key EK, a message $M \in \mathcal{M}$, and a batch label $\mathsf{BL} \in \mathcal{L}$. It outputs a ciphertext CT.

$\mathsf{PreDecrypt}(\mathsf{PK}, \mathsf{SK}_j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}) \to \mathsf{DK}_j$: The pre-decryption algorithm takes as input the public key PK, the secret key SK_j, ciphertexts $(\mathsf{CT}_i)_{i\in[B]}$, and a batch label BL. It outputs a pre-decryption key DK_j.

$\mathsf{VerifyPreDecrypt}(\mathsf{PK}, \mathsf{DK}_j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}) \to \{0,1\}$: The pre-decryption verification algorithm takes as input the public key PK, a pre-decryption key DK_j, ciphertexts $(\mathsf{CT}_i)_{i\in[B]}$, and a batch label BL. It outputs 1 if the pre-decryption key is valid, or 0 otherwise.

$\mathsf{Decrypt}(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}) \to (M_i)_{i\in[B]} \cup \{\perp\}$: This decryption algorithm takes as input the public key PK, pre-decryption keys $(\mathsf{DK}_j)_{j\in A}$ for a subset $A \subseteq [N]$ where $|A| = T$, ciphertexts $(\mathsf{CT}_i)_{i\in[B]}$, and a batch label BL. It outputs the corresponding messages $(M_i)_{i\in[B]} \in \mathcal{M}_\perp^B$ or $\perp$ to indicate failure.

$\mathsf{Delegate}(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}) \to (\mathsf{RT}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in U}) \cup \{\perp\}$: The delegation algorithm takes as input the public key PK, pre-decryption keys $(\mathsf{DK}_j)_{j\in A}$ for a subset $A \subseteq [N]$ where $|A| = T$, ciphertexts $(\mathsf{CT}_i)_{i\in[B]}$, and a batch label BL. It outputs a recovery token RT, a transformation key TK, and work tasks $(\mathsf{WT}_i)_{i\in U}$ where $U \subseteq [B]$, or $\perp$ to indicate failure.

$\mathsf{Transform}(\mathsf{PK}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in U}, \mathsf{BL}) \to (\mathsf{TT}_i)_{i\in U}$: The transformation algorithm takes as input the public key PK, the transformation key TK, the work tasks $(\mathsf{WT}_i)_{i\in U}$, and a batch label BL. It outputs transformed tasks $(\mathsf{TT}_i)_{i\in U}$.

$\mathsf{Recover}(\mathsf{RT}, (\mathsf{TT}_i)_{i\in U}) \to (M_i)_{i\in[B]}$: The recovery algorithm takes as input the recovery token RT and the transformed tasks $(\mathsf{TT}_i)_{i\in U}$. It outputs the recovered messages $(M_i)_{i\in[B]} \in \mathcal{M}_\perp^B$.

Definition 4.2 (Correctness of VOBTE-SS)**.** A VOBTE-SS scheme is correct if for any security parameter λ and any batch size $B \le \mathrm{poly}(\lambda)$, the following four conditions hold:

- Correctness of Public Key: For any size N, any threshold $T \le N$, and any $j \in [N]$,

$$\Pr\left[\mathsf{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j) = 1 \;\middle|\; \begin{array}{l} \mathsf{PP} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T) \\ (\mathsf{SK}_j, \mathsf{PK}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j) \end{array}\right] = 1.$$

- Correctness of Pre-decryption Key: For any size N, any threshold $T \le N$, any messages $(M_i)_{i\in[B]} \in \mathcal{M}^B$, any batch label $\mathsf{BL} \in \mathcal{L}$, and any node index $j \in [N]$,

$$\Pr\left[\begin{array}{r} \mathsf{VerifyPreDecrypt}(\mathsf{PK}, \mathsf{DK}_j, (\mathsf{CT}_i)_{i\in[B]}, \\ \mathsf{BL}) = 1 \end{array} \;\middle|\; \begin{array}{l} \mathsf{PP} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T) \\ \forall j \in [N] : (\mathsf{SK}_j, \mathsf{PK}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j) \\ (\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}(\mathsf{PP}, (\mathsf{PK}_j)_{j\in[N]}) \\ \forall i \in [B] : \mathsf{CT}_i \leftarrow \mathsf{Encrypt}(\mathsf{EK}, M_i, \mathsf{BL}) \\ \mathsf{DK}_j \leftarrow \mathsf{PreDecrypt}(\mathsf{PK}, \mathsf{SK}_j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}) \end{array}\right] = 1.$$

- Correctness of Batch Decryption: For any size N, any threshold $T \le N$, any messages $(M_i)_{i\in[B]} \in \mathcal{M}^B$, any batch label $\mathsf{BL} \in \mathcal{L}$, and any subset $A \subseteq [N]$ such that $|A| = T$,

$$\Pr\left[\begin{array}{r} \mathsf{Decrypt}(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, (\mathsf{CT}_i)_{i\in[B]}, \\ \mathsf{BL}) = (M_i)_{i\in[B]} \end{array} \;\middle|\; \begin{array}{l} \mathsf{PP} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T) \\ \forall j \in [N] : (\mathsf{SK}_j, \mathsf{PK}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j) \\ (\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}(\mathsf{PP}, (\mathsf{PK}_j)_{j\in[N]}) \\ \forall i \in [B] : \mathsf{CT}_i \leftarrow \mathsf{Encrypt}(\mathsf{EK}, M_i, \mathsf{BL}) \\ \forall j \in A : \mathsf{DK}_j \leftarrow \mathsf{PreDecrypt}(\mathsf{PK}, \mathsf{SK}_j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}) \end{array}\right] = 1.$$

- Correctness of Outsourced Batch Decryption: For any size N, any threshold $T \leq N$, any messages $(M_i)_{i\in[B]} \in \mathcal{M}^B$, any batch label $\mathsf{BL} \in \mathcal{L}$, and any subset $A \subseteq [N]$ such that $|A| = T$,

$$\Pr\left[\begin{array}{l|l} \mathsf{Recover}(\mathsf{RT}, (\mathsf{TT}_i)_{i\in[B]}) & \mathsf{PP} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T) \\ = (M_i)_{i\in[B]} & \forall j \in [N] : (\mathsf{SK}_j, \mathsf{PK}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j) \\ & (\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}(\mathsf{PP}, (\mathsf{PK}_j)_{j\in[N]}) \\ & \forall i \in [B] : \mathsf{CT}_i \leftarrow \mathsf{Encrypt}(\mathsf{EK}, M_i, \mathsf{BL}) \\ & \forall j \in A : \mathsf{DK}_j \leftarrow \mathsf{PreDecrypt}(\mathsf{PK}, \mathsf{SK}_j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}) \\ & (\mathsf{RT}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in[B]}) \leftarrow \mathsf{Delegate}(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}) \\ & (\mathsf{TT}_i)_{i\in[B]} \leftarrow \mathsf{Transform}(\mathsf{PK}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in[B]}, \mathsf{BL}) \end{array}\right] = 1.$$

Based on the conventional SE-SC-CCA security model for VOBTE, we extend the security framework to define the selective CCA security with static corrupted-and-malicious nodes (SE-SCM-CCA) for the VOBTE-SS scheme. Specifically, our extended model supports independent key generation by individual decryption nodes, active registration of malicious public keys, and handle-based oracle queries. In the SE-SCM-CCA game, the adversary $\mathcal{A}$ begins by committing to a challenge batch label BL^*, the auxiliary threshold parameters (N, T), and the disjoint subsets of corrupted and malicious nodes $(C_{\text{cor}}, C_{\text{mal}})$. Upon this commitment, $\mathcal{A}$ obtains the secret keys of the corrupted nodes in C_{cor} and the public keys of all non-malicious nodes. Subsequently, $\mathcal{A}$ registers the public keys of the malicious nodes in C_{mal}, in response to which it receives the corresponding encryption key and the aggregated public key. During the query phase, $\mathcal{A}$ is allowed to issue pre-decryption key queries via the $\mathsf{OCreatePreDecrypt}$ and $\mathsf{ORevealPreDecrypt}$ oracles, as well as outsourcing delegation and recovery queries through the $\mathsf{ODelegate}$ and $\mathsf{ORevealRT}$ oracles. In the challenge phase, $\mathcal{A}$ submits two challenge messages M_0^* and M_1^*. The challenger then returns a challenge ciphertext CT^* encrypting M_μ^* under a randomly chosen bit $\mu \in \{0,1\}$. Following this, $\mathcal{A}$ is permitted to make additional oracle queries. Finally, $\mathcal{A}$ outputs a guess μ' for the hidden bit μ, and wins the game if $\mu' = \mu$. To prevent trivial wins, we impose strict constraints on the queries made to the decryption and outsourcing recovery oracles regarding the challenge ciphertext.

A more rigorous formalization of the SE-SCM-CCA security model for the VOBTE-SS scheme is detailed below:

Definition 4.3 (Selective CCA Security with Static Corrupted-and-Malicious Nodes)**.** The selective CCA security with static corrupted-and-malicious nodes (SE-SCM-CCA) of VOBTE-SS is defined via the following experiment $\mathsf{Exp}^{\mathsf{SE\text{-}SCM\text{-}CCA}}_{VOBTE\text{-}SS,\mathcal{A}}(\lambda, B)$ between a challenger $\mathcal{C}$ and a PPT adversary $\mathcal{A}$ where λ and B are given as input:

1. **Init**: $\mathcal{A}$ commits to the challenge batch label BL^*, the decryption committee size N, the threshold $T \leq N$, the disjoint subsets of corrupted and malicious committee nodes $C_{\text{cor}}, C_{\text{mal}} \subseteq [N]$ such that $|C_{\text{cor}} \cup C_{\text{mal}}| < T$. For notational simplicity, we define $C_{\text{all}} = C_{\text{cor}} \cup C_{\text{mal}}$.

2. **Setup**: $\mathcal{C}$ runs $\mathsf{PP} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T)$ and invokes the following initialization oracle:

   - $\mathsf{OInit}()$: It initializes $L_{\text{sk}} \leftarrow \emptyset$, $L_{\text{pdk}} \leftarrow \emptyset$, and $L_{\text{del}} \leftarrow \emptyset$, which are used to track committee keys, pre-decryption keys, and delegation queries, respectively.

   Next, for all nodes $j \in [N] \setminus C_{\text{mal}}$, it generates $(\mathsf{SK}_j, \mathsf{PK}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j)$ and updates $L_{\text{sk}} \leftarrow L_{\text{sk}} \cup \{(j, \mathsf{SK}_j, \mathsf{PK}_j)\}$. It gives $\left(\mathsf{PP}, (\mathsf{SK}_j)_{j\in C_{\text{cor}}}, (\mathsf{PK}_j)_{j\in[N]\setminus C_{\text{mal}}}\right)$ to $\mathcal{A}$.

3. **Preprocess**: $\mathcal{A}$ submits public keys $(\mathsf{PK}_j)_{j\in C_{\mathrm{mal}}}$ of the malicious committee nodes. $\mathcal{C}$ checks that $1 = \mathsf{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j)$ for each $j \in C_{\mathrm{mal}}$. If any check fails, it outputs 0 and aborts the experiment. Otherwise, it updates $L_{\mathrm{sk}} \leftarrow L_{\mathrm{sk}} \cup \{(j, \perp, \mathsf{PK}_j)\}$. Next, it runs $(\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}(\mathsf{PP}, (\mathsf{PK}_j)_{j\in[N]})$ and gives $(\mathsf{EK}, \mathsf{PK})$ to $\mathcal{A}$.

4. **Pre-challenge Queries**: $\mathcal{A}$ adaptively requests $\mathsf{OCreatePreDecrypt}$, $\mathsf{ORevealPreDecrypt}$, $\mathsf{ODelegate}$, and $\mathsf{ORevealRT}$ queries:

   - $\mathsf{OCreatePreDecrypt}(j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$: If $j \in C_{\mathrm{all}}$, it responds with $\perp$. If there exists $(j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}, \mathsf{hk}, \cdot) \in L_{\mathrm{pdk}}$, it responds with hk. Otherwise, it retrieves $(j, \mathsf{SK}_j, \cdot) \leftarrow L_{\mathrm{sk}}$ and computes $\mathsf{DK}_j \leftarrow \mathsf{PreDecrypt}(\mathsf{PK}, \mathsf{SK}_j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$. It picks a unique handle $\mathsf{hk} \in \{0,1\}^\lambda$, updates $L_{\mathrm{pdk}} \leftarrow L_{\mathrm{pdk}} \cup \{(j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}, \mathsf{hk}, \mathsf{DK}_j)\}$, and responds with hk.
   - $\mathsf{ORevealPreDecrypt}(\mathsf{hk})$: If there is no entry $(\cdot, \cdot, \cdot, \mathsf{hk}, \mathsf{DK}_j) \in L_{\mathrm{pdk}}$, it responds with $\perp$. Otherwise, it responds with DK_j.
   - $\mathsf{ODelegate}((\mathsf{hk}_j)_{j\in A\setminus C_{\mathrm{all}}}, (\mathsf{DK}_j)_{j\in A\cap C_{\mathrm{all}}}, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$: Let $(\mathsf{hk}_j)_{j\in A\setminus C_{\mathrm{all}}}$ be handles of honest pre-decryption keys and $(\mathsf{DK}_j)_{j\in A\cap C_{\mathrm{all}}}$ be corrupted pre-decryption keys where $A = (A \setminus C_{\mathrm{all}}) \cup (A \cap C_{\mathrm{all}}) \subseteq [N]$ and $|A| = T$. If there is no entry $(j, \cdot, \cdot, \mathsf{hk}_j, \cdot) \in L_{\mathrm{pdk}}$ for each $j \in A \setminus C_{\mathrm{all}}$, it responds with $\perp$. Otherwise:
     (a) For each honest node $j \in A \setminus C_{\mathrm{all}}$, it retrieves $(j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}_j, \mathsf{hk}_j, \mathsf{DK}_j) \leftarrow L_{\mathrm{pdk}}$. Next, it checks that $(\mathsf{DK}_j)_{j\in A\setminus C_{\mathrm{all}}}$ are associated with the same $((\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$. If any check fails, it responds with $\perp$.
     (b) For each dishonest node $j \in A \cap C_{\mathrm{all}}$, it checks that $1 = \mathsf{VerifyPreDecrypt}(\mathsf{PK}, \mathsf{DK}_j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$. If any check fails, it responds with $\perp$.
     (c) It computes $(\mathsf{RT}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in U}) \leftarrow \mathsf{Delegate}\big(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}\big)$.
     (d) It picks a unique handle $\mathsf{hd} \in \{0,1\}^\lambda$, updates $L_{\mathrm{del}} \leftarrow L_{\mathrm{del}} \cup \{(\mathsf{hd}, \mathsf{RT})\}$, and responds with $(\mathsf{hd}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in U})$.
   - $\mathsf{ORevealRT}(\mathsf{hd})$: If there is no entry $(\mathsf{hd}, \cdot) \in L_{\mathrm{del}}$, it responds with $\perp$. Otherwise, it retrieves $(\mathsf{hd}, \mathsf{RT}) \leftarrow L_{\mathrm{del}}$ and responds with RT.

5. **Challenge**: $\mathcal{A}$ submits $M_0^*, M_1^* \in \mathcal{M}$. $\mathcal{C}$ flips $\mu \in \{0,1\}$, computes a challenge ciphertext $\mathsf{CT}^* \leftarrow \mathsf{Encrypt}(\mathsf{EK}, M_\mu^*, \mathsf{BL}^*)$, and gives CT^* to $\mathcal{A}$.

6. **Post-challenge Queries**: $\mathcal{A}$ continues to request oracle queries. $\mathcal{C}$ handles these queries exactly as in the pre-challenge queries.

7. **Output**: $\mathcal{A}$ outputs $\mu' \in \{0,1\}$. The experiment returns 1 if $\mu' = \mu$ and the following oracle constraints are satisfied:

   (a) **(Challenge Privacy)**: For the challenge label BL^* and the challenge ciphertext CT^*, the adversary $\mathcal{A}$ is restricted as follows:
   - $\mathcal{A}$ never queries $\mathsf{ORevealPreDecrypt}(\mathsf{hk})$ for the handle hk returned by $\mathsf{OCreatePreDecrypt}(j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}^*)$ such that $\mathsf{CT}_i = \mathsf{CT}^*$ for some $i \in [B]$.
   - $\mathcal{A}$ never queries $\mathsf{ORevealRT}(\mathsf{hd})$ for the handle hd returned by $\mathsf{ODelegate}(\cdot, \cdot, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}^*)$ such that $\mathsf{CT}_i = \mathsf{CT}^*$ for some $i \in [B]$.

   (b) **(Label Distinctness)**: To ensure a consistent mapping between batch labels and ciphertext sets, the adversary $\mathcal{A}$ is restricted as follows:

- $\mathcal{A}$ never submits the same index-label pair (j, BL) to multiple $\mathsf{OCreatePreDecrypt}(j, \cdot, \mathsf{BL})$ queries. That is, $\mathcal{A}$ is allowed to observe at most one set of pre-decryption keys for each index-label pair.
- $\mathcal{A}$ never submits the same batch label BL to multiple $\mathsf{ODelegate}$ queries. That is, $\mathcal{A}$ is allowed to observe at most one set of delegation tasks for each unique label.
- For any batch label BL, $\mathcal{A}$ must associate it with only one specific set of ciphertexts $(\mathsf{CT}_i)_{i\in[B]}$ across both $\mathsf{OCreatePreDecrypt}$ and $\mathsf{ODelegate}$ queries. Any query involving a previously used label BL with a different set of ciphertexts $(\mathsf{CT}_j)_{j\in[B]}$ is strictly prohibited.

Otherwise, it returns 0.

The advantage of $\mathcal{A}$ is defined as $\mathsf{Adv}^{\mathsf{SE\text{-}SCM\text{-}CCA}}_{VOBTE\text{-}SS,\mathcal{A}}(\lambda) = \left|\Pr\left[\mathsf{Exp}^{\mathsf{SE\text{-}SCM\text{-}CCA}}_{VOBTE\text{-}SS,\mathcal{A}}(\lambda, B) = 1\right] - \frac{1}{2}\right|$ where the probability is taken over all the randomness of the experiment. A VOBTE-SS scheme is said to be SE-SCM-CCA secure if the advantage is negligible for all PPT adversaries.

Remark 5 (On the Necessity of Strict Challenge Privacy). The strict restriction that completely prohibits $\mathsf{ORevealPreDecrypt}$ queries for any ciphertext sequence containing CT^* under BL^* is mandated by the threshold setting of the security model. Under the static corruption model, the adversary $\mathcal{A}$ is natively empowered to compromise up to $|C_{\text{all}}| = T-1$ committee nodes at the beginning of the experiment, thereby acquiring their secret keys and computing $T-1$ valid pre-decryption keys on its own. Consequently, if the experiment permits even a single $\mathsf{ORevealPreDecrypt}$ query for any honest node $j \in [N] \setminus C_{\text{all}}$ under the challenge session, the total number of valid pre-decryption keys in $\mathcal{A}$'s possession would reach exactly T shares (i.e., $(T-1)+1 = T$). By the threshold reconstruction, collecting T valid shares allows $\mathcal{A}$ to execute $\mathsf{Decrypt}$ and trivially recover the underlying plaintext. Therefore, blocking all reveal pathways on honest nodes for the challenge session is an absolute necessity to preserve the challenge privacy.

By modifying the previously defined SE-SCM-CCA security model for the VOBTE-SS scheme, we define the verifiability (VER) model to formally capture the hardness of an outsourcing adversary tampering with the outsourced result such that it recovers to a value different from the original message. The SE-SCM-VER security model, which formalizes outsourcing verifiability, is mostly identical to the message-hiding SE-SCM-CCA security model, with the exception of the challenge phase and the final output phase. In the challenge phase, the challenger generates a challenge ciphertext CT^* by encrypting a randomly chosen message M^* and transmits it to the adversary $\mathcal{A}$. Finally, in the output phase, $\mathcal{A}$ submits a set of target ciphertexts containing the challenge ciphertext, the handle obtained from the delegation oracle, and the forged transformed tasks $\{\mathsf{TT}_i\}$. The adversary $\mathcal{A}$ wins the game if the recovery operation performed using the forged transformed tasks for the challenge ciphertext successfully outputs a valid message that is different from the original message M^*. To prevent trivial wins, we impose strict constraints on the queries made to the decryption and outsourcing recovery oracles regarding the challenge ciphertext.

A more rigorous formalization of the selective SE-SCM-VER security model for the VOBTE-SS scheme is detailed below:

Definition 4.4 (Selective Verifiability with Static Corrupted-and-Malicious Nodes)**.** The selective verifiability with static corrupted-and-malicious nodes (SE-SCM-VER) of a VOBTE-SS scheme is defined via the following experiment $\mathsf{Exp}^{\mathsf{SE\text{-}SCM\text{-}VER}}_{VOBTE\text{-}SS,\mathcal{A}}(\lambda, B)$ between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ where λ and B are given as input:

1. **Init**: $\mathcal{A}$ commits to the challenge batch label BL^*, the decryption committee size N, the threshold $T \le N$, the disjoint subsets of corrupted and malicious committee nodes $C_{\text{cor}}, C_{\text{mal}} \subseteq [N]$ such that $|C_{\text{cor}} \cup C_{\text{mal}}| < T$. For notational simplicity, we define $C_{\text{all}} = C_{\text{cor}} \cup C_{\text{mal}}$.

2. **Setup**: $\mathcal{C}$ runs $\mathsf{PP} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T)$ and invokes the following initialization oracle:

   - $\mathsf{OInit}()$: It initializes $L_{\mathrm{sk}} \leftarrow \emptyset$, $L_{\mathrm{pdk}} \leftarrow \emptyset$, and $L_{\mathrm{del}} \leftarrow \emptyset$, which are used to track committee keys, pre-decryption keys, and delegation queries, respectively.

   Next, for all nodes $j \in [N] \setminus C_{\mathrm{mal}}$, it generates $(\mathsf{SK}_j, \mathsf{PK}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j)$ and updates $L_{\mathrm{sk}} \leftarrow L_{\mathrm{sk}} \cup \{(j, \mathsf{SK}_j, \mathsf{PK}_j)\}$. It gives $\big(\mathsf{PP}, (\mathsf{SK}_j)_{j \in C_{\mathrm{cor}}}, (\mathsf{PK}_j)_{j \in [N] \setminus C_{\mathrm{mal}}}\big)$ to $\mathcal{A}$.

3. **Preprocess**: $\mathcal{A}$ submits public keys $(\mathsf{PK}_j)_{j \in C_{\mathrm{mal}}}$ of the malicious committee nodes. $\mathcal{C}$ checks that $1 = \mathsf{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j)$ for each $j \in C_{\mathrm{mal}}$. If any check fails, it outputs 0 and aborts the experiment. Otherwise, it updates $L_{\mathrm{sk}} \leftarrow L_{\mathrm{sk}} \cup \{(j, \perp, \mathsf{PK}_j)\}$. Next, it runs $(\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}(\mathsf{PP}, (\mathsf{PK}_j)_{j \in [N]})$ and gives $(\mathsf{EK}, \mathsf{PK})$ to $\mathcal{A}$.

4. **Pre-challenge Queries**: $\mathcal{A}$ adaptively requests $\mathsf{OCreatePreDecrypt}$, $\mathsf{ORevealPreDecrypt}$, $\mathsf{ODelegate}$, and $\mathsf{ORevealRT}$ queries. These oracles are defined and handled by the challenger exactly as specified in the SE-SCM-CCA security experiment (Definition 4.3), interacting with the state lists L_{pdk} and L_{del} in the same manner.

5. **Challenge**: $\mathcal{C}$ selects a random message $M^* \in \mathcal{M}$, computes a challenge ciphertext $\mathsf{CT}^* \leftarrow \mathsf{Encrypt}(\mathsf{EK}, M^*, \mathsf{BL}^*)$, and gives CT^* to $\mathcal{A}$.

6. **Post-challenge Queries**: $\mathcal{A}$ continues to request oracle queries. $\mathcal{C}$ handles these queries exactly as in the pre-challenge queries.

7. **Output**: Finally, $\mathcal{A}$ outputs
$$\big((\widetilde{\mathsf{CT}}_i)_{i \in [B]},\ \mathsf{hd}^*,\ (\widetilde{\mathsf{TT}}_i)_{i \in U}\big)$$
   where $(\widetilde{\mathsf{CT}}_i)_{i \in [B]}$ is a sequence of target ciphertexts such that $\widetilde{\mathsf{CT}}_k = \mathsf{CT}^*$ for an index $k \in [B]$, hd^* is a target handle returned by $\mathsf{ODelegate}((\mathsf{hk}_j)_{j \in A \setminus C_{\mathrm{all}}}, (\mathsf{DK}_j)_{j \in A \cap C_{\mathrm{all}}}, (\widetilde{\mathsf{CT}}_i)_{i \in [B]}, \mathsf{BL}^*)$, and $(\widetilde{\mathsf{TT}}_i)_{i \in U}$ is a sequence of target transformed tasks. Next, $\mathcal{C}$ retrieves $(\mathsf{hd}^*, \mathsf{RT}^*) \leftarrow L_{\mathrm{del}}$ and recovers $(\widetilde{M}_i)_{i \in [B]} \leftarrow \mathsf{Recover}(\mathsf{RT}^*, (\widetilde{\mathsf{TT}}_i)_{i \in U})$.

   The experiment returns 1 if $(\widetilde{M}_k \in \mathcal{M} \wedge \widetilde{M}_k \neq M^*)$ for the index k and the following oracle constraints are satisfied:

   (a) **(Challenge Privacy)**: For the challenge label BL^* and the challenge ciphertext CT^*, the adversary $\mathcal{A}$ is restricted as follows:
   - $\mathcal{A}$ never queries $\mathsf{ORevealPreDecrypt}(\mathsf{hk})$ for the handle hk returned by $\mathsf{OCreatePreDecrypt}(j, (\mathsf{CT}_i)_{i \in [B]}, \mathsf{BL}^*)$ such that $\mathsf{CT}_i = \mathsf{CT}^*$ for some $i \in [B]$.
   - $\mathcal{A}$ never queries $\mathsf{ORevealRT}(\mathsf{hd})$ for the handle hd returned by $\mathsf{ODelegate}(\cdot, \cdot, (\mathsf{CT}_i)_{i \in [B]}, \mathsf{BL}^*)$ such that $\mathsf{CT}_i = \mathsf{CT}^*$ for some $i \in [B]$.

   (b) **(Label Distinctness)**: To ensure a consistent mapping between batch labels and ciphertext sets, the adversary $\mathcal{A}$ is restricted as follows:
   - $\mathcal{A}$ never submits the same index-label pair (j, BL) to multiple $\mathsf{OCreatePreDecrypt}(j, \cdot, \mathsf{BL})$ queries. That is, $\mathcal{A}$ is allowed to observe at most one set of pre-decryption keys for each index-label pair.
   - $\mathcal{A}$ never submits the same batch label BL to multiple $\mathsf{ODelegate}$ queries. That is, $\mathcal{A}$ is allowed to observe at most one set of delegation tasks for each unique label.

- For any batch label BL, $\mathcal{A}$ must associate it with only one specific set of ciphertexts $(\mathsf{CT}_i)_{i\in[B]}$ across both $\mathsf{OCreatePreDecrypt}$ and $\mathsf{ODelegate}$ queries. Any query involving a previously used label BL with a different set of ciphertexts $(\mathsf{CT}_j)_{j\in[B]}$ is strictly prohibited.

Otherwise, it returns 0.

The advantage of $\mathcal{A}$ is defined as $\mathsf{Adv}^{\mathsf{SE\text{-}SCM\text{-}VER}}_{VOBTE\text{-}SS,\mathcal{A}}(\lambda) = \Pr\left[\mathsf{Exp}^{\mathsf{SE\text{-}SCM\text{-}VER}}_{VOBTE\text{-}SS,\mathcal{A}}(\lambda,B)=1\right]$ where the probability is taken over all the randomness of the experiment. A VOBTE-SS scheme is said to be SE-SCM-VER secure if the advantage is negligible for all PPT adversaries.

We define the robustness model to formalize the guarantee that an adversary cannot disrupt the correct decryption of the VOBTE-SS scheme, even when registering the public keys of malicious decryption nodes and submitting their dishonest pre-decryption keys. In this robustness model, the adversary $\mathcal{A}$ is allowed to query the node key generation, node key corruption, and node key registration through the $\mathsf{OCreateKey}$, $\mathsf{OCorruptKey}$, and $\mathsf{ORegisterKey}$ oracles, respectively. In the challenge phase, $\mathcal{A}$ receives a challenge ciphertext CT^*, which is an encryption of a random message under the challenge batch label BL^*. Subsequently, $\mathcal{A}$ is permitted to make pre-decryption key queries via the $\mathsf{OComputePreDecrypt}$ oracle. Finally, in the output phase, $\mathcal{A}$ submits a target authorized subset A^*, dishonest pre-decryption keys, and a sequence of ciphertexts containing the challenge ciphertext. The adversary $\mathcal{A}$ wins this game if the decryption of the challenge ciphertext–performed by combining the honest pre-decryption keys and the submitted dishonest pre-decryption keys–successfully outputs a message that is different from the original message.

A more rigorous formalization of the robustness model against malicious key attacks for the VOBTE-SS scheme is detailed below:

Definition 4.5 (Robustness against Malicious Key Attacks)**.** The robustness against malicious key attacks ($\mathsf{ROB\text{-}MKA}$) of VOBTE-SS is defined via the following experiment $\mathsf{Exp}^{\mathsf{ROB\text{-}MKA}}_{VOBTE\text{-}SS,\mathcal{A}}(\lambda,B)$ between a challenger $\mathcal{C}$ and a PPT adversary $\mathcal{A}$:

1. **Init**: $\mathcal{A}$ commits to the decryption committee size N and the threshold $T \le N$.
2. **Setup**: $\mathcal{C}$ runs $\mathsf{PP} \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^T)$ and invokes the following initialization oracle:
   - $\mathsf{OInit}()$: It initializes $L_{\text{sk}} \leftarrow \emptyset$, $Q_{\text{hon}} \leftarrow \emptyset$, $Q_{\text{cor}} \leftarrow \emptyset$, and $Q_{\text{mal}} \leftarrow \emptyset$, which are used to track secret keys, honest nodes, corrupted nodes, and malicious nodes, respectively. For notational simplicity, we define $Q_{\text{all}} = Q_{\text{hon}} \cup Q_{\text{cor}} \cup Q_{\text{mal}}$.

   Next, it provides PP to $\mathcal{A}$.
3. **Pre-challenge Queries**: $\mathcal{A}$ adaptively requests $\mathsf{OCreateKey}$, $\mathsf{OCorruptKey}$, and $\mathsf{ORegisterKey}$ queries. $\mathcal{C}$ handles these queries as follows:
   - $\mathsf{OCreateKey}(j)$: If $j \notin [N]$ or $j \in Q_{\text{all}}$, it responds with $\perp$. Otherwise, it generates $(\mathsf{SK}_j, \mathsf{PK}_j) \leftarrow \mathsf{KeyGen}(\mathsf{PP}, j)$, updates $L_{\text{sk}} \leftarrow L_{\text{sk}} \cup \{(j, \mathsf{SK}_j, \mathsf{PK}_j)\}$, adds j to Q_{hon}, and responds with PK_j.
   - $\mathsf{OCorruptKey}(j)$: If $j \notin [N]$ or $j \notin Q_{\text{hon}}$, it responds with $\perp$. Otherwise, it retrieves $(j, \mathsf{SK}_j, \cdot) \leftarrow L_{\text{sk}}$, moves j from Q_{hon} to Q_{cor}, and responds with SK_j.
   - $\mathsf{ORegisterKey}(j, \mathsf{PK}_j)$: If $j \notin [N]$ or $j \in Q_{\text{all}}$, it responds with $\perp$. Otherwise, it checks that $1 = \mathsf{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j)$ and responds with 0 if the check fails. Upon a successful check, it updates $L_{\text{sk}} \leftarrow L_{\text{sk}} \cup \{(j, \perp, \mathsf{PK}_j)\}$, adds j to Q_{mal}, and responds with 1.

It is required that $\mathcal{A}$ must saturate the committee nodes by ensuring $Q_{\text{all}} = [N]$ before moving to the next phase. If this condition is not met, the experiment returns 0 and aborts.

4. **Challenge**: $\mathcal{A}$ submits a challenge batch label BL^*. $\mathcal{C}$ proceeds as follows:
   (a) It collects all public keys from L_{sk} and runs $(\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}\big(\mathsf{PP}, (\mathsf{PK}_j)_{j\in[N]}\big)$.
   (b) It selects a random message $M^* \in \mathcal{M}$, computes a challenge ciphertext $\mathsf{CT}^* \leftarrow \mathsf{Encrypt}(\mathsf{EK}, M^*, \mathsf{BL}^*)$, and gives CT^* to $\mathcal{A}$.
5. **Post-challenge Queries**: $\mathcal{A}$ additionally requests $\mathsf{OComputePreDecrypt}$ queries and $\mathcal{C}$ handles the queries as follows:
   - $\mathsf{OComputePreDecrypt}(j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$: If $j \notin Q_{\text{hon}}$, it responds with $\perp$. Otherwise, it retrieves $(j, \mathsf{SK}_j, \cdot) \leftarrow L_{\text{sk}}$, computes $\mathsf{DK}_j \leftarrow \mathsf{PreDecrypt}(\mathsf{PK}, \mathsf{SK}_j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$, and responds with DK_j.
6. **Output**: Finally, $\mathcal{A}$ outputs
$$\big(A^*,\ (\mathsf{DK}^*_j)_{j\in A^*_{\text{dis}}},\ (\widetilde{\mathsf{CT}}_i)_{i\in[B]}\big)$$
where $A^* \subseteq [N]$ is an authorized subset such that $|A^*| = T$ where $A^*_{\text{hon}} = A^* \cap Q_{\text{hon}}$ and $A^*_{\text{dis}} = A^* \cap (Q_{\text{cor}} \cup Q_{\text{mal}})$, $(\mathsf{DK}^*_j)_{j\in A^*_{\text{dis}}}$ is a sequence of dishonest pre-decryption keys, and $(\widetilde{\mathsf{CT}}_i)_{i\in[B]}$ is a sequence of ciphertexts such that $\widetilde{\mathsf{CT}}_k = \mathsf{CT}^*$ for an index $k \in [B]$. Next, $\mathcal{C}$ proceeds with the following steps:
   (a) It checks that $1 = \mathsf{VerifyPreDecrypt}(\mathsf{PK}, \mathsf{DK}^*_j, (\widetilde{\mathsf{CT}}_i)_{i\in[B]}, \mathsf{BL}^*)$ for each dishonest node $j \in A^*_{\text{dis}}$. If any check fails, it returns 0.
   (b) It retrieves $(j, \mathsf{SK}_j, \cdot) \leftarrow L_{\text{sk}}$ and computes the pre-decryption key $\mathsf{DK}^*_j \leftarrow \mathsf{PreDecrypt}(\mathsf{PK}, \mathsf{SK}_j, (\widetilde{\mathsf{CT}}_i)_{i\in[B]}, \mathsf{BL}^*)$ for each honest node $j \in A^*_{\text{hon}}$.
   (c) It recovers the messages $(\widetilde{M}_i)_{i\in[B]} \leftarrow \mathsf{Decrypt}(\mathsf{PK}, (\mathsf{DK}^*_j)_{j\in A^*}, (\widetilde{\mathsf{CT}}_i)_{i\in[B]}, \mathsf{BL}^*)$.

   The experiment returns 1 if $A^*_{\text{dis}} \neq \emptyset$ and $\widetilde{M}_k \neq M^*$. Otherwise, it returns 0.

The advantage of $\mathcal{A}$ is defined as $\mathsf{Adv}^{\mathsf{ROB\text{-}MKA}}_{\mathit{VOBTE\text{-}SS},\mathcal{A}}(\lambda) = \Pr\left[\mathsf{Exp}^{\mathsf{ROB\text{-}MKA}}_{\mathit{VOBTE\text{-}SS},\mathcal{A}}(\lambda, B) = 1\right]$ where the probability is taken over all the randomness of the experiment. A VOBTE-SS scheme is said to be ROB-MKA secure if this advantage is negligible for all PPT adversaries.

4.2 VOBTE-SS Construction

To construct the VOBTE-SS scheme, we primarily leverage the design principles of the conventional VOBTE scheme previously proposed by Lee [22]. The pivotal advancement in our design is the replacement of the underlying O-TBIBE scheme with the O-TBIBE-SS scheme, which natively supports both a silent threshold setup and malicious public key registrations. Specifically, to guarantee the CCA security of ciphertexts, we employ the classic CHK transformation, which embeds the verification key of the OTS signature into the identity string. Furthermore, to verify the outsourced computations, the client validates the tags embedded in the ciphertext via a suitably chosen PRF. To adapt the system to a fully decentralized setting, we introduce the dedicated public key verification and pre-decryption key verification algorithms, thereby enabling the validation of independent keys and shares generated by individual decryption nodes. Crucially, since the proposed VOBTE-SS scheme inherits the underlying outsourcing efficiency inherent to the O-TBIBE-SS

scheme, the total outsourcing overhead remains tightly bound to $O(B)$. This preservation ensures that our scheme consistently provides a constant-rate outsourcing efficiency.

A detailed specification of our proposed VOBTE-SS scheme is presented below:

VOBTE-SS.Setup$(1^\lambda, 1^B, 1^N, 1^T)$**:** It generates $\mathsf{PP}_{OSS} \leftarrow$ O-TBIBE-SS.Setup$(1^\lambda, 1^B, 1^N, 1^T)$ and selects a hash function: $\mathsf{H}_s : \{0,1\}^* \to \mathcal{I}$ from $\mathcal{H}$. It outputs the public parameters $\mathsf{PP} = \big(\mathsf{PP}_{OSS}, \mathsf{H}_s\big)$.

VOBTE-SS.KeyGen(PP, j)**:** Let $\mathsf{PP} = (\mathsf{PP}_{OSS}, \mathsf{H}_s)$ and $j \in [N]$. It generates $(\mathsf{SK}_{j,OSS}, \mathsf{PK}_{j,OSS}, \mathsf{HT}_{j,OSS}) \leftarrow$ O-TBIBE-SS.KeyGen(PP_{OSS}, j). It outputs a secret key $\mathsf{SK}_j = \mathsf{SK}_{j,OSS}$ and a public key $\mathsf{PK}_j = \big(\mathsf{PK}_{j,OSS}, \mathsf{HT}_{j,OSS}\big)$.

VOBTE-SS.VerifyPubKey$(\mathsf{PP}, \mathsf{PK}_j)$**:** Let $\mathsf{PP} = (\mathsf{PP}_{OSS}, \mathsf{H}_s)$ and $\mathsf{PK}_j = (\mathsf{PK}_{j,OSS}, \mathsf{HT}_{j,OSS})$. It checks that $1 \stackrel{?}{=}$ O-TBIBE-SS.VerifyPubKey$(\mathsf{PP}_{OSS}, \mathsf{PK}_{j,OSS}, \mathsf{HT}_{j,OSS})$. If the check passes, it outputs 1; otherwise, it outputs 0.

VOBTE-SS.Preprocess$(\mathsf{PP}, (\mathsf{PK}_j)_{j\in[N]})$**:** Let $\mathsf{PP} = (\mathsf{PP}_{OSS}, \mathsf{H}_s)$ and $\mathsf{PK}_j = (\mathsf{PK}_{j,OSS}, \mathsf{HT}_{j,OSS})$ for $j \in [N]$. It generates $(\mathsf{EK}_{OSS}, \mathsf{PK}_{OSS}) \leftarrow$ O-TBIBE-SS.Preprocess$(\mathsf{PP}_{OSS}, (\mathsf{HT}_{j,OSS})_{j\in[N]})$. It outputs the public encryption key $\mathsf{EK} = (\mathsf{EK}_{OSS}, \mathsf{H}_s)$, and the aggregated public key $\mathsf{PK} = \big(\mathsf{PP}_{OSS}, \mathsf{PK}_{OSS}, (\mathsf{PK}_{j,OSS})_{j\in[N]}, \mathsf{H}_s\big)$.

VOBTE-SS.Encrypt$(\mathsf{EK}, M, \mathsf{BL})$**:** Let $\mathsf{EK} = (\mathsf{EK}_{OSS}, \mathsf{H}_s)$.

1. It generates $(\mathsf{SK}_{OTS}, \mathsf{VK}_{OTS}) \leftarrow \mathsf{OTS.KeyGen}(1^\lambda)$ and derives $\mathsf{ID} = \mathsf{H}_s(\mathsf{VK}_{OTS})$.
2. It computes $(\mathsf{CH}_{OSS}, \mathsf{CK}_{OSS}) \leftarrow$ O-TBIBE-SS.Encaps$(\mathsf{EK}_{OSS}, \mathsf{ID}, \mathsf{BL})$ and derives the key-tag value $(K \| \eta) = \mathsf{PRF}(\mathsf{CK}_{OSS}, \mathsf{ID} \| \mathsf{BL})$.
3. Then, it encrypts the message as $\mathsf{CT}_{SKE} \leftarrow \mathsf{SKE.Encrypt}(K, M)$ and generates a signature $\sigma \leftarrow \mathsf{OTS.Sign}(\mathsf{SK}_{OTS}, \mathsf{BL} \| \mathsf{CH}_{OSS} \| \mathsf{CT}_{SKE} \| \eta)$.
4. Finally, it outputs a ciphertext $\mathsf{CT} = \big(\mathsf{VK}_{OTS}, \mathsf{CH}_{OSS}, \mathsf{CT}_{SKE}, \eta, \sigma\big)$.

VOBTE-SS.PreDecrypt$(\mathsf{PK}, \mathsf{SK}_j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$**:** Let $\mathsf{PK} = (\mathsf{PP}_{OSS}, \mathsf{PK}_{OSS}, (\mathsf{PK}_{j,OSS})_{j\in[N]}, \mathsf{H}_s)$, $\mathsf{SK}_j = \mathsf{SK}_{j,OSS}$, and $\mathsf{CT}_i = (\mathsf{VK}_{i,OTS}, \mathsf{CH}_{i,OSS}, \mathsf{CT}_{i,SKE}, \eta_i, \sigma_i)$ for $i \in [B]$.

1. It initializes $U = \emptyset$ and $S = \emptyset$, and for each $i \in [B]$, proceeds as follows:
   (a) It checks that $1 \stackrel{?}{=} \mathsf{OTS.Verify}(\mathsf{VK}_{i,OTS}, \mathsf{BL} \| \mathsf{CH}_{i,OSS} \| \mathsf{CT}_{i,SKE} \| \eta_i, \sigma_i)$. If the check fails, it skips to the next iteration.
   (b) It computes $\mathsf{ID}_i = \mathsf{H}_s(\mathsf{VK}_{i,OTS})$ and updates $U \leftarrow U \cup \{i\}$ and $S \leftarrow S \cup \{\mathsf{ID}_i\}$.
2. It derives $\mathsf{DIG} \leftarrow$ O-TBIBE-SS.Digest(PP_{OSS}, S) and computes a pre-decryption key $\mathsf{DK}_{j,OSS} \leftarrow$ O-TBIBE-SS.ComputeKeyShare$(\mathsf{SK}_{j,OSS}, \mathsf{DIG}, \mathsf{BL})$.
3. Finally, it outputs the pre-decryption key $\mathsf{DK}_j = \mathsf{DK}_{j,OSS}$.

VOBTE-SS.VerifyPreDecrypt$(\mathsf{PK}, \mathsf{DK}_j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$**:** Let $\mathsf{PK} = (\mathsf{PP}_{OSS}, \mathsf{PK}_{OSS}, (\mathsf{PK}_{j,OSS})_{j\in[N]}, \mathsf{H}_s)$, $\mathsf{DK}_j = \mathsf{DK}_{j,OSS}$, and $\mathsf{CT}_i = (\mathsf{VK}_{i,OTS}, \mathsf{CH}_{i,OSS}, \mathsf{CT}_{i,SKE}, \eta_i, \sigma_i)$ for $i \in [B]$.

1. It initializes $S = \emptyset$, and for each $i \in [B]$, it proceeds as follows:
   (a) It checks that $1 \stackrel{?}{=} \mathsf{OTS.Verify}(\mathsf{VK}_{i,OTS}, \mathsf{BL} \| \mathsf{CH}_{i,OSS} \| \mathsf{CT}_{i,SKE} \| \eta_i, \sigma_i)$. If the check fails, it skips to the next iteration.

(b) It computes $\mathsf{ID}_i = \mathsf{H}_s(\mathsf{VK}_{i,OTS})$ and updates $S \leftarrow S \cup \{\mathsf{ID}_i\}$.

2. It derives $\mathsf{DIG} \leftarrow$ O-TBIBE-SS.Digest(PP_{OSS}, S). It checks that $1 \stackrel{?}{=}$ O-TBIBE-SS.VerifyKeyShare $(\mathsf{PK}_{j,OSS}, \mathsf{DK}_{j,OSS}, \mathsf{DIG}, \mathsf{BL})$.
3. If the check passes , it outputs 1. Otherwise, it outputs 0.

VOBTE-SS.Decrypt$(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$**:** Let $\mathsf{PK} = (\mathsf{PP}_{OSS}, \mathsf{PK}_{OSS}, (\mathsf{PK}_{j,OSS})_{j\in[N]}, \mathsf{H}_s)$, $\mathsf{DK}_j = \mathsf{DK}_{j,OSS}$ for $j \in A$ where $|A| = T$, and $\mathsf{CT}_i = (\mathsf{VK}_{i,OTS}, \mathsf{CH}_{i,OSS}, \mathsf{CT}_{i,SKE}, \eta_i, \sigma_i)$ for $i \in [B]$.

1. It initializes $U = \emptyset$ and $S = \emptyset$, and for each $i \in [B]$, it proceeds as follows:
   (a) It checks that $1 \stackrel{?}{=} \mathsf{OTS.Verify}(\mathsf{VK}_{i,OTS}, \mathsf{BL} \| \mathsf{CH}_{i,OSS} \| \mathsf{CT}_{i,SKE} \| \eta_i, \sigma_i)$. If the check fails, it sets $M_i = \perp$ and skips to the next iteration.
   (b) It computes $\mathsf{ID}_i = \mathsf{H}_s(\mathsf{VK}_{i,OTS})$ and updates $U \leftarrow U \cup \{i\}$, $S \leftarrow S \cup \{\mathsf{ID}_i\}$.
2. It derives $\mathsf{DIG} \leftarrow$ O-TBIBE-SS.Digest(PP_{OSS}, S). It checks that $1 \stackrel{?}{=}$ O-TBIBE-SS.VerifyKeyShare $(\mathsf{PK}_{j,OSS}, \mathsf{DK}_{j,OSS}, \mathsf{DIG}, \mathsf{BL})$ for each $j \in A$. If any check fails, it outputs $\perp$.
3. For each $i \in U$, it recovers $\mathsf{CK}_{i,OSS} \leftarrow$ O-TBIBE-SS.Decaps$(\mathsf{PK}_{OSS}, (\mathsf{DK}_{j,OSS})_{j\in A}, S, \mathsf{CH}_{i,OSS}, \mathsf{ID}_i, \mathsf{BL})$, from which it derives $(K_i \| \eta_i) = \mathsf{PRF}(\mathsf{CK}_{i,OSS}, \mathsf{ID}_i \| \mathsf{BL})$ and decrypts the message $M_i \leftarrow \mathsf{SKE.Decrypt}(K_i, \mathsf{CT}_{i,SKE})$.
4. Finally, it outputs the corresponding messages $(M_i)_{i\in[B]}$.

VOBTE-SS.Delegate$(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$**:** Let $\mathsf{PK} = (\mathsf{PP}_{OSS}, \mathsf{PK}_{OSS}, (\mathsf{PK}_{j,OSS})_{j\in[N]}, \mathsf{H}_s)$, $\mathsf{DK}_j = \mathsf{DK}_{j,OSS}$ for $j \in A$, and $\mathsf{CT}_i = (\mathsf{VK}_{i,OTS}, \mathsf{CH}_{i,OSS}, \mathsf{CT}_{i,SKE}, \eta_i, \sigma_i)$ for $i \in [B]$.

1. It initializes $U = \emptyset$ and $S = \emptyset$, and for each $i \in [B]$, it proceeds as follows:
   (a) It checks that $1 \stackrel{?}{=} \mathsf{OTS.Verify}(\mathsf{VK}_{i,OTS}, \mathsf{BL} \| \mathsf{CH}_{i,OSS} \| \mathsf{CT}_{i,SKE} \| \eta_i, \sigma_i)$. If the check fails, it skips to the next iteration.
   (b) It computes $\mathsf{ID}_i = \mathsf{H}_s(\mathsf{VK}_{i,OTS})$ and updates $U \leftarrow U \cup \{i\}$, $S \leftarrow S \cup \{\mathsf{ID}_i\}$.
2. It derives $\mathsf{DIG} \leftarrow$ O-TBIBE-SS.Digest(PP_{OSS}, S). It checks that $1 \stackrel{?}{=}$ O-TBIBE-SS.VerifyKeyShare $(\mathsf{PK}_{j,OSS}, \mathsf{DK}_{j,OSS}, \mathsf{DIG}, \mathsf{BL})$ for each $j \in A$. If any check fails, it outputs $\perp$.
3. It generates $(\mathsf{RK}_{OSS}, \mathsf{TK}_{OSS}, (\mathsf{WT}_{i,OSS})_{i\in U}) \leftarrow$ O-TBIBE-SS.Delegate$(\mathsf{PK}_{OSS}, (\mathsf{DK}_{j,OSS})_{j\in A}, (\mathsf{CH}_{i,OSS})_{i\in U}, \mathsf{BL})$.
4. It sets a recovery token $\mathsf{RT} = \big(\mathsf{RK}_{OSS}, (\mathsf{CT}_{i,SKE}, \eta_i)_{i\in U}, \mathsf{BL}\big)$, a transformation key $\mathsf{TK} = \mathsf{TK}_{OSS}$, and a work task $\mathsf{WT}_i = (\mathsf{WT}_{i,OSS}, \mathsf{VK}_{i,OTS})$ for each $i \in U$.
5. Finally, it outputs $(\mathsf{RT}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in U})$.

VOBTE-SS.Transform$(\mathsf{PK}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in U}, \mathsf{BL})$**:** Let $\mathsf{PK} = (\mathsf{PP}_{OSS}, \mathsf{PK}_{OSS}, (\mathsf{PK}_{j,OSS})_{j\in[N]}, \mathsf{H}_s)$, $\mathsf{TK} = \mathsf{TK}_{OSS}$ and $\mathsf{WT}_i = (\mathsf{WT}_{i,OSS}, \mathsf{VK}_{i,OTS})$ for $i \in U$.

1. It computes $\mathsf{ID}_i = \mathsf{H}_s(\mathsf{VK}_{i,OTS})$ for all $i \in U$.
2. It obtains $(\mathsf{TT}_{i,OSS})_{i\in U} \leftarrow$ O-TBIBE-SS.Transform$(\mathsf{PK}_{OSS}, \mathsf{TK}_{OSS}, (\mathsf{WT}_{i,OSS})_{i\in U}, (\mathsf{ID}_i)_{i\in U}, \mathsf{BL})$ and sets $\mathsf{TT}_i = (\mathsf{TT}_{i,OSS}, \mathsf{ID}_i)$ for $i \in U$.
3. Finally, it outputs the transformed tasks $(\mathsf{TT}_i)_{i\in U}$.

VOBTE-SS.Recover$(\mathsf{RT}, (\mathsf{TT}_i)_{i\in U})$**:** Let $\mathsf{RT} = (\mathsf{RK}_{OSS}, (\mathsf{CT}_{i,SKE}, \eta_i)_{i\in U}, \mathsf{BL})$ and $\mathsf{TT}_i = (\mathsf{TT}_{i,OSS}, \mathsf{ID}_i)$ for $i \in U$.

1. It recovers capsule keys $(\mathsf{CK}_{i,OSS})_{i\in U} \leftarrow \mathsf{O\text{-}TBIBE\text{-}SS.Recover}(\mathsf{RK}_{OSS}, (\mathsf{TT}_{i,OSS})_{i\in U})$.
2. For each $i \in [B]$, it proceeds as follows:
   (a) If $i \notin U$, it sets $M_i = \perp$ and skips to the next iteration.
   (b) It derives the key-tag value $(K_i' \| \eta_i') = \mathsf{PRF}(\mathsf{CK}_{i,OSS}, \mathsf{ID}_i \| \mathsf{BL})$.
   (c) If the verification $\eta_i \stackrel{?}{=} \eta_i'$ holds, it decrypts the message $M_i \leftarrow \mathsf{SKE.Decrypt}(K_i', \mathsf{CT}_{i,SKE})$; otherwise, it sets $M_i = \perp\!\!\!\perp$.
3. Finally, it outputs the recovered messages $(M_i)_{i\in[B]} \in \mathcal{M}_{\perp\!\!\!\perp}^B$.

4.3 Correctness

Theorem 4.1. *The proposed VOBTE-SS scheme is correct, provided that the underlying O-TBIBE-SS scheme, the OTS scheme, and the SKE scheme satisfy their respective correctness properties.*

Proof. The correctness of the VOBTE-SS scheme is established by demonstrating the correctness of public keys, pre-decryption keys, batch decryption, and outsourced batch decryption.

Public Key Correctness. The public key correctness in the VOBTE-SS scheme is immediately established by that of the underlying O-TBIBE-SS scheme. Specifically, because both the key generation and public key verification algorithms of the VOBTE-SS scheme directly invoke their counterparts in the O-TBIBE-SS scheme, any valid public key is natively preserved.

Pre-decryption Key Correctness. The correctness of the OTS scheme guarantees the consistency of the verification keys $\mathsf{VK}_{i,OTS}$ for all $i \in [B]$. By the deterministic property of the hash function, the same identity set $S = \{\mathsf{ID}_i\}_{i\in[B]}$ is derived from these verification keys. Given the consistent identity set S and the identical batch label BL as inputs, the key-share correctness of the underlying O-TBIBE-SS scheme immediately ensures that every honestly generated key share DK_j successfully passes its key-share verification.

Batch Decryption Correctness. By the correctness of the OTS scheme, the identity set $S = \{\mathsf{ID}_i\}_{i\in[B]}$ derived from the verification keys $\{\mathsf{VK}_{i,OTS}\}$ is consistently established. Given this consistent set S and the identical batch label BL provided, the key-share correctness and decryption correctness of the underlying O-TBIBE-SS scheme collectively guarantee the successful recovery of the underlying capsule keys. Since PRF is a deterministic function, the reconstructed SKE keys K_i are identical to the original keys used during encryption. Consequently, the original messages M_i are correctly restored via SKE decryption by the correctness of the SKE scheme.

Outsourced Batch Decryption Correctness. In the outsourced setting, the consistency of the identity set S is ensured by the correctness of the OTS scheme and the consistency of the batch label BL is ensured by the same input. If the identity set S and the batch label BL are same, the recover algorithm of the O-TBIBE-SS scheme effectively derives the valid capsule keys $\mathsf{CK}_{i,OSS}$ by the outsourced batch decryption correctness of the O-TBIBE-SS scheme. In the recover algorithm of the VOBTE-SS scheme, the computed tag η_i' from PRF remains identical to the η_i stored in the state, validating the integrity of the outsourced computation. Finally, the deterministic PRF ensures that the recovered symmetric key K_i' is identical to the symmetric key, allowing for the recovery of the original messages $(M_i)_{i\in[B]}$ by the correctness of the SKE scheme. □

4.4 Security Analysis

In this section, we perform rigorous reduction proofs for the proposed VOBTE-SS scheme to evaluate its CCA security, outsourcing verifiability, and robustness. These reduction proofs are systematically constructed by leveraging the provable security of the underlying cryptographic primitives.

Theorem 4.2. *The proposed VOBTE-SS scheme satisfies SE-SCM-CCA security for any polynomial batch size B, assuming that the OTS scheme is SUF secure, and the hash function is collision-resistant, the underlying O-TBIBE-SS scheme is SE-SCM-CPA secure, the PRF is pseudo-random, and the SKE scheme is IND-OT secure.*

The proof of CCA security for this theorem is given in Appendix C.1.

Theorem 4.3. *The proposed VOBTE-SS scheme satisfies SE-SCM-VER security for any polynomial batch size B, assuming that the OTS scheme is SUF secure, and the hash function is collision-resistant, underlying O-TBIBE-SS scheme is SE-SCM-CPA secure, and the PRF is pseudo-random.*

The proof of verifiability for this theorem is given in Appendix C.2.

Theorem 4.4. *The proposed VOBTE-SS scheme satisfies ROB-MKA security for any polynomial batch size B, assuming that the underlying O-TBIBE-SS scheme is ROB-MKA secure.*

The proof of robustness for this theorem is given in Appendix C.3.

5 Conclusion

In this paper, we presented a communication- and computation-efficient decryption outsourcing mechanism for batched threshold encryption with silent setup (BTE-SS), specifically tailored for fully decentralized environments such as blockchains. We first introduced a novel O-TBIBE-SS scheme by augmenting the baseline TBIBE-SS scheme with proofs of knowledge and public key verification to handle malicious public key registrations, while integrating decryption outsourcing via partial re-randomization of decryption key shares, and formally proved its security. Subsequently, by synthesizing the O-TBIBE-SS scheme with additional cryptographic primitives, we constructed a VOBTE-SS scheme, which simultaneously supports decryption outsourcing and verifiability of the outsourced computation. By delegating heavy decryption tasks to an untrusted cloud server, our proposed VOBTE-SS scheme reduces the client-side computational complexity for evaluating B batched ciphertexts from $O(N \cdot B \log B)$ down to $O(B)$, while achieving optimal communication efficiency through a constant-ratio outsourcing efficiency.

References

[1] Amit Agarwal, Rex Fernando, and Benny Pinkas. Efficiently-thresholdizable batched identity based encryption, with applications. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology - CRYPTO 2025, Part III*, Lecture Notes in Computer Science, pages 69–100. Springer, 2025.

[2] Joseph Bebel and Dev Ojha. Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Paper 2022/898, 2022.

[3] Mihir Bellare, Juan A. Garay, and Tal Rabin. Fast batch verification for modular exponentiation and digital signatures. In Kaisa Nyberg, editor, *Advances in Cryptology - EUROCRYPT '98*, volume 1403 of *Lecture Notes in Computer Science*, pages 236–250. Springer, 1998.

[4] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the Weil pairing. In Joe Kilian, editor, *Advances in Cryptology - CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 213–229. Springer, 2001.

[5] Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyan Qu. BEAT-MEV: epochless approach to batched threshold encryption for MEV prevention. In Lujo Bauer and Giancarlo Pellegrino, editors, *34th USENIX Security Symposium, USENIX Security 2025*, pages 3457–3476. USENIX Association, 2025.

[6] Vitalik Buterin. A next-generation smart contract and decentralized application platform. `https://ethereum.org/en/whitepaper/`, 2014.

[7] Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security Symposium, USENIX Security 2024*. USENIX Association, 2024.

[8] Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. Practical mempool privacy via one-time setup batched threshold encryption. In Lujo Bauer and Giancarlo Pellegrino, editors, *USENIX Security Symposium, USENIX Security 2025*, pages 3477–3495. USENIX Association, 2025.

[9] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *IEEE Symposium on Security and Privacy - SP 2020*, pages 910–927. IEEE, 2020.

[10] Nico Döttling, Lucjan Hanzlik, Bernardo Magri, and Stella Wohnig. Mcfly: Verifiable encryption to the future made practical. In Foteini Baldimtsi and Christian Cachin, editors, *Financial Cryptography and Data Security - FC 2023, Part I*, Lecture Notes in Computer Science, pages 252–269. Springer, 2023.

[11] Stefan Dziembowski, Sebastian Faust, and Jannik Luhn. Shutter network: Private transactions from threshold cryptography. Cryptology ePrint Archive, Paper 2024/1981, 2024.

[12] Marc Fischlin. Communication-efficient non-interactive proofs of knowledge with online extractors. In Victor Shoup, editor, *Advances in Cryptology - CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 152–168. Springer, 2005.

[13] Sanjam Garg, Aarushi Goel, and Mingyuan Wang. How to prove statements obliviously? In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024, Part X*, Lecture Notes in Computer Science, pages 449–487. Springer, 2024.

[14] Sanjam Garg, Mohammad Hajiabadi, Dimitris Kolonelos, Abhiram Kothapalli, and Guru-Vamsi Policharla. A framework for witness encryption from linearly verifiable snarks and applications. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology - CRYPTO 2025, Part III*, Lecture Notes in Computer Science, pages 504–539. Springer, 2025.

[15] Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. hinTS: Threshold signatures with silent setup. In *IEEE Symposium on Security and Privacy, SP 2024*, pages 3034–3052. IEEE, 2024.

[16] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 352–386. Springer, 2024.

[17] Rosario Gennaro, Steven Goldfeder, and Arvind Narayanan. Threshold-optimal DSA/ECDSA signatures and an application to bitcoin wallet security. In Mark Manulis, Ahmad-Reza Sadeghi, and Steve A. Schneider, editors, *Applied Cryptography and Network Security - ACNS 2016*, volume 9696 of *Lecture Notes in Computer Science*, pages 156–174. Springer, 2016.

[18] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. *J. Cryptol.*, 20(1):51–83, 2007.

[19] Junqing Gong, Brent Waters, Hoeteck Wee, and David J. Wu. Threshold batched identity-based encryption from pairings in the plain model. In Joan Daemen and Emmanuel Thomé, editors, *Advances in Cryptology - EUROCRYPT 2026, Part V*, Lecture Notes in Computer Science, pages 548–578. Springer, 2026.

[20] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology - ASIACRYPT 2010*, Lecture Notes in Computer Science, pages 177–194. Springer, 2010.

[21] Chelsea Komlo and Ian Goldberg. FROST: flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson Jr., and Colin O'Flynn, editors, *Selected Areas in Cryptography - SAC 2020*, volume 12804 of *Lecture Notes in Computer Science*, pages 34–65. Springer, 2020.

[22] Kwangsu Lee. VOBE: Verifiable outsourced batched encryption for secure delegation of batched decryption. Cryptology ePrint Archive, Paper 2026/1197, 2026.

[23] Peyman Momeni, Sergey Gorbunov, and Bohan Zhang. Fairblock: Preventing blockchain front-running with minimal overheads. In Fengjun Li, Kaitai Liang, Zhiqiang Lin, and Sokratis K. Katsikas, editors, *Security and Privacy in Communication Networks - SecureComm 2022*, Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, pages 250–271. Springer, 2022.

[24] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. `https://bitcoin.org/bitcoin.pdf`, 2008.

[25] Victor Shoup and Rosario Gennaro. Securing threshold cryptosystems against chosen ciphertext attack. *J. Cryptol.*, 15(2):75–96, 2002.

[26] Brent Waters and David J. Wu. Silent threshold cryptography from pairings: Expressive policies in the plain model. In Joan Daemen and Emmanuel Thomé, editors, *Advances in Cryptology - EUROCRYPT 2026, Part V*, Lecture Notes in Computer Science, pages 517–547. Springer, 2026.

A Correctness of TBIBE-SS-KEM Scheme

The correctness for the underlying TBIBE-SS-KEM scheme is established through the joint formulation of the preceding sub-components. Specifically, by invoking Lemma A.1, the integrity and algebraic consistency of each decryption key share are successfully verified. By applying these validated shares to the multi-linear matrix assembly as shown in Lemma A.2, the dense two-dimensional Toeplitz cross-terms are completely canceled by the preprocessed public matrix keys and parameters. Consequentially, all non-kernel entities successfully cancel out to the identity element, leaving only the targeted session capsule key. This completes the proof of correctness.

Lemma A.1. *The underlying TBIBE-SS-KEM scheme satisfies the correctness of key shares.*

Proof. When a benign verifier evaluates the right-hand side of the verification equation utilizing the public key element PK_j, the bilinear pairings expand by substituting the explicit secret components $K_{j,1} = \hat{g}^{c^{N+1}\alpha_j} \cdot (\mathsf{DIG})^{w_j y_j} \cdot (\hat{u}^{\mathsf{BL}}\hat{h})^{r_j}$ and $K_{j,2} = \hat{g}^{r_j}$:

$$
\begin{aligned}
& e(g, K_{j,1}) \cdot e(g^{w_j}, \mathsf{DIG})^{-y_j} \cdot e(u^{\mathsf{BL}}h, K_{j,2})^{-1} \\
&= e\Big(g,\, \hat{g}^{c^{N+1}\alpha_j} \cdot \big(\hat{g}^{c^{N+1}F_S(\tau)}\big)^{w_j y_j} \cdot \big(\hat{u}^{\mathsf{BL}}\hat{h}\big)^{r_j}\Big) \cdot e\big(g^{w_j},\, \hat{g}^{c^{N+1}F_S(\tau)}\big)^{-y_j} \cdot e\big(u^{\mathsf{BL}}h,\, \hat{g}^{r_j}\big)^{-1} \\
&= e(g,\hat{g})^{c^{N+1}\alpha_j} \cdot \Big[e(g,\hat{g})^{c^{N+1}F_S(\tau)w_j y_j} \cdot e(g,\hat{g})^{-c^{N+1}F_S(\tau)w_j y_j}\Big] \cdot \Big[e(u^{\mathsf{BL}}h,\hat{g})^{r_j} \cdot e(u^{\mathsf{BL}}h,\hat{g})^{-r_j}\Big].
\end{aligned}
$$

Due to the symmetric scalar shifting property within the bilinear pairing map, the identity polynomial elements parameterized by $w_j y_j$ inside the first bracket cancel out perfectly. Concurrently, the multi-linear random coins r_j bound to the blinding parameters BL inside the second bracket annihilate each other to identity. Consequently, the expression collapses precisely to the public element $e(g,\hat{g})^{c^{N+1}\alpha_j}$. □

Lemma A.2. *The underlying TBIBE-SS-KEM scheme satisfies the correctness of decryption.*

Proof. We present the systematic, step-by-step algebraic verification for the correctness of the basic Decaps algorithm. To elegantly dissolve the dense algebraic matrix structures generated by the silent setup phase, the verification proof is structured into the following five logical phases.

Step 1: Identity Polynomial and Witness Interlocking. In the first phase, we analyze the identity-based polynomial interaction embedded within the ciphertext components and the node's structural witnesses. By exploiting the polynomial relation $F_S(\tau) = F_{S\setminus\{\mathsf{ID}\}}(\tau) \cdot (\tau - \mathsf{ID})$, the pairing terms parameterized by the ciphertext elements and witnesses yield the following relation:

$$
\begin{aligned}
& e(\pi_j, C_2) \cdot e(C_1, \phi_j)^{-1} \\
&\quad = e\Big(g^{F_{S\setminus\{\mathsf{ID}\}}(\tau)\cdot c^{N+1-j}}, \hat{g}^{(\tau-\mathsf{ID})\cdot wt}\Big) \cdot e\Big(g^t, \hat{g}^{F_S(\tau)\cdot d_j}\Big)^{-1} \\
&\quad = e\Big(g^{F_{S\setminus\{\mathsf{ID}\}}(\tau)\cdot(\tau-\mathsf{ID})}, \hat{g}^{c^{N+1-j}\cdot(c^j w_j + \sum_{k\in[N],k\neq j} c^k w_k)}\Big)^t \cdot e\Big(g^t, \hat{g}^{F_S(\tau)\cdot d_j}\Big)^{-1} \\
&\quad = e\Big(g^{F_S(\tau)}, \hat{g}^{c^{N+1}w_j + \sum_{k\in[N],k\neq j} c^{N+1-j+k} w_k}\Big)^t \cdot e\Big(g^{F_S(\tau)}, \hat{g}^{d_j}\Big)^{-t} \\
&\quad = e\Big(g^{F_S(\tau)}, \hat{g}^{c^{N+1}w_j}\Big)^t \cdot e\Big(g^{F_S(\tau)}, \hat{g}^{d_j}\Big)^t \cdot e\Big(g^{F_S(\tau)}, \hat{g}^{d_j}\Big)^{-t} \\
&\quad = e\big(g^t, \hat{g}^{F_S(\tau)\cdot c^{N+1}w_j}\big).
\end{aligned}
$$

Step 2: Component E_j Collapse via Secret Keys. By incorporating the resulting relation from Step 1 into the structural decryption component E_j, and utilizing the dynamic cancelation of the random coin r

with the blinding factors, the decryption share component for each node completely collapses into a single deterministic node-specific matrix entry:

$$\begin{aligned}
E_j &= e(C_1, K_{j,1}) \cdot \big(e(\pi_j, C_2) \cdot e(C_1, \phi_j)^{-1}\big)^{-y_j} \cdot e(C_3, K_{j,2})^{-1} \\
&= e\big(g^t, \hat{g}^{c^{N+1}\alpha_j}\big(\hat{g}^{F_S(\tau)\cdot c^{N+1}}\big)^{w_j y_j}\big(\hat{u}^{\mathsf{BL}}\hat{h}\big)^{r_j}\big) \cdot e\big(g^t, \hat{g}^{F_S(\tau)\cdot c^{N+1} w_j}\big)^{-y_j} \cdot e\big((u^{\mathsf{BL}}h)^t, \hat{g}^{r_j}\big)^{-1} \\
&= e\big(g^t, \hat{g}^{c^{N+1}\alpha_j}\big).
\end{aligned}$$

Step 3: Master Key Vectorized Decomposition. To prepare for the committee-wide matrix reconstruction, we analyze the structural composition of the element $\hat{g}^{\gamma}$ utilized during the encapsulation phase. By mapping the master key vector $\mathbf{b}$ and the node private exponent vector α, the parameter $\hat{g}^{\gamma}$ can be explicitly decomposed into two products of elements:

$$\hat{g}^{\gamma} = \hat{g}^{\gamma_0} \cdot \prod_{k\in[N]} g^{c^k\alpha_k} = \prod_{k\in[N]} \hat{g}^{c^k \mathbf{m}_k^{\top}\mathbf{b}} \cdot \prod_{k\in[N]} \hat{g}^{c^k\alpha_k}.$$

Step 4: Ciphertext C_4 Aggregation via Threshold Interpolation. The decapsulation algorithm takes the ciphertext header component $C_4 = (\hat{g}^{\gamma})^t$ and binds it with the public parameters using the threshold interpolation vector $\boldsymbol{\lambda}_A$. This step expands the aggregate pairing form across the authorized decryption committee subset A:

$$\begin{aligned}
&e\Big(\prod_{j\in A}\big(g^{c^{N+1-j}}\big)^{\lambda_{j,A}}, C_4\Big) \\
&= e\Big(\prod_{j\in A} g^{c^{N+1-j}\lambda_{j,A}}, \Big(\prod_{k\in[N]} \hat{g}^{c^k \mathbf{m}_k^{\top}\mathbf{b}} \cdot \prod_{k\in[N]} \hat{g}^{c^k\alpha_k}\Big)^t\Big) \\
&= e\Big(\prod_{j\in A} g^{c^{N+1-j}\lambda_{j,A}}, \hat{g}^{c^j \mathbf{m}_j^{\top}\mathbf{b}} \cdot \hat{g}^{c^j\alpha_j} \cdot \prod_{k\in[N],k\neq j} \hat{g}^{c^k \mathbf{m}_k^{\top}\mathbf{b}} \cdot \prod_{k\in[N],k\neq j} \hat{g}^{c^k\alpha_k}\Big)^t \\
&= e\Big(\prod_{j\in A} g^{c^{N+1-j}\lambda_{j,A}}, \hat{g}^{c^j \mathbf{m}_j^{\top}\mathbf{b}}\Big)^t \cdot e\Big(\prod_{j\in A} g^{c^{N+1-j}\lambda_{j,A}}, \hat{g}^{c^j\alpha_j}\Big)^t \cdot \\
&\quad e\Big(\prod_{j\in A} g^{c^{N+1-j}\lambda_{j,A}}, \prod_{k\in[N],k\neq j} \hat{g}^{c^k \mathbf{m}_k^{\top}\mathbf{b}}\Big)^t \cdot e\Big(\prod_{j\in A} g^{c^{N+1-j}\lambda_{j,A}}, \prod_{k\in[N],k\neq j} \hat{g}^{c^k\alpha_k}\Big)^t.
\end{aligned}$$

Step 5: Quad-Decomposition of Toeplitz Matrix and Final Fusion. When the pairing operation from Step 4 is fully expanded over the index space $(j,k) \in A \times [N]$, it yields a heavy two-dimensional algebraic matrix structure. By strategically sorting the resulting exponents based on the algebraic alignment ($j = k$ vs $j \neq k$), we separate the matrix into two diagonal components ($\mathbf{Diag}_1$ and $\mathbf{Diag}_2$) and two off-diagonal components ($\mathbf{OffDiag}_1, \mathbf{OffDiag}_2$):

$$\begin{aligned}
\mathbf{Diag}_1 &= e\Big(\prod_{j\in A} g^{c^{N+1-j}\lambda_{j,A}}, \hat{g}^{c^j \mathbf{m}_j^{\top}\mathbf{b}}\Big)^t = \prod_{j\in A} e(g,\hat{g})^{c^{N+1}(\mathbf{m}_j^{\top}\mathbf{b}\cdot\lambda_{j,A})t} = e(g,\hat{g})^{c^{N+1}\beta t}, \\
\mathbf{Diag}_2 &= e\Big(\prod_{j\in A} g^{c^{N+1-j}\lambda_{j,A}}, \hat{g}^{c^j\alpha_j}\Big)^t = \prod_{j\in A} e(g,\hat{g})^{c^{N+1}\alpha_j\lambda_{j,A}t}, \\
\mathbf{OffDiag}_1 &= e\Big(\prod_{j\in A} g^{c^{N+1-j}\lambda_{j,A}}, \prod_{k\in[N],k\neq j} \hat{g}^{c^k \mathbf{m}_k^{\top}\mathbf{b}}\Big)^t = e\Big(g^t, \prod_{j\in A}\Big(\prod_{k\in[N],k\neq j} \hat{g}^{c^{N+1-j+k} \mathbf{m}_k^{\top}\mathbf{b}}\Big)^{\lambda_{j,A}}\Big) \\
&= e\Big(g^t, \prod_{j\in A}\big(\hat{g}^{v_{j,0}}\big)^{\lambda_{j,A}}\Big),
\end{aligned}$$

$$\mathbf{OffDiag}_2 = e\Big(\prod_{j\in A} g^{c^{N+1-j}\lambda_{j,A}}, \prod_{k\in[N],k\neq j} \hat{g}^{c^k\alpha_k}\Big)^t = e\Big(g^t, \prod_{j\in A}\Big(\prod_{k\in[N],k\neq j} \hat{g}^{c^{N+1-j+k}\alpha_k}\Big)^{\lambda_{j,A}}\Big)$$
$$= e\Big(g^t, \prod_{j\in A}\Big(\prod_{k\in[N],k\neq j} A_{k,N+1-j+k}\Big)^{\lambda_{j,A}}\Big).$$

The ultimate reconstruction formula in Decaps synthesizes these four components under the following elimination logic:

- $\mathbf{Diag}_1$ is strictly preserved to derive the targeted session capsule key Ω^t.
- $\mathbf{Diag}_2$ is completely canceled by the accumulated committee decryption shares from Step 2, mapped via the term $\prod_{j\in A} E_j^{-\lambda_{j,A}}$.
- The combined structural off-diagonal matrices ($\mathbf{OffDiag}_1 \cdot \mathbf{OffDiag}_2$) map perfectly to the preprocessed public matrix keys, and are entirely canceled out by the pairing element $e\big(C_1, \prod_{j\in A} A_j^{\lambda_{j,A}}\big)^{-1}$.

Consequentially, all non-kernel entities successfully cancel out to the identity element, yielding: $\mathsf{CK} = \Omega^t$ which establishes the correctness of decryption. □

B A Simple Construction of O-TBIBE-SS

In this section, we describe a simple construction of the O-TBIBE-SS scheme by natively adopting the outsourced decryption framework with complete re-randomization introduced by Lee [22] in the design of O-BIBE schemes. Except for the three algorithms dedicated to the outsourcing mechanism, all remaining algorithms strictly coincide with the specifications defined in Section 3.2. Here, we focus on formalizing the outsourcing-related algorithms, namely Delegate, Transform, and Recover.

The core intuition behind this simple construction is that the Delegate algorithm re-randomizes the group elements $V_{j,1}$ and $V_{j,2}$ during the transformation key (TK) generation using the decryption key shares of the nodes, while simultaneously selecting an additional random exponent $y_{j,1}$ to re-randomize the exponent value y_j. Since the resulting TK provides perfect algebraic independence from the decryption key shares of the nodes, it greatly simplifies the security reduction. However, this methodology introduces a structural limitation: the cloud server cannot eliminate the additional random exponent $y_{j,1}$ embedded within the TK during the execution of the Transform algorithm. As a result, the cloud server is forced to provide the element valuations $\{E_{i,j,1}\}$ for each individual node j. This requirement inevitably causes the size of the transformed tasks submitted by the cloud to scale up to $O(NB)$, creating a critical performance bottleneck that increases proportionally with both the number of decryption nodes N and the ciphertext batch size B.

The precise formal specifications of these outsourcing algorithms are detailed as follows.

O-TBIBE-SS.Delegate$(\mathsf{PK}, (\mathsf{DK}_j)_{j\in A}, (\mathsf{CH}_i)_{i\in U}, \mathsf{BL})$**:** Let $\mathsf{DK}_j = (K_{j,1}, K_{j,2}, y_j)$ for $j \in A$ and $\mathsf{CH}_i = (C_{i,1}, C_{i,2}, C_{i,3}, C_{i,4})$ for $i \in U$ where $|A| = T$ and $U \subseteq [B]$.

1. It computes the interpolation vector $\boldsymbol{\lambda}_A = (\lambda_{j,A})_{j\in[N]} \in \mathbb{Z}_p^N$ such that $\boldsymbol{\lambda}_A^\top \mathbf{M} = \mathbf{e}_1^\top$ and $\lambda_{j,A} = 0$ for all $j \notin A$ where $\mathbf{M}$ is the share-generation matrix for the T-out-of-N threshold policy.
2. It samples $y_{j,1} \in \mathbb{Z}_p^*$ and $z_{j,1}, z_{j,2} \in \mathbb{Z}_p$ uniformly at random for each $j \in A$ and sets a recover key $\mathsf{RK} = (\{z_{j,1},\ \lambda_{j,A}/y_{j,1} \mod p\}_{j\in A})$. It then constructs a transformation key $\mathsf{TK} = (\{V_{j,1}, V_{j,2}, y_{j,n}\}_{j\in A})$ by computing, for all $j \in A$:

$$V_{j,1} = K_{j,1}^{y_{j,1}} \cdot \hat{g}^{z_{j,1}} \cdot (\hat{u}^{\mathsf{BL}}\hat{h})^{z_{j,2}},\ V_{j,2} = K_{j,2}^{y_{j,1}} \hat{g}^{z_{j,2}}, \quad \text{and} \quad y_{j,n} = y_j \cdot y_{j,1} \mod p.$$

3. It sets a work task $\mathsf{WT}_i = (D_{i,1} = C_{i,1}, D_{i,2} = C_{i,2}, D_{i,3} = C_{i,3}, D_{i,4} = C_{i,4})$ for each $i \in U$.
4. It outputs $(\mathsf{RK},\ \mathsf{TK},\ (\mathsf{WT}_i)_{i\in U})$.

O-TBIBE-SS.Transform$(\mathsf{PK}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in U}, (\mathsf{ID}_i)_{i\in U}, \mathsf{BL})$**:** Let $\mathsf{TK} = (\{V_{j,1}, V_{j,2}, y_{j,n}\}_{j\in A})$ for $j \in A$ and $\mathsf{WT}_i = (D_{i,1}, D_{i,2}, D_{i,3}, D_{i,4})$ for $i \in U$. It sets $S = \{\mathsf{ID}_i\}_{i\in U}$.

1. It computes the interpolation vector $\boldsymbol{\lambda}_A = (\lambda_{j,A})_{j\in[N]} \in \mathbb{Z}_p^N$ such that $\boldsymbol{\lambda}_A^\top \mathbf{M} = \mathbf{e}_1^\top$ and $\lambda_{j,A} = 0$ for all $j \notin A$ where $\mathbf{M}$ is the share-generation matrix for the T-out-of-N threshold policy.
2. It defines polynomials $F_S(x) = \prod_{\mathsf{ID}_k\in S}(x - \mathsf{ID}_k) = \sum_{k\in[0,|S|]} f_k x^k$ and $F_{S\setminus\{\mathsf{ID}_i\}}(x) = \prod_{\mathsf{ID}_k\in S\setminus\{\mathsf{ID}_i\}}(x - \mathsf{ID}_k) = \sum_{k\in[0,|S|-1]} \tilde{f}_{i,k} x^k$ for $i \in U$. Then, it computes the witnesses $\pi_{i,j}$ and ϕ_j that satisfy

$$\pi_{i,j} = \prod_{k\in[0,|S|-1]} \left(g^{\tau^k c^{N+1-j}}\right)^{\tilde{f}_{i,k}} = g^{F_{S\setminus\{\mathsf{ID}_i\}}(\tau)\cdot c^{N+1-j}} \quad \forall i \in U, j \in A \quad \text{and}$$
$$\phi_j = \prod_{k\in[0,|S|]} \left(W_{k,j}\right)^{f_k} = \prod_{k\in[0,|S|]} \left(\hat{g}^{\tau^k d_j}\right)^{f_k} = \hat{g}^{F_S(\tau)\cdot d_j} \quad \forall j \in A.$$

3. It pre-computes the following fixed components that depend only on the set A as

$$T_{A,1} = \prod_{j\in A} \left(g^{c^{N+1-j}}\right)^{\lambda_{j,A}} \quad \text{and} \quad T_{A,2} = \prod_{j\in A} A_j^{\lambda_{j,A}}.$$

4. For each $i \in U$, it proceeds as follows:
   (a) For each $j \in A$, it computes a node-dependent element

$$E_{i,j,1} = e(D_{i,1}, V_{j,1}) \cdot \left(e(\pi_{i,j}, D_{i,2}) \cdot e(D_{i,1}, \phi_j)^{-1}\right)^{-y_{j,n}} \cdot e(D_{i,3}, V_{j,2})^{-1}.$$

   (b) It constructs the transformed task $\mathsf{TT}_i = (E_{i,1}, \{E_{i,j,1}\}_{j\in A}, E_{i,2})$, where the task elements $E_{i,1}$ and $E_{i,2}$ are computed as

$$E_{i,1} = e\left(T_{A,1}, D_{i,4}\right) \cdot e\left(D_{i,1}, T_{A,2}\right)^{-1} \quad \text{and} \quad E_{i,2} = e(D_{i,1}, \hat{g}).$$

5. It outputs transformed tasks $(\mathsf{TT}_i)_{i\in U}$.

O-TBIBE-SS.Recover$(\mathsf{RK}, (\mathsf{TT}_i)_{i\in U})$**:** Let $\mathsf{RK} = (\{z_{j,1}, \lambda_{j,A}/y_{j,1}\}_{j\in A})$ and $\mathsf{TT}_i = (E_{i,1}, \{E_{i,j,1}\}_{j\in A}, E_{i,2})$ for $i \in U$.

1. For each $i \in U$, it recovers the capsule key $\mathsf{CK}_i = E_i$ by computing

$$E_i = E_{i,1} \cdot \prod_{j\in A} \left(E_{i,j,1} \cdot E_{i,2}^{-z_{j,1}}\right)^{-\lambda_{j,A}/y_{j,1}}.$$

2. It outputs the capsule keys $(\mathsf{CK}_i)_{i\in U}$.

Theorem B.1. *The simple O-TBIBE-SS scheme satisfies SE-SCM-CPA security for any polynomial batch size B if the underlying TBIBE-SS-KEM scheme is SE-SCM-CPA secure.*

Proof. The SE-SCM-CPA security of the simple O-TBIBE-SS scheme is established through a sequence of hybrid games $\mathbf{G}_0, \mathbf{G}_1, \mathbf{G}_2$, and $\mathbf{G}_3$, adapted from the security framework of Theorem 3.6. Crucially, except for the initial transition from $\mathbf{G}_0$ to $\mathbf{G}_1$, the entire sequence is executed identically to the reduction proof in Theorem 3.6.

In the transition from $\mathbf{G}_0$ to $\mathbf{G}_1$, the simulator alters the handling of the two oracles: the $\mathsf{OCreateKeyShare}$ oracle bypasses the generation of decryption key shares for the challenge domain and instead fills them with the symbol $\perp$ for lazy evaluation, while the $\mathsf{ODelegate}$ oracle responds to queries for the challenge domain by substituting the transformation key with uniformly sampled random elements. As shown in Lemma B.2, this initial transition induces an identical distribution. Since the remaining transitions from $\mathbf{G}_1$ to $\mathbf{G}_2$ and subsequently to $\mathbf{G}_3$ are the same as those in the reduction of Theorem 3.6, the cumulative advantage of the adversary across these hybrid games is strictly bounded by the advantage of the underlying TBIBE-SS-KEM scheme. Thus, the overall advantage of any PPT adversary against the simple construction collapses to a negligible function, completing the proof. □

Lemma B.2. *The games $\mathbf{G}_0$ and $\mathbf{G}_1$ are statistically indistinguishable.*

Proof. The proof establishes that the transition from $\mathbf{G}_0$ to $\mathbf{G}_1$ introduces no statistical divergence in the view of the adversary $\mathcal{A}$. For all oracle queries falling into non-challenge domains ($\mathsf{BL} \neq \mathsf{BL}^*$ or ($\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \notin S$)), the simulator $\mathcal{B}$ follows the exact honest specifications. For the oracle query falling into the challenge domain ($\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \in S$), $\mathcal{B}$ employs a lazy evaluation and uniform sampling strategy, where the decryption key share in $\mathsf{OCreateKeyShare}$ is set as $\perp$ and the transformation key TK in $\mathsf{ODelegate}$ is emulated via uniform random sampling. The detailed procedures for each oracle in $\mathbf{G}_1$ are given as follows:

- $\mathsf{OCreateKeyShare}(j, S, \mathsf{BL})$: If $j \notin [N]$ or $j \in C_{\text{all}}$, it responds with $\perp$. Otherwise, if there exists $(j, S, \mathsf{BL}, \dots) \in L_{\text{dks}}$, it retrieves $(j, \cdot, \cdot, \mathsf{hk}, \cdot) \leftarrow L_{\text{dks}}$ associated with (j, S, BL) and responds with hk. Otherwise:
  1. If ($\mathsf{BL} \neq \mathsf{BL}^*$) or ($\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \notin S$): It computes $\mathsf{DK}_j \leftarrow \mathsf{ComputeKeyShare}(\mathsf{SK}_j, \mathsf{Digest}(\mathsf{PP}, S), \mathsf{BL})$ by retrieving $\mathsf{SK}_j \leftarrow L_{\text{sk}}$.
  2. Otherwise ($\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \in S$): It sets $\mathsf{DK}_j = \perp$.
  3. It picks a unique handle $\mathsf{hk} \in \{0,1\}^\lambda$, updates $L_{\text{dks}} \leftarrow L_{\text{dks}} \cup \{(j, S, \mathsf{BL}, \mathsf{hk}, \mathsf{DK}_j)\}$, and responds with hk.
- $\mathsf{ODelegate}((\mathsf{hk}_j)_{j \in A \setminus C_{\text{all}}}, (\mathsf{DK}_j)_{j \in A \cap C_{\text{all}}}, (\mathsf{CH}_i)_{i \in U})$: Let $(\mathsf{hk}_j)_{j \in A \setminus C_{\text{all}}}$ be handles of honest key shares and $(\mathsf{DK}_j)_{j \in A \cap C_{\text{all}}}$ be dishonest key shares where $A = (A \setminus C_{\text{all}}) \cup (A \cap C_{\text{all}}) \subseteq [N]$ and $|A| = T$. If there is no entry $(j, \cdot, \cdot, \mathsf{hk}_j, \cdot) \in L_{\text{dks}}$ for each $j \in A \setminus C_{\text{all}}$, it responds with $\perp$. Otherwise:
  1. For each node $j \in A \setminus C_{\text{all}}$, it retrieves $(j, S_j, \mathsf{BL}_j, \mathsf{hk}_j, \mathsf{DK}_j) \leftarrow L_{\text{dks}}$. Next, it checks that $(\mathsf{DK}_j)_{j \in A \setminus C_{\text{all}}}$ are associated with the same (S, BL). If any check fails, it responds with $\perp$.
  2. For each node $j \in A \cap C_{\text{all}}$, it checks that $1 = \mathsf{VerifyKeyShare}(\mathsf{PK}_j, \mathsf{DK}_j, \mathsf{Digest}(\mathsf{PP}, S), \mathsf{BL})$ by retrieving $\mathsf{PK}_j \leftarrow L_{\text{sk}}$. If any check fails, it responds with $\perp$.
  3. If ($\mathsf{BL} \neq \mathsf{BL}^*$) or ($\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \notin S$): It computes $(\mathsf{RK}, \mathsf{TK}, (\mathsf{WT}_i)_{i \in U}) \leftarrow \mathsf{Delegate}\big(\mathsf{PK}, (\mathsf{DK}_j)_{j \in A}, (\mathsf{CH}_i)_{i \in U}, \mathsf{BL}\big)$.
  4. Otherwise ($\mathsf{BL} = \mathsf{BL}^* \wedge \mathsf{ID}^* \in S$):
     (a) It samples $\{V_{j,1}, V_{j,2}\}_{j \in A} \in \hat{\mathbb{G}}^{2|A|}$ and $\{y_{j,n}\} \in (\mathbb{Z}_p^*)^{|A|}$ uniformly at random.

(b) It sets $\mathsf{RK} = \perp$ and $\mathsf{TK} = (\{V_{j,1}, V_{j,2}, y_{j,n}\}_{j \in A})$. It derives $(\mathsf{WT}_i)_{i \in U}$ from $(\mathsf{CH}_i)_{i \in U}$ accordingly.

5. It picks a unique handle $\mathsf{hd} \in \{0,1\}^\lambda$, updates $L_{\mathrm{del}} \leftarrow L_{\mathrm{del}} \cup \{(\mathsf{hd}, \mathsf{RK})\}$, and responds with $(\mathsf{hd}, \mathsf{TK}, (\mathsf{WT}_i)_{i \in U})$.

To establish the perfect statistical indistinguishability between the honest execution and the random emulation in the challenge domain, we analyze the structure of the simulated components. For each node $j \in A$, the relationship between the emulated elements $(V_{j,1}, V_{j,2}, y_{j,n}) \in \hat{\mathbb{G}}^2 \times \mathbb{Z}_p$ and the underlying randomized exponents $(y_{j,1}, z_{j,1}, z_{j,2}) \in \mathbb{Z}_p^3$ can be explicitly described in the discrete logarithm domain as follows:

$$\begin{pmatrix} \log_{\hat{g}}(V_{j,1}) \\ \log_{\hat{g}}(V_{j,2}) \\ y_{j,n} \end{pmatrix} = \begin{pmatrix} \log_{\hat{g}}(K_{j,1}) & 1 & \log_{\hat{g}}(\hat{u}^{\mathsf{BL}}\hat{h}) \\ \log_{\hat{g}}(K_{j,2}) & 0 & 1 \\ y_j & 0 & 0 \end{pmatrix} \begin{pmatrix} y_{j,1} \\ z_{j,1} \\ z_{j,1} \end{pmatrix}.$$

Let $\mathbf{A} \in \mathbb{Z}_p^{3\times 3}$ denote the transformation matrix specified above. It immediately follows that the determinant of the matrix is $\det(\mathbf{A}) = -y_j \pmod p$ and $y_j \in \mathbb{Z}_p^*$, implying that $\mathbf{A}$ is strictly invertible over $\mathbb{Z}_p$. Due to this invertibility, the linear mapping induced by $\mathbf{A}$ constitutes a perfect bijection between the exponent vector $(y_{j,1}, z_{j,1}, z_{j,2})$ and the discrete logarithm space of the emulated vector $(\log_{\hat{g}}(V_{j,1}), \log_{\hat{g}}(V_{j,2}), y_{j,n})$. When the exponents $y_{j,1}, z_{j,1}$, and $z_{j,2}$ are chosen uniformly at random from $\mathbb{Z}_p$, the resulting elements $V_{j,1}$, $V_{j,2}$, and $y_{j,n}$ are distributed uniformly at random. This bijection guarantees that the view of $\mathcal{A}$ in $\mathbf{G}_1$ introduces zero divergence from $\mathbf{G}_0$. This completes the proof. □

C Security Proofs for VOBTE-SS

C.1 Proof of Theorem 4.2 (SE-SCM-CCA Security)

Proof. To prove the SE-SCM-CCA security of the VOBTE-SS scheme, we employ the hybrid game technique. We define a sequence of hybrid games $\mathbf{G}_0, \mathbf{G}_1, \cdots, \mathbf{G}_6$ where the first game will be the original SE-SCM-CCA security game and the last one will be a game in which an adversary $\mathcal{A}$ has no advantage. Let $\mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_j}$ be the advantage of $\mathcal{A}$ in the game $\mathbf{G}_j$. The hybrid games are defined as follows:

Game $\mathbf{G}_0$. This game corresponds to the original SE-SCM-CCA security experiment in Definition 4.3. The simulator of this game uses the secret key $\mathsf{SK}_j = \mathsf{SK}_{j,OSS}$ for each honest node $j \in [N] \setminus C_{\mathrm{all}}$ to handle all queries of $\mathcal{A}$. By the definition, it follows that $\mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_0} = \mathsf{Adv}^{\mathsf{SE\text{-}SCM\text{-}CCA}}_{VOBTE\text{-}SS,\mathcal{A}}(\lambda)$.

Game $\mathbf{G}_1$. This game is identical to $\mathbf{G}_0$ except for the handling of $\mathsf{OCreatePreDecrypt}$ and $\mathsf{ODelegate}$ queries. The simulator picks a challenge OTS key pair $(\mathsf{SK}^*_{OTS}, \mathsf{VK}^*_{OTS})$ for the challenge ciphertext at the beginning of the game. Let BAD_1 be the event that the adversary $\mathcal{A}$ submits a query containing a ciphertext $\mathsf{CT}_i = (\mathsf{VK}_{i,OTS}, \mathsf{CH}_{i,OSS}, \mathsf{CT}_{i,SKE}, \eta_i, \sigma_i)$ for some $i \in [B]$ such that

$$\mathsf{VK}_{i,OTS} = \mathsf{VK}^*_{OTS} \wedge (\mathsf{BL}, \mathsf{CH}_{i,OSS}, \mathsf{CT}_{i,SKE}, \eta_i) \neq (\mathsf{BL}^*, \mathsf{CH}^*_{OSS}, \mathsf{CT}^*_{SKE}, \eta^*) \wedge$$
$$\mathsf{OTS.Verify}(\mathsf{VK}^*_{OTS}, (\mathsf{BL} \| \mathsf{CH}_{i,OSS} \| \mathsf{CT}_{i,SKE} \| \eta_i), \sigma_i) = 1$$

where $\mathsf{CT}^* = (\mathsf{VK}^*_{OTS}, \mathsf{CH}^*_{OSS}, \mathsf{CT}^*_{SKE}, \eta^*, \sigma^*)$ is the challenge ciphertext. If BAD_1 occurs, it sets $\mathsf{BAD}_1 = 1$ and aborts the game. Since the probability of aborting in this game is bounded by the SUF of the OTS scheme, we have $\left|\mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_0} - \mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_1}\right| \leq \Pr[\mathsf{BAD}_1] \leq \mathsf{Adv}^{\mathsf{SUF}}_{OTS,\mathcal{B}}(\lambda)$.

Game $\mathbf{G}_2$. This game is identical to $\mathbf{G}_1$ except that the simulator additionally aborts in $\mathsf{OCreatePreDecrypt}$ and $\mathsf{ODelegate}$ queries. Let BAD_2 be the event that the adversary $\mathcal{A}$ submits a query containing $\mathsf{VK}_{i,OTS}$ such that $\mathsf{VK}_{i,OTS} \neq \mathsf{VK}^*_{OTS} \wedge \mathsf{H}_s(\mathsf{VK}_{i,OTS}) = \mathsf{H}_s(\mathsf{VK}^*_{OTS})$. If BAD_2 occurs, it sets $\mathsf{BAD}_2 = 1$ and aborts the game. Since the probability of this aborting is bounded by the collision resistance of the hash function, we have $\left|\mathsf{Adv}^{\mathbf{G}_1}_{\mathcal{A}} - \mathsf{Adv}^{\mathbf{G}_2}_{\mathcal{A}}\right| \leq \Pr[\mathsf{BAD}_2] \leq \mathsf{Adv}^{\mathsf{CR}}_{HF,\mathcal{B}}(\lambda)$.

Game $\mathbf{G}_3$. In this game, the simulator no longer uses the honest secret key $\mathsf{SK}_j = \mathsf{SK}_{j,OSS}$. Instead, it simulates the VOBTE-SS oracles by invoking the O-TBIBE-SS oracles.

Specifically, for queries under the challenge label BL^* where $\mathsf{CT}^* \notin (\mathsf{CT}_i)_{i\in[B]}$, the condition $\mathsf{ID}^* \notin S$ is strictly guaranteed because the events BAD_1 (reuse of the challenge verification key) and BAD_2 (hash collision) did not occur in the previous games. This ensures that all such queries remain valid within the O-TBIBE-SS security model and can be perfectly simulated by the O-TBIBE-SS challenger. Due to the correctness of the underlying O-TBIBE-SS scheme, the distribution of responses is identical to that in $\mathbf{G}_2$. Thus, we have $\mathsf{Adv}^{\mathbf{G}_3}_{\mathcal{A}} = \mathsf{Adv}^{\mathbf{G}_2}_{\mathcal{A}}$.

Game $\mathbf{G}_4$. This game is identical to $\mathbf{G}_3$ except that the capsule key CK^*_{OSS} of the challenge ciphertext CT^* is replaced by a random capsule key $\widetilde{\mathsf{CK}} \in \mathbb{G}_T$. Since this change relies on the SE-SCM-CPA security of the O-TBIBE-SS scheme from Lemma C.3, we have $\left|\mathsf{Adv}^{\mathbf{G}_3}_{\mathcal{A}} - \mathsf{Adv}^{\mathbf{G}_4}_{\mathcal{A}}\right| \leq \mathsf{Adv}^{\mathsf{SE\text{-}SCM\text{-}CPA}}_{O\text{-}TBIBE\text{-}SS,\mathcal{B}}(\lambda)$.

Game $\mathbf{G}_5$. In the challenge ciphertext CT^*, the simulator replaces the PRF outputs $(K^*\|\eta^*) = \mathsf{PRF}(\widetilde{\mathsf{CK}}, \mathsf{ID}^*\|\mathsf{BL}^*)$ with a random string $(\widetilde{K}\|\widetilde{\eta})$ chosen uniformly at random. Since this transition relies on the pseudo-randomness of PRF, we have $\left|\mathsf{Adv}^{\mathbf{G}_4}_{\mathcal{A}} - \mathsf{Adv}^{\mathbf{G}_5}_{\mathcal{A}}\right| \leq \mathsf{Adv}^{\mathsf{PR}}_{PRF,\mathcal{B}}(\lambda)$.

Game $\mathbf{G}_6$. In the final game, the simulator replaces the challenge SKE ciphertext $\mathsf{CT}^*_{SKE} \leftarrow \mathsf{SKE.Encrypt}(\widetilde{K}, M^*_\mu)$ with an encryption of a constant zero-message $\mathsf{CT}^*_{SKE} \leftarrow \mathsf{SKE.Encrypt}(\widetilde{K}, 0^{|M|})$. Since $\widetilde{K}$ is a fresh random key unknown to $\mathcal{A}$, this transition is bounded by the IND-OT security of the SKE scheme $\left|\mathsf{Adv}^{\mathbf{G}_5}_{\mathcal{A}} - \mathsf{Adv}^{\mathbf{G}_6}_{\mathcal{A}}\right| \leq \mathsf{Adv}^{\mathsf{IND\text{-}OT}}_{SKE,\mathcal{B}}(\lambda)$. Since the challenge ciphertext CT^* in the final game is not related to the random coin μ, we have $\mathsf{Adv}^{\mathbf{G}_6}_{\mathcal{A}} = 0$.

By applying the advantage differences established in Lemmas C.1, C.2, C.3, C.4, and C.5, we obtain the following upper bound on the total advantage:

$$
\begin{aligned}
&\mathsf{Adv}^{\mathsf{SE\text{-}SCM\text{-}CCA}}_{VOBTE\text{-}SS,\mathcal{A}}(\lambda) \\
&\leq \sum_{j=1}^{6} \left|\mathsf{Adv}^{\mathbf{G}_{j-1}}_{\mathcal{A}} - \mathsf{Adv}^{\mathbf{G}_j}_{\mathcal{A}}\right| + \mathsf{Adv}^{\mathbf{G}_6}_{\mathcal{A}} \\
&\leq \mathsf{Adv}^{\mathsf{SUF}}_{OTS,\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{CR}}_{HF,\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{SE\text{-}SCM\text{-}CPA}}_{O\text{-}TBIBE\text{-}SS,\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{PR}}_{PRF,\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{IND\text{-}OT}}_{SKE,\mathcal{B}}(\lambda).
\end{aligned}
$$

This completes the proof. □

Lemma C.1. *The games $\boldsymbol{G}_0$ and $\boldsymbol{G}_1$ are computationally indistinguishable, assuming that the OTS scheme is SUF secure.*

Proof. The indistinguishability between $\mathbf{G}_0$ and $\mathbf{G}_1$ is conditioned strictly on the occurrence of the event BAD_1. Because the execution of these two games is identical unless BAD_1 is triggered, the Difference Lemma guarantees that $|\mathsf{Adv}^{\mathbf{G}_0}_{\mathcal{A}} - \mathsf{Adv}^{\mathbf{G}_1}_{\mathcal{A}}| \leq \Pr[\mathsf{BAD}_1]$.

To bound this probability, we construct a reduction simulator $\mathcal{B}$ that leverages the occurrence of BAD_1 to break the SUF security of the underlying OTS scheme. At the initialization phase, $\mathcal{B}$ receives a target verification key VK^*_{OTS} from the external OTS challenger. $\mathcal{B}$ then establishes the execution environment by running $\mathsf{VOBTE\text{-}SS.Setup}$ to generate the public parameters PP, and subsequently executes $\mathsf{VOBTE\text{-}SS.KeyGen}$ to generate the non-malicious key-pairs $\{(\mathsf{SK}_j, \mathsf{PK}_j)\}_{j\in[N]\setminus C_{\text{mal}}}$. Throughout the query phase, $\mathcal{B}$ perfectly emulates the $\mathsf{OCreatePreDecrypt}$ oracle queries initiated by $\mathcal{A}$ using the honestly generated key shares $\{\mathsf{SK}_j\}$. Upon entering the challenge phase, $\mathcal{B}$ constructs the target challenge ciphertext $\mathsf{CT}^* = (\mathsf{VK}^*_{OTS}, \mathsf{CH}^*_{OSS}, \mathsf{CT}^*_{SKE}, \eta^*, \sigma^*)$, where the OTS signature σ^* is obtained from the OTS signing oracle on the message string $(\mathsf{BL}^* \| \mathsf{CH}^*_{OSS} \| \mathsf{CT}^*_{SKE} \| \eta^*)$.

If the event BAD_1 occurs during the subsequent oracle query phase, it implies that $\mathcal{A}$ has successfully submitted a valid ciphertext $\mathsf{CT}_i = (\mathsf{VK}_{i,OTS}, \mathsf{CH}_{i,OSS}, \mathsf{CT}_{i,SKE}, \eta_i, \sigma_i)$ for some index $i \in [B]$ such that $\mathsf{VK}_{i,OTS} = \mathsf{VK}^*_{OTS}$ but $\mathsf{CT}_i \neq \mathsf{CT}^*$. Since CT^* is the unique ciphertext signed under VK^*_{OTS} by the simulator, such a submission CT_i inherently contains a valid, non-trivial OTS signature forgery for VK^*_{OTS}. Consequently, $\mathcal{B}$ can immediately output $(\mathsf{BL}_i \| \mathsf{CH}_{i,OSS} \| \mathsf{CT}_{i,SKE} \| \eta_i, \sigma_i)$ as a successful forgery against the OTS primitive, which directly implies $\Pr[\mathsf{BAD}_1] \leq \mathsf{Adv}^{\mathsf{SUF}}_{OTS,\mathcal{B}}(\lambda)$. This completes the proof. □

Lemma C.2. *The games $\boldsymbol{G}_1$ and $\boldsymbol{G}_2$ are computationally indistinguishable, assuming that the hash function is collision-resistant.*

Proof. The execution of games $\mathbf{G}_1$ and $\mathbf{G}_2$ is identical unless the event BAD_2 is triggered. By the Difference Lemma, the divergence between the adversary's advantages across these two games is strictly bounded as $|\mathsf{Adv}^{\mathbf{G}_1}_{\mathcal{A}} - \mathsf{Adv}^{\mathbf{G}_2}_{\mathcal{A}}| \leq \Pr[\mathsf{BAD}_2]$.

We construct a reduction simulator $\mathcal{B}$ that exploits the occurrence of BAD_2 to find a non-trivial collision in the underlying hash function. At the initialization step, $\mathcal{B}$ receives a target hash seed s for the instance H_s from the external HF challenger. $\mathcal{B}$ then establishes the simulation environment by executing $\mathsf{VOBTE\text{-}SS.Setup}$ and $\mathsf{VOBTE\text{-}SS.KeyGen}$ to generate the public parameters and the set of non-malicious key-pairs, respectively. Utilizing these secret keys, $\mathcal{B}$ emulates all oracle queries initiated by the adversary $\mathcal{A}$. Additionally, $\mathcal{B}$ generates the target OTS key-pair $(\mathsf{SK}^*_{OTS}, \mathsf{VK}^*_{OTS})$ for the challenge ciphertext via $\mathsf{OTS.KeyGen}$ and computes the target hash identity as $\mathsf{ID}^* = \mathsf{H}_s(\mathsf{VK}^*_{OTS})$.

If $\mathcal{A}$ triggers the event BAD_2 during the oracle query phase, it implies the submission of a valid ciphertext CT_i associated with a derived identity ID_i and a verification key $\mathsf{VK}_{i,OTS}$ such that $\mathsf{ID}_i = \mathsf{ID}^*$ but $\mathsf{VK}_{i,OTS} \neq \mathsf{VK}^*_{OTS}$. By definition, the identity relations yield $\mathsf{H}_s(\mathsf{VK}_{i,OTS}) = \mathsf{ID}_i = \mathsf{ID}^* = \mathsf{H}_s(\mathsf{VK}^*_{OTS})$, which directly gives the relation $\mathsf{H}_s(\mathsf{VK}_{i,OTS}) = \mathsf{H}_s(\mathsf{VK}^*_{OTS})$. Given the guarantee that $\mathsf{VK}_{i,OTS} \neq \mathsf{VK}^*_{OTS}$, $\mathcal{B}$ can immediately output $(\mathsf{VK}_{i,OTS}, \mathsf{VK}^*_{OTS})$ as a successful collision pair against the HF primitive. Therefore, the probability of the event is bounded as $\Pr[\mathsf{BAD}_2] \leq \mathsf{Adv}^{\mathsf{CR}}_{HF,\mathcal{B}}(\lambda)$, completing the proof. □

Lemma C.3. *The games $\boldsymbol{G}_3$ and $\boldsymbol{G}_4$ are computationally indistinguishable, assuming that the underlying O-TBIBE-SS scheme is SE-SCM-CPA secure.*

Proof. Suppose there exists an adversary $\mathcal{A}$ that breaks the SE-SCM-CCA security of the VOBTE-SS scheme with a non-negligible advantage. Let $\mathcal{C}$ be an O-TBIBE-SS challenger. A simulator $\mathcal{B}$ that breaks the SE-SCM-CPA security of the O-TBIBE-SS scheme by using $\mathcal{A}$ is described as follows:

Init: $\mathcal{A}$ submits a challenge batch label BL^*, the decryption committee size N, the threshold $T \leq N$, and the subsets of corrupted and malicious nodes $C_{\text{cor}}, C_{\text{mal}}$. $\mathcal{B}$ selects a hash function $\mathsf{H}_s \leftarrow \mathcal{H}$, generates $(\mathsf{SK}^*_{OTS}, \mathsf{VK}^*_{OTS}) \leftarrow \mathsf{OTS.KeyGen}(1^\lambda)$ for the challenge ciphertext, and computes $\mathsf{ID}^* = \mathsf{H}_s(\mathsf{VK}^*_{OTS})$. It forwards $(\mathsf{ID}^*, \mathsf{BL}^*, N, T, C_{\text{cor}}, C_{\text{mal}})$ to $\mathcal{C}$.

Setup: $\mathcal{B}$ receives $(\mathsf{PP}_{OSS}, (\mathsf{SK}_{j,OSS})_{j\in C_{\mathrm{cor}}}, (\mathsf{PK}_{j,OSS})_{j\in[N]\setminus C_{\mathrm{mal}}}, (\mathsf{HT}_{j,OSS})_{j\in[N]\setminus C_{\mathrm{mal}}})$ from $\mathcal{C}$. $\mathcal{B}$ proceeds as follows:

1. It runs the $\mathsf{OInit}()$ oracle to initialize lists L_{sk}, L_{pdk}, and L_{del}. It sets $\mathsf{PP} = (\mathsf{PP}_{OSS}, \mathsf{H}_s)$, $\mathsf{SK}_j = \mathsf{SK}_{j,OSS}$ for $j \in C_{\mathrm{cor}}$, and $\mathsf{PK}_j = (\mathsf{PK}_{j,OSS}, \mathsf{HT}_{j,OSS})$ for $j \in [N] \setminus C_{\mathrm{mal}}$.
2. Next, it updates $L_{\mathrm{sk}} \leftarrow L_{\mathrm{sk}} \cup \{(j, \perp, \mathsf{PK}_j)\}_{j\in[N]\setminus C}$ and gives $\big(\mathsf{PP}, (\mathsf{SK}_j)_{j\in C_{\mathrm{cor}}}, (\mathsf{PK}_j)_{j\in[N]\setminus C_{\mathrm{mal}}}\big)$ to $\mathcal{A}$.

Preprocess: $\mathcal{A}$ submits public keys $(\mathsf{PK}_j = (\mathsf{PK}_{j,OSS}, \mathsf{HT}_{j,OSS}))_{j\in C_{\mathrm{mal}}}$ of the malicious nodes. $\mathcal{B}$ proceeds as follows:

1. It checks that $1 = \mathsf{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j)$ for each $j \in C_{\mathrm{mal}}$. If any check fails, it outputs 0 and aborts the experiment. Otherwise, it updates $L_{\mathrm{sk}} \leftarrow L_{\mathrm{sk}} \cup \{(j, \perp, \mathsf{PK}_j)\}_{j\in C_{\mathrm{mal}}}$.
2. It submits $(\mathsf{PK}_{j,OSS}, \mathsf{HT}_{j,OSS})_{j\in C_{\mathrm{mal}}}$ to $\mathcal{C}$ and receives the O-TBIBE-SS challenge ciphertext pair $(\mathsf{CH}^*_{OSS}, \mathsf{CK}^*_{OSS})$.
3. Next, it runs $(\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}(\mathsf{PP}, (\mathsf{PK}_j)_{j\in[N]})$ and gives $(\mathsf{EK}, \mathsf{PK})$ to $\mathcal{A}$.

Pre-challenge Queries: $\mathcal{B}$ handles the queries as follows:

- $\mathsf{OCreatePreDecrypt}(j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$: Let $\mathsf{CT}_i = (\mathsf{VK}_{i,OTS}, \mathsf{CH}_{i,OSS}, \mathsf{CT}_{i,SKE}, \eta_i, \sigma_i)$ for $i \in [B]$. If $j \in C_{\mathrm{all}}$, it responds with $\perp$. If there exists $(j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}, \mathsf{hk}, \cdot) \in L_{\mathrm{pdk}}$, it responds with hk. Otherwise:
  1. It initializes $U = \emptyset$ and $S = \emptyset$, and for each $i \in [B]$, it proceeds as follows: If the OTS signature in CT_i is invalid, it skips to the next iteration. Otherwise, it computes $\mathsf{ID}_i = \mathsf{H}_s(\mathsf{VK}_{i,OTS})$ and updates $U \leftarrow U \cup \{i\}$, $S \leftarrow S \cup \{\mathsf{ID}_i\}$.
  2. It queries the $\mathsf{OCreateKeyShare}(j, S, \mathsf{BL})$ oracle of $\mathcal{C}$ and receives hk_{OSS}.
  3. It picks a unique handle $\mathsf{hk} \in \{0,1\}^\lambda$, updates $L_{\mathrm{pdk}} \leftarrow L_{\mathrm{pdk}} \cup \{(j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL}, \mathsf{hk}, \mathsf{hk}_{OSS})\}$, and responds with hk.
- $\mathsf{ORevealPreDecrypt}(\mathsf{hk})$: If there is no entry $(\cdot, \cdot, \cdot, \mathsf{hk}, \mathsf{hk}_{OSS}) \in L_{\mathrm{pdk}}$, it responds with $\perp$. Otherwise:
  1. It retrieves $\mathsf{hk}_{OSS} \leftarrow L_{\mathrm{pdk}}$. It queries the $\mathsf{ORevealKeyShare}(\mathsf{hk}_{OSS})$ oracle of $\mathcal{C}$ and receives $\mathsf{DK}_{j,OSS}$.
  2. It sets $\mathsf{DK}_j = \mathsf{DK}_{j,OSS}$ and responds with DK_j.
- $\mathsf{ODelegate}((\mathsf{hk}_j)_{j\in A\setminus C_{\mathrm{all}}}, (\mathsf{DK}_j)_{j\in A\cap C_{\mathrm{all}}}, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$: Let $\mathsf{CT}_i = (\mathsf{VK}_{i,OTS}, \mathsf{CH}_{i,OSS}, \mathsf{CT}_{i,SKE}, \eta_i, \sigma_i)$ for $i \in [B]$. If there is no entry $(\cdot, \cdot, \cdot, \mathsf{hk}_j, \mathsf{hk}_{j,OSS}) \in L_{\mathrm{pdk}}$ for each $j \in A \setminus C_{\mathrm{all}}$, it responds with $\perp$. Otherwise:
  1. It initializes $U = \emptyset$ and $S = \emptyset$, and for each $i \in [B]$, it proceeds as follows: If the OTS signature in CT_i is invalid, it skips to the next iteration. Otherwise, it computes $\mathsf{ID}_i = \mathsf{H}_s(\mathsf{VK}_{i,OTS})$ and updates $U \leftarrow U \cup \{i\}$, $S \leftarrow S \cup \{\mathsf{ID}_i\}$.
  2. For each $j \in A \setminus C_{\mathrm{all}}$, it retrieves $\mathsf{hk}_{j,OSS} \leftarrow L_{\mathrm{pdk}}$. Next, it checks that $(\mathsf{hk}_{j,OSS})_{j\in A\setminus C_{\mathrm{all}}}$ are associated with the same $((\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$. If any check fails, it responds with $\perp$.
  3. For each $j \in A \cap C_{\mathrm{all}}$, it checks that $1 = \mathsf{VerifyPreDecrypt}(\mathsf{PK}, \mathsf{DK}_j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$. If any check fails, it responds with $\perp$.

4. It queries the $\mathsf{ODelegate}((\mathsf{hk}_{j,OSS})_{j\in A\setminus C_{\text{all}}}, (\mathsf{DK}_{j,OSS})_{j\in A\cap C_{\text{all}}}, (\mathsf{CH}_{i,OSS})_{i\in U})$ oracle of $\mathcal{C}$ and receives $(\mathsf{hd}_{OSS}, \mathsf{TK}_{OSS}, (\mathsf{WT}_{i,OSS})_{i\in U})$. It sets $\mathsf{TK} = \mathsf{TK}_{OSS}$ and $\mathsf{WT}_i = (\mathsf{WT}_{i,OSS}, \mathsf{VK}_{i,OTS})$ for all $i \in U$.
5. It picks a unique handle $\mathsf{hd} \in \{0,1\}^\lambda$, updates $L_{\text{del}} \leftarrow L_{\text{del}} \cup \{(\mathsf{hd}, \mathsf{hd}_{OSS}, (\mathsf{CT}_{i,SKE}, \eta_i)_{i\in U}, \mathsf{BL})\}$, and responds with $(\mathsf{hd}, \mathsf{TK}, (\mathsf{WT}_i)_{i\in U})$.

- $\mathsf{OReveaIRT}(\mathsf{hd})$: If there is no entry $(\mathsf{hd}, \cdot, \cdot, \cdot) \in L_{\text{del}}$, it responds with $\perp$. Otherwise:
  1. It retrieves $(\mathsf{hd}, \mathsf{hd}_{OSS}, (\mathsf{CT}_{i,SKE}, \eta_i)_{i\in U}, \mathsf{BL}) \leftarrow L_{\text{del}}$. It queries the $\mathsf{OReveaIRK}(\mathsf{hk}_{OSS})$ oracle of $\mathcal{C}$ and receives RK_{OSS}.
  2. It sets $\mathsf{RT} = (\mathsf{RK}_{OSS}, (\mathsf{CT}_{i,SKE}, \eta_i)_{i\in U}, \mathsf{BL})$ and responds with RT.

Challenge: $\mathcal{A}$ submits two messages M_0^*, M_1^*, $\mathcal{B}$ proceeds as follows:

1. It derives $(K^* \| \eta^*) \leftarrow \mathsf{PRF}(\mathsf{CK}_{OSS}^*, \mathsf{ID}^* \| \mathsf{BL}^*)$. It flips a random coin $\mu \in \{0,1\}$ and computes $\mathsf{CT}_{SKE}^* \leftarrow \mathsf{SKE.Encrypt}(K^*, M_\mu^*)$. Next, it generates $\sigma^* \leftarrow \mathsf{OTS.Sign}(\mathsf{SK}_{OTS}^*, \mathsf{BL}^* \| \mathsf{CH}_{OSS}^* \| \mathsf{CT}_{SKE}^* \| \eta^*)$.
2. It sets $\mathsf{CT}^* = (\mathsf{VK}_{OTS}^*, \mathsf{CH}_{OSS}^*, \mathsf{CT}_{SKE}^*, \eta^*, \sigma^*)$ and gives CT^* to $\mathcal{A}$.

Post-challenge Queries: $\mathcal{B}$ responds to these queries in the same manner as the pre-challenge queries, subject to the restrictions of the security game.

Output: $\mathcal{A}$ outputs a guess μ'. If $\mu = \mu'$, then $\mathcal{B}$ outputs 1. Otherwise, it outputs 0.

Analysis of Simulation: The validity of the simulator $\mathcal{B}$ is grounded in the following points:

Challenge Privacy Preservation. For any $\mathsf{OCreatePreDecrypt}$ or $\mathsf{ODelegate}$ query that does not involve CT^* under the target label BL^*, the absence of the events BAD_1 and BAD_2 guarantees that $\mathsf{ID}^* \notin S$ holds for the underlying session. This algebraic alignment enables $\mathcal{B}$ to safely simulate subsequent reveal queries without any security violations. Consequently, the challenge privacy constraints of the VOBTE-SS security model directly ensure the challenge privacy constraints of the underlying O-TBIBE-SS scheme.

Label Distinctness and Coherence. The label distinctness constraints of the VOBTE-SS security model directly ensure the label coherence constraints of the underlying O-TBIBE-SS scheme for $\mathsf{OCreatePreDecrypt}$ and $\mathsf{ODelegate}$ queries involving the target label BL^*. Specifically, since the label distinctness rules prohibit $\mathcal{A}$ from submitting the identical index-label pair (j, BL^*) to multiple $\mathsf{OCreatePreDecrypt}$ queries or reusing BL^* across multiple $\mathsf{ODelegate}$ queries, $\mathcal{B}$ is guaranteed to invoke at most one $\mathsf{OCreateKeyShare}(j, S_j, \mathsf{BL}^*)$ query for each honest node j and at most one $\mathsf{ODelegate}$ query where all honest handles natively originate from the identical (S, BL^*).

Probability Reduction: If $\mathcal{C}$ provides a real capsule key CK_{OSS}^*, $\mathcal{A}$'s view is identical to $\mathbf{G}_3$. If $\mathcal{C}$ provides a truly random string, $\mathcal{A}$'s view corresponds to $\mathbf{G}_4$. Consequently, the advantage of $\mathcal{A}$ in distinguishing these two games is bounded by the advantage of $\mathcal{B}$ in breaking the SE-SCM-CPA security of the underlying O-TBIBE-SS scheme $\left|\mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_3} - \mathsf{Adv}_{\mathcal{A}}^{\mathbf{G}_4}\right| \leq \mathsf{Adv}_{O\text{-}TBIBE\text{-}SS,\mathcal{B}}^{\mathsf{SE\text{-}SCM\text{-}CPA}}(\lambda)$. This completes the proof. □

Lemma C.4. *The games $\boldsymbol{G}_4$ and $\boldsymbol{G}_5$ are computationally indistinguishable, assuming that the PRF is pseudo-random.*

Proof. The divergence between games $\mathbf{G}_4$ and $\mathbf{G}_5$ is strictly isolated within the generation of the string $(K^* \| \eta^*)$ for the target challenge ciphertext CT^*. In $\mathbf{G}_4$, this combined string is derived via the pseudo-random evaluation $\mathsf{PRF}(\mathsf{CK}_{OSS}^*, \mathsf{ID}^* \| \mathsf{BL}^*)$, whereas in $\mathbf{G}_5$, it is completely substituted by a truly random bitstring of the identical length.

We construct a reduction simulator $\mathcal{B}$ that leverages the indistinguishability between these games to break the pseudo-randomness of the underlying PRF primitive. The simulator $\mathcal{B}$ is given oracle access to an external oracle $\mathsf{O}(\cdot)$, which functions either as a real PRF instance $\mathsf{F}(k,\cdot)$ keyed by a hidden random key k or as a truly random function $f(\cdot)$. $\mathcal{B}$ arranges the execution environment for the adversary $\mathcal{A}$ by perfectly replicating the oracle specifications established in $\mathbf{G}_4$. Crucially, due to the transition finalized in $\mathbf{G}_3$, the capsule key CK^*_{OSS} is now distributed as a uniform random string, thereby perfectly sustaining the role of the hidden PRF key k. During the challenge phase, $\mathcal{B}$ bypasses the local computation of the PRF. Instead, it queries its external oracle $\mathsf{O}(\cdot)$ with the evaluation input $(\mathsf{ID}^*\|\mathsf{BL}^*)$ and directly assigns the returned string as the challenge component $(K^*\|\eta^*)$. If $\mathsf{O}(\cdot)=\mathsf{F}(k,\cdot)$, the overall view of $\mathcal{A}$ is identical to the environment of $\mathbf{G}_4$. Conversely, if $\mathsf{O}(\cdot)=f(\cdot)$, the view of $\mathcal{A}$ coincides with the distribution of $\mathbf{G}_5$.

Since the events BAD_1 and BAD_2 are excluded and the security of the underlying O-TBIBE-SS scheme guarantees that CK^*_{OSS} remains completely hidden from $\mathcal{A}$, the specific PRF valuation is never leaked through any other oracle queries. Consequently, the distinguishing advantage of $\mathcal{A}$ across these games is tightly bounded by the advantage of $\mathcal{B}$ against the PRF primitive as $\left|\mathsf{Adv}^{\mathbf{G}_4}_{\mathcal{A}}-\mathsf{Adv}^{\mathbf{G}_5}_{\mathcal{A}}\right|\leq \mathsf{Adv}^{\mathsf{PR}}_{PRF,\mathcal{B}}(\lambda)$. This completes the proof. □

Lemma C.5. *The games $\mathbf{G}_5$ and $\mathbf{G}_6$ are computationally indistinguishable, assuming that the SKE scheme is IND-OT secure.*

Proof. The divergence between games $\mathbf{G}_5$ and $\mathbf{G}_6$ is strictly isolated within the symmetric key encryption component CT^*_{SKE} of the target challenge ciphertext CT^*. In $\mathbf{G}_5$, CT^*_{SKE} is generated as a valid encryption of the message M^*_μ, whereas in $\mathbf{G}_6$, it is substituted by an encryption of a fixed dummy string $0^{|M^*|}$.

We construct a reduction simulator $\mathcal{B}$ that leverages the indistinguishability between these games to break the IND-OT security of the underlying SKE scheme. Interacting with the external SKE challenger, $\mathcal{B}$ arranges the execution environment for the adversary $\mathcal{A}$ by replicating the oracle specifications established in $\mathbf{G}_5$. During the challenge phase, $\mathcal{B}$ randomly samples a bit $\mu\in\{0,1\}$ and submits two challenge messages, $m_0=M^*_\mu$ and $m_1=0^{|M^*|}$, to the SKE challenger to receive the ciphertext CT^*_{SKE}. $\mathcal{B}$ then embeds this symmetric component to the VOBTE-SS challenge ciphertext CT^*, which is subsequently delivered to $\mathcal{A}$.

Since the symmetric key K^* in $\mathbf{G}_5$ is distributed as a truly random string and is never exposed in any other part of the simulation, it perfectly functions as the hidden one-time key in the IND-OT security experiment. Consequently, if the SKE challenger encrypts m_0, the overall view of $\mathcal{A}$ is identical to the environment of $\mathbf{G}_5$. Conversely, if the SKE challenger encrypts m_1, the view of $\mathcal{A}$ strictly coincides with the distribution of $\mathbf{G}_6$. In $\mathbf{G}_6$, the challenge ciphertext is rendered completely independent of the bit μ, implying that $\mathcal{A}$ retains zero advantage in guessing the bit. The advantage of $\mathcal{A}$ in distinguishing these games is thus bounded by $\left|\mathsf{Adv}^{\mathbf{G}_5}_{\mathcal{A}}-\mathsf{Adv}^{\mathbf{G}_6}_{\mathcal{A}}\right|\leq \mathsf{Adv}^{\mathsf{IND-OT}}_{SKE,\mathcal{B}}(\lambda)$. This completes the proof. □

C.2 Proof of Theorem 4.3 (Verifiability)

Proof. The proof for SE-SCM-VER security follows a sequence of hybrid games similar to the SE-SCM-CCA proof in Theorem 4.2. We focus on the transition from the ciphertext's algebraic structure to its statistical independence.

Games $\mathbf{G}_0\sim\mathbf{G}_4$. These games are defined identically to those in the proof of Theorem 4.2. Specifically, $\mathbf{G}_1$ and $\mathbf{G}_2$ handle OTS strong unforgeability and hash collisions. In $\mathbf{G}_3$, the simulator transitions to the O-TBIBE-SS security model, handling static corruption and oracles via the O-TBIBE-SS challenger. In $\mathbf{G}_4$, the challenge capsule key CK^*_{OTB} is replaced by a random value $\widetilde{\mathsf{CK}}\in\mathbb{G}_T$. The advantage

difference up to this point is

$$\left|\mathsf{Adv}^{\mathbf{G}_0}_{\mathcal{A}} - \mathsf{Adv}^{\mathbf{G}_4}_{\mathcal{A}}\right| \leq \mathsf{Adv}^{\mathsf{SUF}}_{OTS,\mathcal{B}} + \mathsf{Adv}^{\mathsf{CR}}_{HF,\mathcal{B}} + \mathsf{Adv}^{\mathsf{SE\text{-}SCM\text{-}CPA}}_{O\text{-}TBIBE\text{-}SS,\mathcal{B}}.$$

Game $\mathbf{G}_5$. In the challenge ciphertext CT^*, the simulator replaces the PRF outputs $(K^*\|\eta^*) = \mathsf{PRF}(\widetilde{\mathsf{CK}}, \mathsf{ID}^*\|\mathsf{BL}^*)$ with a random string $(\widetilde{K}\|\widetilde{\eta})$ chosen uniformly at random. Since this transition relies on the pseudo-randomness of PRF, we have $\left|\mathsf{Adv}^{\mathbf{G}_4}_{\mathcal{A}} - \mathsf{Adv}^{\mathbf{G}_5}_{\mathcal{A}}\right| \leq \mathsf{Adv}^{\mathsf{PR}}_{PRF,\mathcal{B}}(\lambda)$.

In this final game $\mathbf{G}_5$, the tag η^* (and any tag computed during the simulation) is no longer the output of a PRF but a purely random string sampled uniformly from $\{0,1\}^\lambda$. This random string is statistically independent of any information available to $\mathcal{A}$ in its view. For $\mathcal{A}$ to successfully forge a valid ciphertext, it must output a tag that matches the randomly chosen string by the simulator. Since the choice of the tag is independent of the adversary's capabilities, the probability of a successful forgery is the probability of guessing a λ-bit string $\mathsf{Adv}^{\mathbf{G}_5}_{\mathcal{A}} = \frac{1}{2^\lambda}$.

By combining the advantage differences, we obtain the following upper bound on the total advantage:

$$\begin{aligned}&\mathsf{Adv}^{\mathsf{SE\text{-}SCM\text{-}VER}}_{VOBTE\text{-}SS,\mathcal{A}}(\lambda)\\ &\leq \mathsf{Adv}^{\mathsf{SUF}}_{OTS,\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{CR}}_{HF,\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{SE\text{-}SCM\text{-}CPA}}_{O\text{-}TBIBE\text{-}SS,\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{PR}}_{PRF,\mathcal{B}}(\lambda) + \frac{1}{2^\lambda}.\end{aligned}$$

This completes the proof. □

C.3 Proof of Theorem 4.4 (Robustness)

Proof. Suppose there exists an adversary $\mathcal{A}$ that breaks the ROB-MKA security of the VOBTE-SS scheme with a non-negligible advantage. Let $\mathcal{C}$ be an O-TBIBE-SS challenger. A simulator $\mathcal{B}$ that breaks the ROB-MKA security of the O-TBIBE-SS scheme by using $\mathcal{A}$ is described as follows:

Init: $\mathcal{A}$ commits to the decryption committee size N and the threshold $T \leq N$. Then, $\mathcal{B}$ forwards (N,T) to $\mathcal{C}$.

Setup: $\mathcal{B}$ receives PP_{OSS} from $\mathcal{C}$, and invokes the $\mathsf{OInit}()$ oracle to initialize the lists L_{sk}, Q_{hon}, Q_{cor}, and Q_{mal}. Then, it selects a hash function $\mathsf{H}_s \leftarrow \mathcal{H}$ and provides $\mathsf{PP} = (\mathsf{PP}_{OSS}, \mathsf{H}_s)$ to $\mathcal{A}$.

Pre-challenge Queries: $\mathcal{B}$ handles the queries of $\mathcal{A}$ as follows:

- $\mathsf{OCreateKey}(j)$: If $j \notin [N]$ or $j \in Q_{\text{all}}$, it responds with $\perp$. Otherwise:
  1. It queries the $\mathsf{OCreateKey}(j)$ oracle of $\mathcal{C}$ and receives $(\mathsf{PK}_{j,OSS}, \mathsf{HT}_{j,OSS})$, and sets $\mathsf{PK}_j = (\mathsf{PK}_{j,OSS}, \mathsf{HT}_{j,OSS})$.
  2. It updates $L_{\text{sk}} \leftarrow L_{\text{sk}} \cup \{(j, \perp, \mathsf{PK}_j)\}$, adds j to Q_{hon}, and responds with PK_j.
- $\mathsf{OCorruptKey}(j)$: If $j \notin [N]$ or $j \notin Q_{\text{hon}}$, it responds with $\perp$. Otherwise:
  1. It queries the $\mathsf{OCorruptKey}(j)$ oracle of $\mathcal{C}$ and receives $\mathsf{SK}_{j,OSS}$, and sets $\mathsf{SK}_j = \mathsf{SK}_{j,OSS}$.
  2. It fills the vacant slot for j in L_{sk} with SK_j, moves j from Q_{hon} to Q_{cor}, and responds with SK_j.
- $\mathsf{ORegisterKey}(j, \mathsf{PK}_j)$: Let $\mathsf{PK}_j = (\mathsf{PK}_{j,OSS}, \mathsf{HT}_{j,OSS})$. If $j \notin [N]$ or $j \in Q_{\text{all}}$, it responds with $\perp$. Otherwise:
  1. It checks that $1 = \mathsf{VerifyPubKey}(\mathsf{PP}, \mathsf{PK}_j)$ and responds with 0 if the check fails.

2. It queries the $\mathsf{ORegisterKey}(j, \mathsf{PK}_{j,OSS}, \mathsf{HT}_{j,OSS})$ oracle of $\mathcal{C}$ and receives resp. If $\mathsf{resp} \neq 1$, it responds with resp.
3. It updates $L_{\mathrm{sk}} \leftarrow L_{\mathrm{sk}} \cup \{(j, \perp, \mathsf{PK}_j)\}$, adds j to Q_{mal}, and responds with 1.

Challenge: $\mathcal{A}$ submits a challenge batch label BL^*, $\mathcal{B}$ proceeds as follows:

1. It first retrieves all public keys $(\mathsf{PK}_j)_{j\in[N]}$ from L_{sk} and runs $(\mathsf{EK}, \mathsf{PK}) \leftarrow \mathsf{Preprocess}\big(\mathsf{PP}, (\mathsf{PK}_j)_{j\in[N]}\big)$.
2. It generates $(\mathsf{SK}^*_{OTS}, \mathsf{VK}^*_{OTS}) \leftarrow \mathsf{OTS.KeyGen}(1^\lambda)$ and computes $\mathsf{ID}^* = \mathsf{H}_s(\mathsf{VK}^*_{OTS})$.
3. Next, it forwards the challenge identity-label pair $(\mathsf{ID}^*, \mathsf{BL}^*)$ to $\mathcal{C}$ and receives $(\mathsf{CH}^*_{OSS}, \mathsf{CK}^*_{OSS})$.
4. It derives $(K^* \| \eta^*) \leftarrow \mathsf{PRF}(\mathsf{CK}^*_{OSS}, \mathsf{ID}^* \| \mathsf{BL}^*)$. It then selects a random message $M^* \in \mathcal{M}$, encrypts it via $\mathsf{CT}^*_{SKE} \leftarrow \mathsf{SKE.Encrypt}(K^*, M^*)$, and generates $\sigma^* \leftarrow \mathsf{OTS.Sign}(\mathsf{SK}^*_{OTS}, \mathsf{BL}^* \| \mathsf{CH}^*_{OSS} \| \mathsf{CT}^*_{SKE} \| \eta^*)$.
5. It sets $\mathsf{CT}^* = (\mathsf{VK}^*_{OTS}, \mathsf{CH}^*_{OSS}, \mathsf{CT}^*_{SKE}, \eta^*, \sigma^*)$ and gives CT^* to $\mathcal{A}$.

Post-challenge Queries: $\mathcal{B}$ handles the queries of $\mathcal{A}$ as follows:

- $\mathsf{OComputePreDecrypt}(j, (\mathsf{CT}_i)_{i\in[B]}, \mathsf{BL})$: If $j \notin Q_{\mathrm{hon}}$, it responds with $\perp$. Otherwise:
  1. It initializes $S = \emptyset$, and for each $i \in [B]$, it proceeds as follows: If the OTS signature in CT_i is invalid, it skips to the next iteration. Otherwise, it computes $\mathsf{ID}_i = \mathsf{H}_s(\mathsf{VK}_{i,OTS})$ and updates $S \leftarrow S \cup \{\mathsf{ID}_i\}$.
  2. It queries the $\mathsf{OComputeKeyShare}(j, S, \mathsf{BL})$ oracle of $\mathcal{C}$ and receives $\mathsf{DK}_{j,OSS}$.
  3. It sets $\mathsf{DK}_j = \mathsf{DK}_{j,OSS}$ and responds with DK_j.

Output: Finally, $\mathcal{A}$ outputs $\big(A^*, (\mathsf{DK}^*_j)_{j\in A^*_{\mathrm{dis}}}, (\widetilde{\mathsf{CT}}_i)_{i\in[B]}\big)$ such that $\widetilde{\mathsf{CT}}_k = \mathsf{CT}^*$ for an index $k \in [B]$. Let $\mathsf{DK}^*_j = \mathsf{DK}^*_{j,OSS}$ and $\widetilde{\mathsf{CT}}_i = (\mathsf{VK}_{i,OTS}, \mathsf{CH}_{i,OSS}, \mathsf{CT}_{i,SKE}, \eta_i, \sigma_i)$ for $i \in [B]$. Next, $\mathcal{B}$ proceeds as follows:

1. It checks that $1 = \mathsf{VerifyPreDecrypt}(\mathsf{PK}, \mathsf{DK}^*_j, (\widetilde{\mathsf{CT}}_i)_{i\in[B]}, \mathsf{BL}^*)$ for each dishonest node $j \in A^*_{\mathrm{dis}}$. If any check fails, it returns 0.
2. It initializes $S^* = \emptyset$, and for each $i \in [B]$, it proceeds as follows: If the OTS signature in $\widetilde{\mathsf{CT}}_i$ is invalid, it skips to the next iteration. Otherwise, it computes $\mathsf{ID}_i = \mathsf{H}_s(\mathsf{VK}_{i,OTS})$ and updates $S^* \leftarrow S^* \cup \{\mathsf{ID}_i\}$. Note that since $\widetilde{\mathsf{CT}}_k = \mathsf{CT}^*$, the valid signature σ^* ensures that $\mathsf{ID}^* \in S^*$ holds.
3. Finally, it outputs $(A^*, (\mathsf{DK}^*_{j,OSS})_{j\in A^*_{\mathrm{dis}}}, S^*)$ to $\mathcal{C}$ as its own forged robustness instance.

Analysis of Simulation Perfectness. We first analyze that the simulation executed by $\mathcal{B}$ is perfectly identical to the actual ROB-MKA security game from the view of $\mathcal{A}$. The public parameters $\mathsf{PP} = (\mathsf{PP}_{OSS}, \mathsf{H}_s)$ provided to $\mathcal{A}$ are distributed identically to the real setup since PP_{OSS} is generated by the O-TBIBE-SS challenger and H_s is uniformly sampled from the hash family. All pre-challenge and post-challenge oracle queries are handled perfectly without any difference: $\mathsf{OCreateKey}$ and $\mathsf{OCorruptKey}$ queries directly preserve the underlying distributions through $\mathcal{C}$, and $\mathsf{ORegisterKey}$ strictly replicates the verification by combining $\mathsf{VerifyPubKey}$ with the underlying registration oracle. In the challenge phase, the target ciphertext CT^* is well-formed, where the OTS signature σ^* and the SKE encryption CT^*_{SKE} are computed using independent randomness. Since the internal lists $(Q_{\mathrm{hon}}, Q_{\mathrm{cor}}, Q_{\mathrm{mal}})$ maintained by $\mathcal{B}$ precisely track the adversarial operations, the simulation of $\mathcal{B}$ is indistinguishable from the real ROB-MKA experiment.

Analysis of Robustness Forgery. We next show that a successful robustness attack by $\mathcal{A}$ in the VOBTE-SS game implies a successful robustness attack by $\mathcal{B}$ in the underlying O-TBIBE-SS game. Suppose $\mathcal{A}$ wins the game by outputting $(A^*, (\mathsf{DK}_j^*)_{j\in A_{\mathrm{dis}}^*}, (\widetilde{\mathsf{CT}}_i)_{i\in[B]})$. By the winning conditions of the VOBTE-SS robustness game, it is guaranteed that $A_{\mathrm{dis}}^* \neq \emptyset$, all dishonest pre-decryption keys pass the verification, and the decrypted message satisfies $\widetilde{M}_k \neq M^*$ for the target index k. Upon receiving the outputs of $\mathcal{A}$, $\mathcal{B}$ reconstructs the identity set S^* by executing the OTS verification. Because $\widetilde{\mathsf{CT}}_k = \mathsf{CT}^*$ possesses a valid OTS signature σ^*, the challenge identity $\mathsf{ID}^* = \mathsf{H}_s(\mathsf{VK}_{OTS}^*)$ is included in S^*, satisfying $\mathsf{ID}^* \in S^*$. To formally bridge the message difference to the capsule key difference, we exploit the deterministic property of the batch decryption process. The SKE decryption of the VOBTE-SS primitive ensures that $\widetilde{M}_k \leftarrow \mathsf{SKE.Decrypt}(K', \mathsf{CT}_{SKE}^*)$, where the key K' is derived via $(K'\|\eta') \leftarrow \mathsf{PRF}(\mathsf{CK}'_{OSS}, \mathsf{ID}^*\|\mathsf{BL}^*)$ from the recovered capsule key CK'_{OSS}. Conversely, the challenge message satisfies $M^* = \mathsf{SKE.Decrypt}(K^*, \mathsf{CT}_{SKE}^*)$ with $(K^*\|\eta^*) \leftarrow \mathsf{PRF}(\mathsf{CK}_{OSS}^*, \mathsf{ID}^*\|\mathsf{BL}^*)$. By the correctness of the SKE scheme, the message difference $(\widetilde{M}_k \neq M^*)$ implies the symmetric key difference $(K' \neq K^*)$. Since the inputs $\mathsf{ID}^*\|\mathsf{BL}^*$ of the PRF are identically fixed, the symmetric key difference $(K' \neq K^*)$ logically demands that the recovered capsule key CK' must deviate from the challenge capsule key CK_{OSS}^*, establishing the capsule key difference $(\mathsf{CK}' \neq \mathsf{CK}_{OSS}^*)$. Therefore, the forged instance $(A^*, (\mathsf{DK}_{j,OSS}^*)_{j\in A_{\mathrm{dis}}^*}, S^*)$ submitted by $\mathcal{B}$ satisfies the winning conditions of the O-TBIBE-SS robustness game. This completes the proof. □